

第一部分 经典密码学介绍

第1章 概 论

1.1 密码学和现代密码学

《牛津简明词典》(2006 版)定义密码学为:编码或者解码的艺术。这个定义可能从历史角度看是准确的,但是它并没有抓住现代密码学的本质。首先,它仅仅关注秘密通信问题。理由是:定义中明确说明了“编码(code)”,将其定义为“一种预先安排好的信号系统,专用于确保传输消息时的保密性”。其次,该定义认为密码学是一种艺术。实际上,直到 20 世纪(或可认为直到 20 世纪末),密码学都是一种艺术。不论是构造好的编码,还是破解存在的编码,都依赖于创造性和个人技巧。只有非常少的理论可以依靠,甚至没有一个清晰的定义来说明什么构成好的编码。

到 20 世纪末,密码学的面貌从根本上得到了改变。出现了丰富的理论,于是可以将密码学作为一种科学来严格地学习。而且,当前密码学的领域拥有比秘密通信多得多的内容。它解决很多问题,如消息鉴别、数字签名、密钥交换协议、认证协议、电子拍卖和选举、数字现金等。实际上,现代密码学关注任何分布式计算中产生的,来源于内部和外部攻击的问题。这里不试图给出对现代密码学的完美定义,我们认为它是:对安全地进行数字信息和事务处理、分布式计算所需技术的科学研究。

另外,经典密码学(上世纪 80 年代以前)和现代密码学之间一个非常重要的区别是使用者。历史上,密码学的主要消费者是军事和智囊机构。但是现在,密码学无处不在,依靠密码学的安全机制几乎是计算机系统的一个集成部件。用户在访问一个安全的站点时(常常没有意识到)依靠了密码学。在多用户操作系统中,密码学方法确保了访问控制,并阻止窃贼从丢失的笔记本计算机中窃取商业机密。软件保护方法使用加密、认证和其他工具来阻止复制。我们可以继续举出很多这样的例子。

简言之,密码学从一种军事中处理秘密通信的艺术变成一种使全世界普通人的系统变得安全的科学。这意味着密码学成为计算机科学中一个越来越核心的主题。

本书关注的是“现代密码学”。但是将从密码学的上述变化之前开始学习,以便读者更容易进入主题,并了解密码学的来源,以及加深对改变程度的认识。经典密码学充满了临时的编码构造方法和相对简单的破解方式,这也是本书中采用严格方法的主要原因^①。

^① 这是我们编写该内容的主要目的,本章不应作为具有代表性的密码学历史报告。对密码学的历史感兴趣的读者请查阅章后所附参考文献。

1.2 对称密钥加密的基本设置

如上所述，在历史上密码学主要用于秘密通信。特别是使用密码学的加密术（现在叫做加密方案），使事先共享某种信息的两方能够秘密通信。该方式中，通信双方实现共享某种秘密信息，现在称为秘密密钥（**private-key**）（或对称密钥（**symmetric-key**））。在描述历史加密术之前，我们从通用术语的角度讨论秘密密钥情形和加密。

在对称密钥加密的情形中，双方共享一些密码信息，称为密钥（**key**），当他们希望安全地通信时，使用这个密钥。发送方使用密钥加密（或者“混合”）消息，接收方收到后使用相同的密钥解密（或者“去混合”）来恢复消息。消息本身称为明文（**plaintext**），从发送方传输到接收方的经加密的消息称为密文（**ciphertext**），如图 1.1 所示。共享的密钥用来从所有窃听者中区分通信方（假定通过公共信道传输）。

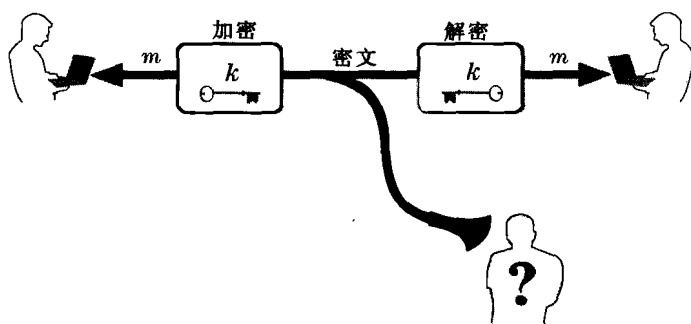


图 1.1 对称密钥加密的基本情形

在这一方式中，在明文和密文间变换时使用同一个密钥。这解释了为什么这一情形也被称为对称密钥（**symmetric-key**），这里对称是指双方拥有相同的密钥来加密和解密。这和非对称（**asymmetric**）加密（在第 9 章中介绍）相对应，后者的发送和接收方并不共享任何秘密并且有不同的密钥用于加密和解密。对称密钥是经典的，通过本章后面的内容将会了解到这一点。

在任何使用对称密钥加密的系统中，都隐含着假设：通信方可以使用某种秘密的方式建立初始的共享密钥（注意，如果一方通过公共信道简单地发送密钥给另一方，窃听者将也能获得密钥）。在军事领域中，这不是一个严重的问题，因为通信方能够在一个安全的位置实现物理上的相遇以认同一个密钥。但是当今在很多情况下，不可能安排通信方的碰面。如将在第 9 章中所述，这一点非常值得关注，并且实际上限制了仅依靠对称密钥的密码学系统的可应用性。尽管如此，对称密钥方法仍在很多情形下是够用的，且被广泛使用。例如：磁盘加密中同一个用户（在不同的时间点）使用固定的密钥来读写磁盘。正如将在第 10 章进一步讨论的，对称密钥加密也普遍和非对称密钥密码学联合使用。

加密的语法。对称密钥加密方案由三个算法组成：密钥产生，加密，解密。它们的功能如下：

(1) 密钥产生算法 **Gen** 是一个概率算法，能够根据方案定义的某种分布选择并输出一个密钥 k 。

(2) 加密算法 Enc ，输入为密钥 k 和明文 m ，输出为密文 c 。把使用密钥 k 加密明文 m 记为 $\text{Enc}_k(m)$ 。

(3) 解密算法 Dec ，输入为密钥 k 和密文 c ，输出为明文 m 。把使用密钥 k 解密密文记为 $\text{Dec}_k(c)$ 。

· 密钥产生函数输出的所有可能的密钥称为密钥空间，用 $\mathcal{K}$ 表示。通常， Gen 简单地从密钥空间中均匀随机地选择一个密钥（事实上，这种假设不失一般性）。所有“合法”消息（也就是那些被加密算法支持的消息）的集合记做 $\mathcal{M}$ ，并称为明文（或消息）空间。因为任何密文是通过使用密钥加密明文来获得的，集合 $\mathcal{K}$ 和 $\mathcal{M}$ 一起定义了所有可能密文的集合 $\mathcal{C}$ 。一个加密方案可通过明确三个算法（ Gen ， Enc ， Dec ）和明文空间 $\mathcal{M}$ 来完全定义。

对任意加密方案的基本要求是：对于任意通过 Gen 输出的密钥 k ，每个明文消息 $m \in \mathcal{M}$ ，满足

$$\text{Dec}_k(\text{Enc}_k(m)) = m$$

通过文字说明，解密一个密文（使用相应密钥）生成原来加密的原始消息。

回顾前面的讨论，一个加密方案可被通信双方以如下方式所使用：首先， Gen 被运行获得各方共享的密钥 k 。当一方需要发送明文 m 到另一方，他首先计算 $c := \text{Enc}_k(m)$ ，然后通过公共信道发送密文 c 到另一方。另一方接收到 c 后，计算 $m := \text{Dec}_k(c)$ 恢复出原来的消息^②。

密钥和 Kerckhoffs 原则。从前面所述可知，如果窃听敌手知道算法 Dec 以及通信双方共享的密钥 k ，则敌手可以解密他们间的所有通信。正因如此，通信双方必须安全地共享密钥 k ，并且保持 k 完全保密。但是，是否需要保持解密算法 Dec 为一个秘密？是不是所有组成加密方案的算法（即 Gen 和 Enc ）都需要保密？（注意通常认为明文空间 $\mathcal{M}$ 是已知的，例如，可能由英文句子组成。）

在 19 世纪后期，Auguste Kerckhoffs 在一篇论文中对这一问题给出了意见，他发表了军用重要加密术的设计原则。其中最重要的（现在称作 Kerckhoffs 原则）是：加密方法不必保密，它可以被敌人轻易获得。换句话说，加密方案本身不必保密，唯一需要保密的是通信双方共享的秘密密钥。

Kerckhoffs 的意图是：加密方案应该设计成只要敌手不知道使用的密钥，即使知道方案的组成算法细节，方案依然是安全的。换句话说，Kerckhoff 原则要求安全性仅依靠于密钥的安全性。

接受 Kerckhoffs 原则有三个主要的理由。①与维护算法的保密性相比，通信各方非常容易维护短小密钥的保密性。共享一个短（如 100bit）串并安全地保存这个串，与共享和安全地保存一个长度大数千倍的程序相比，要容易得多。而且，算法的细节能够泄漏（可能被内部人员）或者通过逆向工程学习到；而当秘密信息是从随机产生的串中选取时，这些是不可能的。②万一密钥暴露了，诚实参与方将非常容易改变密钥而不是替换算法。实际上，良好的安全实践会要求：即使密钥没有暴露，也要经常刷新（替换）密钥。而替换软件程序则麻烦得多。③万一有多对人员（如同一个公司中）需要加密他们的通信，对所有参与方而言，使用同样的算法（或者程序）和不同的密钥，与使用不同的（取决于通信方的）程序相比要容易得多。

现在，Kerckhoffs 原则被理解为：提倡安全性不能建立在对算法的保密上，更提倡要公开

② 全书将使用“:=”表示赋值操作。常用符号的列表见书尾。

这些算法。这全然站在了“通过隐晦产生安全”的对立面，“隐晦产生安全”的思想是通过隐藏密码学算法来提高安全性。公开算法设计说明的“开放密码学设计”的优势在于：

(1) 公开的设计承受公开的钻研和分析，因此可能更加健壮。多年的经验表明：构造良好的密码学方案是非常困难的。因此，如果方案被广泛地（被方案的设计者之外的专家们）研究过，且没有发现弱点，就更相信方案的安全性。

(2) 安全缺陷，如果存在的话，被“正义黑客”所发现（希望能导致系统的修改），而不是被敌对方所发现会更好。

(3) 系统的安全取决于算法的保密性，代码的逆向（或者被工业间谍泄漏）导致严重的安全威胁。相比之下，安全密钥不是代码的一部分，所以不存在逆向工程的弱点。

(4) 公开设计使标准的建立成为可能。

听上去简单明了，但开放的密码学设计这一原则（即 Kerckhoffs 原则）被一次次忽视，导致了灾难性的后果。使用专利算法（即公司密码设计的非标准算法）是非常危险的，只有公开试用和测试的算法才能使用。值得庆幸的是，很多优秀的算法已经标准化，并不受专利保护，所以没有理由到现在还使用其他算法。

攻击场景。下面简要介绍一些基本的攻击类型来归纳加密方案。根据严重程度，包括以下几种攻击类型。

(1) 唯密文攻击 (Ciphertext-only attack)：这是最基本的攻击方式，表示敌手只能观察到密文（或者多个密文），并且试图确定相应的明文（或者多个明文）。

(2) 已知明文攻击 (Known-plaintext attack)：这里，敌手学习一个或者多个使用相同密钥加密的明文/密文对。目标是确定其他密文对应的明文（这里“其他”是指事先不知道明文的密文）。

(3) 选择明文攻击 (Chosen-plaintext attack)：这种攻击中，敌手能够选择明文，并得到相应密文。试图确定其他密文对应的明文。

(4) 选择密文攻击 (Chosen-ciphertext attack)：最后一种攻击类型是敌手甚至可以选择密文并得到相应的明文。敌手的目的依然是确定其他密文的明文（其明文不能直接得到）。

前两种攻击类型是被动的，敌手只是获得一些密文（有可能包括相应明文）并且发起攻击。相对而言，后两种攻击是主动的，敌手能够适应性地有选择地请求加密和解密。

上述前两种攻击明显是符合实际的。唯密文攻击在实践中最容易实现，敌手唯一需要做的事就是偷听传输密文的公共通信线路。在已知明文攻击中，假定敌手获得所见密文对应的明文。这是符合实际的，因为不是所有加密的消息都是保密的，至少不是无限期的保密。举一个简单的例子，当开始通信时，通信两方可能通常加密“hello”消息。一个更复杂的例子，加密可用来保证季度盈利的机密直到公开的那一天。该例中，任何偷听并获得密文的人，将在后来获得相应的明文。因此，任何合理的加密方案必须对发起已知明文攻击的敌手保持安全。

后两种主动攻击看上去有点奇怪并需要论证。（什么时候通信方会加密和解密敌手所需要的信息？）详细的讨论将放在对该类攻击进行形式化定义的时候：选择明文攻击在 3.5 节讨论，选择密文攻击在 3.7 节讨论。

不同的加密应用可能需要加密方案能够对付不同的攻击类型。通常并不要求必须使用可抵御“最强”攻击的加密方案，因为它们可能没有可抵御“弱一点”攻击的加密方案效率高。因此，如果后者足够满足应用的要求，则倾向于使用后者。

1.3 古典加密术及其密码分析

在学习“经典密码学”过程中，将回顾一些古典加密术并说明它们是完全不安全的。正如前面提及的，本节的主要目标是：①突出密码学中“临时”方法的弱点，并因而引出后续讨论的现代的严格方法；②演示企图用“简单的方法”达到安全加密是不可能成功的，并给出其原因。过程中将展示一些从这些弱点中学到的密码学的核心原则。

本节（且仅限本节），明文字符使用小写字母表示，密文字符使用大写字母表示。当描述攻击时，常使用 Kerckhoff 原则并假定方案被敌手所知（但是密钥不被敌手所知）。

恺撒（Caesar）加密。有记载的最古老的加密方法是恺撒加密，约公元 110 年在“De Vita Caesarum, Divus Iulius”（“The Lives of the Caesars, The Deified Julius”）中记载：

在他给 Cicero 的信中，以及给他的密友关于私人事务的信中，特别是后者，如果他有什么需要保密的话，他就用加密术来写，即通过改变字母表中字母的次序，使得没有一个词可以读得出来。如果任何人想解密，得到其中的涵义，他必须替换字母表中的第 4 个字母，即用 D 替换 A，依此类推。

也就是说，Julius Caesar 通过移动三个位置的字母进行加密：a 被 D 替换，b 被 E 替换，以此类推。当然，在字母表的最后，字母折返回来，即 x 被 A 替换，y 被 B 替换，z 被 C 替换。举例来说，短消息“begin the attack now”，去掉空格，将被加密成为：

EHJLQWKHDWWDFNQRZ

使得它不可读。

这种加密方法的一个问题是：方法是固定的。因而任何人学习到恺撒是如何加密的，便可以轻易地解密。如果用前面描述的加密语法来解释恺撒加密，密钥生成算法Gen过于简单（也就是什么都没做），即没有密钥。

有趣的是，这种加密方法的变种 ROT-13（移动 13 个位置，而不是 3 个）现在被广泛使用在很多在线论坛中。可以理解这并不提供任何密码学安全，ROT-13 仅仅用在保证文本（比如影评）不可读，除非读者有意识地试图解密。

移位加密以及充分密钥空间原则。恺撒加密的问题是加密方式总是一样的，且没有密钥。移位加密类似于恺撒加密，但是引入了密钥^③。特别是在移位加密中，密钥 k 是一个 0 到 25 之间的数。那么加密时，字母移动 k 个位置，映射到加密语法中，意味着算法Gen输出一个在 $\{0, \dots, 25\}$ 中的随机数 k ；算法Enc使用密钥 k 将英文字母组成的明文的每个字母向前移动 k 个位置（在 z 处折回到 a）；算法Dec使用 k 将密文的每个字母向后移动 k 个位置（在 a 处折回到 z）。明文消息空间 $\mathcal{M}$ 定义为所有由英语字母表中字符组成的有限长度字符串（该方案中不含数字、标点，或者其他字符）。

若视字母表为数字 $\{0, \dots, 25\}$ （而不是英文字符），则可以得到一个更数学化的描述。首先，介绍符号：若 a 是一个整数且 N 是一个大于 1 的整数，定义 $[a \bmod N]$ 为 a 被 N 除后的余数。注意 $[a \bmod N]$ 是一个 0 到 $N - 1$ 闭区间的整数。我们称映射 a 到 $[a \bmod N]$ 为模 N 消减，将在第 7 章具体介绍。

使用这种符号，用密钥 k 来加密明文字符 m_i ，得到密文字符 $[(m_i + k) \bmod 26]$ ，解密密

^③ 在有些书中，“移位加密”和“恺撒加密”可以互换。

文字符 c_i ，则为 $[(c_i - k) \bmod 26]$ 。从这一视角，消息空间 $\mathcal{M}$ 定义为任何有限长度整数序列，整数的范围为 $\{0, \dots, 25\}$ 。

移位加密安全吗？在继续阅读之前，设法解密下面使用密钥 k 进行移位加密的密文（不破解密钥的值）：

OVDTHUFWVZZPISLRLFZHYLAOLYL

在不知道 k 的情况下能解密该消息吗？实际上，由于只有 26 个可能的密钥，因而非常容易尝试每个密钥，并观察哪个密钥解密密文后得到的明文“有意义”。这种对加密方案的攻击称为蛮力（brute-force）攻击或者穷举（exhaustive）攻击。很明显任何安全的加密方案必须能够抵御这种蛮力攻击；否则将被完全攻破，无论加密算法有多复杂。于是得到一个简单而重要的原则，叫做“密钥空间充分性原则”：

任何安全的加密方案必须拥有一个能够抵御穷举搜索的密钥空间^④。

在当前这个年代，穷举搜索可能使用非常强大的计算机，或者使用分布在世界各地的上千台 PC 机。因而可能的密钥数量必须非常多（至少 2^{60} 或者 2^{70} ）。

强调上述原则给出的是安全的必要条件，不是充分条件。后面将看到加密方案虽有非常大的密钥空间，但仍不安全。

单字母替换（substitution）。移位加密映射每个明文字符到一个不同的密文字符，但是在每种情况下映射有相同的移位（该值由密钥决定）。单字母替换的思想是映射每个明文字符到一个不同的密文字符，以一种可任意确定的方式，只要映射是一一对应的以确保能够解密。如果使用英文字母表，密钥空间则由多个字母的置换组成，意味着密钥空间的规模为 $26! = 26 \cdot 25 \cdot 24 \cdots 2 \cdot 1$ （或大约为 2^{88} ）。例如，密钥

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
X	E	U	A	D	N	B	K	V	M	R	O	C	Q	F	S	Y	H	W	G	L	Z	I	J	P	T

其中 **a** 映射到 **X**，等等，加密消息 **tellhimaboutme** 为 **GDOOKVCXEFLGCD**。针对密钥空间的蛮力攻击，即使使用现在已知的最强大的计算机，也需要比人的一生长得多的时间。但是，这并不是说这种加密方法是安全的。事实上，将会看到，这种加密方法容易攻破，即使它拥有非常大的密钥空间。

假定被加密的是英语文本（即文本是语法正确的书面英语，不仅是由字母表的字符构成的文本）。一种可能的对单字母替换加密的攻击是使用英语的统计模式（当然，这种攻击适用于任何语言）。这种加密方法的两种特性将会在攻击中被利用：

(1) 这种加密方法中，每个字符的映射是固定的，因此如果 **e** 映射到 **D**，那么每次 **e** 在明文出现的时候，都会导致 **D** 在密文中出现。

(2) 英语（或者其他语言）中单个字母的概率分布是已知的。也就是说，不同英文字母在不同文本中的平均出现频率通常一样。当然，文本越长，频率计算越接近平均值。但是，即使是相对较短的文本（仅有几十个字）已经足够接近平均值的分布。

该攻击将密文的概率分布进行列表，然后与已知的英语字母的概率分布（见图 1.2）进行比较。得到的概率分布是密文中每个字母的频率（即表格记录 **A** 出现 4 次，**B** 出现 11 次

^④ 实际上该原则仅在消息空间大于密钥空间时正确（参见第 2 章的一个例子，它使用小的密钥空间获得安全性，只要消息空间更小）。实践中，当非常长的消息被同样的密钥加密时，密钥空间必须不怕穷举搜索。

等)。于是，可以根据频率计数来初步猜测映射关系。例如，既然 e 是英语中使用最频繁的字母，将猜测密文中最频繁的字母对应的明文为 e，如此类推。除非密文非常长，某些猜测可能错误。但是，即使对非常短的密文而言，猜测也可能快速解密（特别是使用英语语言的其他知识，如字母 t 和 e，可能出现的字母是 h，以及 u 通常跟在 q 后面）。

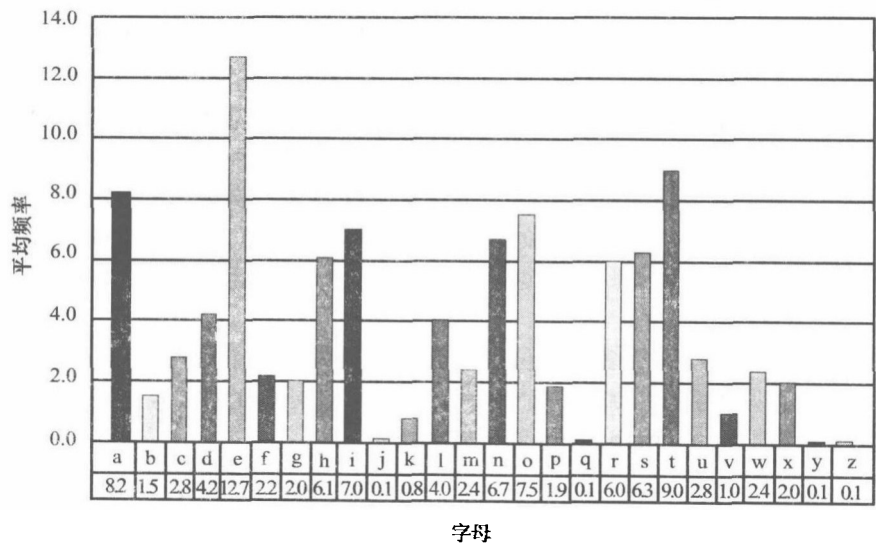


图 1.2 英文文本中字母的平均频率

实际上，单字母替换加密会很快被攻破，这并不令人惊讶，既然基于这种加密术的谜题出现在报纸上（且被人们在早餐咖啡前解决）。建议读者尝试解密下面的消息，这将帮助读者相信实施攻击是多么容易（当然，应该借助于图 1.2 的帮助）：

JGRMQOYGHMVBWJWRWQFPWEGFFDQGFPRKBEEBJZQQOCIBZKLFAGQVFZF
WWEOWOPFGFWOLPHLRLOLFDMFGQWBLWBWQOLKFWBYLBLYLFSFTJGR
MQBOLWJVFPFWQVHQWFFPQOQVFPQOCFPOGFWFJIGFQVHLHLROQVFGWJVF
PFOLFHGQVQVFILEOGQILHQFQGIQVVOVSFAFGBWQVHQWIJVMJVFPFWHGFIV
H22RAGBABHZQOCGFHX

结论是：虽然单字母加密有非常大的密钥空间，但它仍然是完全不安全的。

一种对移位加密改进的攻击。可以使用字母频率表改进对移位加密的攻击。特别是前面的攻击需要使用每个可能的密钥解密密文，然后查看哪个密钥给出的明文结果是“有意义的”。这种方法的缺点是很难自动进行，因为对计算机而言查看明文是否“有意义”比较困难。（并不是说不可能，这个当然可以通过包含有效英文单词的字典来查看，这里只是说这种查看是不简单的。）并且，有些情况（后面将给出一种）明文字符符合英文文本的分布规律，但是明文本身不是有效的英文文本，使得问题变得更加困难。

同前所述，用 $0, \dots, 25$ 表示英文字母。令 p_i ($0 < i \leq 25$) 表示在普通英文文本中第 i 个字母的概率。对已知的值 p_i 容易计算

$$\sum_{i=0}^{25} p_i^2 \approx 0.065 \tag{1.1}$$

现在，给定一些密文并令 q_i 表示第 i 个字符在密文中的概率（ q_i 是第 i 个字符出现的次数除以密文的长度）。如果密钥是 k ，那么期望对于每个 i ， q_{i+k} 约等于 p_i 。（使用 $i+k$ 替代烦琐的 $[i+k \bmod 26]$ 。）对应地，如果对每个 $j \in \{0, \dots, 25\}$ 计算

$$I_j \stackrel{\text{def}}{=} \sum_{i=0}^{25} p_i \cdot q_{i+j}$$

那么，希望发现 $I_k \approx 0.065$ ，这里 k 是密钥（然而 I_j ， $j \neq k$ 则不为 0.065）。这导致密钥恢复（key-recovery）攻击非常容易自动进行：对于所有 j 计算 I_j ，并输出所有 I_k 接近 0.065 的 k 。

Vigenère（多字母移位）加密。正如前面描述的，对单字母替换密码进行统计攻击是因为每个字符的映射是固定的。因而，消除这种攻击可通过将相同的明文字符映射到不同的密文字符。这拥有将密文中字符的概率“均匀平滑化”的效果。例如，e 有时候映射到 G，有时候映射到 P，有时候到 Y。这样，密文字符 G、P 和 Y 将很可能没有突出的出现频率，因为其他出现频率较小的字符可能映射到这些字符。因而，计算字符的频率将不提供关于映射的信息。

Vigenere 加密通过依次运用多个移位加密。也就是说，一个短的密码字被选作密钥，明文的加密是通过每个明文字符“加”每个密钥的字符（如同移位加密），必要的时候折回。例如，使用密钥cafe，消息tellhimaboutme的密文如下：

明文: tellhimaboutme
密钥: cafecafecafeca
密文: WFRQKJSFEPAYPF

（密钥没必要为一个英文单词）。这其实和移位加密一样，加密第一个、第五个、第九个等字符和密钥 $k=3$ 的移位加密情况是一样的。加密第二个、第六个、第十个等字符和密钥 $k=1$ 时一样。加密第三个、第七个等和 $k=6$ 一样，加密第四个、第八个等和 $k=5$ 一样。因此，只是多次使用不同密钥的移位加密。注意在上例中l一次映射到R，一次映射到Q。而且，密文字符F有时候从e获得，有时候从a获得。因此，密文中字符的频率“平滑了”，正是需要的。

如果密钥是充分长的字（随机选择），则破解这种加密方法看上去很困难。其实，它被很多人认为是不可攻破的加密方法，虽然在 16 世纪发明，但对该方法的攻击却在几百年后才发现。

破解 Vigenère 加密。攻击的第一个观察是如果密钥的长度已知，则破解任务相对容易。特别地，如果长度为 t （有时候叫周期），则密文能被分成 t 个部分，每个部分被视为使用单个移位加密。也就是说，令 $k = k_1, \dots, k_t$ 是密钥（每个 k_i 是字母表中的一个字母），令 $c_1, c_2, \dots$ 是密文字符。于是，对任意 j （ $1 \leq j \leq t$ ），字符集合

$$c_j, c_{j+t}, c_{j+2t}, \dots$$

这些都是被密钥为 k_j 的移位加密方法来加密的。所以，剩下的工作就是对于每个 j ，决定 26 个可能的密钥中哪个是正确的密钥。这并不像移位加密的情况那样简单，因为不再可能通过解密“有意义”来猜测密钥的单个字符。而且，同时检查所有 j 将需要蛮力搜索 26^t 个可能的密钥（当 t 大于诸如 15 时，是计算不可行的）。尽管如此，还是可以使用前面描述的统计方法。即对于与给定密钥（即每个 j 值）相关联的密文字符的每个集合，可以将每个密文字符的

频率进行制表，然后检查26个可能的移位中哪一个产生“正确的”概率分布。既然这一方法对于每个密钥能够分别进行，所以攻击执行得非常快；所有要做的只是建立 t 频率表（一张表对应一个字符集合）以及与实际的概率分布进行比较。

一个某种程度上更简单的方法是使用改进的方法攻击移位加密。重提一下这种改进的攻击不依赖于检查明文是否“有意义”，而是仅仅依赖于明文中字符的概率分布。

当密钥长度已知时，上面的任一方法都能给出成功的攻击。剩下的问题是如何确定密钥的长度。

Kasiski 的方法，在 19 世纪中发现的一种解决这个问题方法。第一步是发现密文中长度为 2 或者 3 的重复模式。这些可能是某种英文中出现非常频繁的 2 字词或者 3 字词。例如，考虑词“the”在英文文本中出现非常频繁。很明显，“the”将映射不同的密文字符，取决于在文本中的位置。但是，如果在同样的相对位置出现两次，则它将映射到密文中的同一个字符。例如，如果出现在位置 $t+j$ 和 $2t+i$ ($i \neq j$)，则每次它将映射到不同字符。但是，如果出现在位置 $t+j$ 和 $2t+j$ ，则它将映射到相同的密文字符。在一个足够长的文本中，“the”很可能映射到相同密文字符。

考虑下面的实例，用密钥beads（加空格是为了清楚）：

明文: the man and the woman retrieved the letter from the post office
密钥: bea dsb ead sbe adsbe adsbeadsb ean sdeads bead sbe adsb eadbea
密文: VMF QTP FOH MJJ XSFCS SIMTNFZXF YIS EIYUIK HWPQ MJJ QSLV TGJKGF

单词the有时映射到VMF，有时映射到MJJ，有时是YIS。但是，它两次映射到MJJ，在足够长的文本中，它可能多次映射到每种可能的结果。Kasiski 观察到这些多次出现的距离（除了巧合情况）是周期长度的倍数。（在上例中，周期长度是 5，而且两次MJJ出现的距离是 40，它是周期长度的 8 倍。）因此，所有距离的最大公约数即为周期 t 或者是 t 的倍数。

另一种可选的方法称为巧合指数方法，更像是一种算法，所以更容易自动化。重提一下，如果密钥长度是 t ，则密文字符为：

$$c_1, c_{1+t}, c_{1+2t}, \dots$$

是使用同样的移位加密。这意味着除了某个移位次序，在这个序列中字符的频率和标准的英文文本中字符的频率应该是一样的。更详细地说，令 q_i 表示第 i 个英文字符在上述序列中出现的频率（第 i 个字符的出现次数被序列中的总字符数除）。如果移动是 k_1 （即密钥中的第一个字符），则期望 q_{i+k_1} 约等于 p_i （对于所有的 i ），这里 p_i 是标准英文文本中的第 i 个字符的频率。但是，这意味着序列 $p_0, \dots, p_{25}$ 即为 $q_0, \dots, q_{25}$ 移动 k_1 个位置。因此，希望（见等式(1.1)）：

$$\sum_{i=0}^{25} q_i^2 = \sum_{i=0}^{25} p_i^2 \approx 0.0065$$

这导致一个更好的方法确定密钥的长度 t 。对于 $\tau = 1, 2, \dots$ ，观察密文字符序列 $c_1, c_{1+\tau}, c_{1+2\tau}, \dots$ ，并且对这个序列计算 $q_0, \dots, q_{25}$ 。于是计算

$$S_\tau \stackrel{\text{def}}{=} \sum_{i=0}^{25} q_i^2$$

当 $\tau = t$ 时，我们希望如上所讨论的能看到 $S_\tau \approx 0.065$ 。另一方面，对于 $\tau \neq t$ ，希望（粗略地说）在序列 $c_1, c_{1+\tau}, c_{1+2\tau}, \dots$ 中所有字符将大约以等概率出现，所以希望对于所有的 i ，

$q_i \approx 1/26$ 。在这种情况下，得到

$$S_r \approx \sum_{i=0}^{25} \left(\frac{1}{26} \right)^2 \approx 0.038$$

这与 0.065 有足够的差异。

密文长度和密码分析攻击。在上面对 **Vigenère** 的攻击中，需要比前面方案更长的密文。例如，如果使用 **Kasiski** 方法，需要更长的密文来确定周期。而且，统计需要对 t 个不同的密文部分进行，消息的频率表随着其长度的增长收敛到一个平均值（而且，密文因此需要近似 t 倍长于单字母替换加密的情况）。类似地，攻击单字母替换加密方法与攻击移位密码的方法（可攻击仅仅由单个单词组成的消息）相比，需要更长的密文。这种现象不是偶然的，它和加密方案中密钥空间的大小相关。

唯密文攻击与已知明文攻击的比较。上面描述的攻击都是唯密文攻击（这是实践中最简单实施的攻击方式）。如果敌手能够执行已知明文攻击，则所有以上加密方法都会攻破；举例留作练习。

结论和讨论。我们仅仅表述了少量的历史上的加密方法。除了了解历史以外，我们的目的是列举一些重要的关于密码设计的经验教训。简要地说，这些经验是：

(1) 充分密钥空间原则：假设加密充分长的消息，一个安全的加密方案必须有不能在合理时间内被穷举的密钥空间。但是，一个大的密钥空间并不就意味着安全（例如，单字母替换密码有大的密钥空间但仍很容易破解）。因此，大的密钥空间只是必要条件，而不是充分条件。

(2) 设计安全的加密方法是一个困难的任务：**Vigenère** 加密保持很长一段时间未被攻破，部分归因于它假定的复杂性。更多的复杂方案被使用，例如德国的 **Enigma**。但是，这种复杂性并不意味着安全，而且所有历史上的加密术都能被完全攻破。通常，设计安全的加密方案是困难的，而且这种设计要留给专家。

经典加密方案是有趣的，无论是方法，还是对密码学和密码分析学对世界历史（例如，第一、二次世界大战）的影响。这里，仅仅试图给出一些基本方法的介绍，关注的是现代密码学能从这些方法中学到什么。

1.4 现代密码学的基本原则

前面一节给出古典密码学的一些介绍。公平地说，从历史角度看密码学更多的是一门艺术，而不是科学：方案以一种临时的方式设计，然后根据臆断的复杂性或者聪明才智来评价。不幸的是，正如已经看到的，所有这些方案（无论多么聪明）最终都被破解了。

现代密码学，基于更严格和更科学的基础，给出打破“构建方案，攻破方案”这一循环反复怪圈的希望。本节描述那些能够将现代密码学从经典密码学区分开来的主要原则和范例。这里明确提出三个主要原则：

(1) 原则 1：解决任何密码学问题的第一步是公式化的表述严格且精确的安全定义。

(2) 原则 2：当密码学构造方案的安全性依赖于某个未被证明的假设时，这种假设必须精确地陈述，而且，所假设的要尽可能地少。

(3) 原则 3：密码学构造方案应当伴随有严格的安全证明，跟随符合原则 1 的安全定义，以及与原则 2 陈述的假设有关（如果假设是需要的）。

我们现在更深入地讨论上述每个原则。

1.4.1 原则 1——形成精确的定义

现代密码学的一个关键的贡献在于：形式化的安全定义是设计、使用或者研究任何密码学原语或者协议的基本先决条件。下面依次解释这三个方面。

(1) 对设计的重要性：假定对构造安全的加密方案感兴趣。如果没有完全清晰地理解想达到什么，怎么可能知道是否（或何时）达到。在头脑中拥有精确的定义使得更好地定位设计的努力方向，以及更好地评价构造方案的质量，从而改进最终的构造方案。特别需要指出的是，要首先定义需要什么，然后才开始设计阶段，而不是在设计结束的时候，事后定义达到了什么。后者的风险在于：当设计者的耐心没有了（而不是当目标达到的时候），设计阶段就结束了；或者可能导致构造的方案达到的多于需要的，从而效率上不如更好的方案。

(2) 对使用的重要性：假定想在某些更大的系统中使用加密方案。如何知道使用哪个加密方案？如果有多个候选方案，如何辨别哪个方案对应用已经足够？拥有给定方案的精确安全定义，以及与形式化的安全假设相关的安全证明（在原则 2 和原则 3 中讨论的），可以回答这些问题。特别是定义系统所需要的安全（见上面第(1)点），然后验证给定加密方案满足的安全是否达到所需要的安全。另外，明确说明需要的定义，并寻找满足这些定义的加密方案。注意，选择“最安全的”方案可能是不明智的，这是因为更弱的安全定义可能对应用来说已经足够，而且，也许能够使用更加高效的方案。

(3) 对研究的重要性：给定两种加密方案，如何比较它们？没有任何安全定义，唯一可比较的是效率，但是仅仅考虑效率是不够的，因为高效率的方案可能是完全不安全的，因而不可用。方案可以达到的安全级别的精确定义提供比较的另一端。如果两个方案是等效率的，但是第一个比第二个满足更强的安全定义，则选第一个^⑤。在安全和效率之间可能有所权衡，但是，至少精确的定义能帮助理解权衡的细节。

当然，精确的定义同时使得严格的证明成为可能（将在原则 3 处讨论），但是即使不考虑这个，上述三个原因依然成立。

一个错误的想法是形式化的定义是不必要的，因为“我们有对什么是安全的直觉感受”。对初学者，不同的人对什么是安全有不同的直觉。甚至一个人对安全有多种直觉，取决于应用的环境。例如，第 3 章将研究四种不同的对称密钥加密的安全定义，每种方法应用在不同的场景中。在任何情况下，形式化的定义对传递所谓的“直觉想法”是必要的。

例子：安全加密。认为形式化定义是简单的也是一个错误。例如，如何形式化地描述对称密钥加密方案所需要的安全概念？（读者可能想停下来思考一下再继续读下去。）我们曾经多次问学生安全加密如何来定义，回答依次如下：

(1) 回答 1：如果没有敌手能够在给定密文的情况下发现秘密的密钥，则加密方案是安全的。这一定义不妥。加密的目标是保护加密的消息，而密钥仅仅是达到这一目的的手段。用一个看似荒唐的例子，考虑一种加密方案忽略密钥，直接输出明文。明显地，没有敌手能够发现密钥。但是，这种方案明显没有安全可言^⑥。

(2) 回答 2：当没有敌手能够发现密文对应的明文，则加密方案是安全的。这个定义看上

^⑤ 当然，事情很少有这么简单。

^⑥ 你可能会说：“但是，这不是我想表达的意思！”是的，这就是问题的关键：通常形式化地描述想表达的想法不是一件容易的事情。

去比前面那个更好，甚至可以从某些密码学教科书中见到。但是，经过片刻思考，会发现它同样是不尽人意的。例如，根据这一定义，一个加密方案透露 90% 的明文，将仍然被认为是安全的，只要很难发现剩下的 10%。但是，在大多数加密应用中，这明显不能接受。例如，雇佣合同中大部分为普通文本，仅仅薪水部分是机密的；如果薪水在被泄漏的明文的 90% 部分中，则加密没有得到任何好处。

如果你发现上述反例很愚昧，请再次参见脚注⑥。关键点依旧是：如果陈述的定义不是想表达的，则方案所证明的安全没有真正提供需要的安全保护级别。（这个例子说明了为什么精确定义是重要的。）

(3) 回答 3: 如果没有敌手能够确定密文所对应的明文中的任何字符，则加密方案是安全的。这看上去已经像是一个良好的定义。但是，其他微妙的地方出现了。回到雇佣合同的例子，不可能确定任何薪水值或者甚至是任何其中的数字。但是，如果泄露了加密的薪水是多于还是少于每年 10 万美元，加密方案还是安全的吗？很明显不是。这导致下一个回答。

(4) 回答 4: 如果没有敌手能够从密文中确定关于明文的任何有意义的信息，则加密方案是安全的。这已经接近真正的定义。但是，它缺少一个方面：没有定义什么信息是“有意义的”。不同的信息可能在不同的应用中意义不同。这导致一个非常重要的原则，它关系到密码学原语的安全定义：安全定义应当对所有潜在的应用是充分的。这是基本要求，因为没有人知道将来会有什么应用。而且，其实现通常是通用密码库的一部分，可用于多个不同的场合和应用中。理想情况下安全应当在所有可能的应用中都得到保证。

(5) 最终回答: 如果没有敌手能够从密文中计算任何关于明文的函数，则加密方案是安全的。这提供一个非常强的保证，而且进行了恰当的形式化陈述，被认为是“正确的”对加密的安全定义。甚至这里，还需要考虑攻击模型问题，以及应当如何定义。

虽然找到了安全加密的正确要求，从概念上来说，还需要将这些要求通过数学语言描述和进行形式化的描述，这不是一件简单的事情（将在第 2 章和第 3 章详细讨论）。

正如在“最终回答”中提到的，形式化的定义必须能明确地定义攻击模型：也就是说，是唯密文攻击还是选择明文攻击。这里列举的是通用的原则。特别地，为了完整地定义密码学任务的安全，有两个不同的问题必须明确。第一个是什么是“攻破”，第二个是敌手的能力是如何假定的。前面提过，加密方案被攻破是指：敌手从密文中学习到关于明文的某些函数。敌手的能力与其假定拥有的行为能力相关，包括计算能力。前者指敌手是否仅仅假定只能偷听加密的消息（即唯密文攻击），还是敌手可以主动请求任意感兴趣的明文的密文（即选择明文攻击）。后者是指敌手的计算能力。在本书中，除第 2 章外，都希望能抵御“有效”敌手的攻击，即任何多项式时间运行的敌手。（关于此问题的完整讨论见 3.2.1 节。此后，“有效的”即指能在一个生命期内完成。“可行的”是更精确的术语。）转变成实际的术语，即要求安全性达到任何敌手使用超级计算机需要计算几十年的时间。

总之，任何安全定义将使用如下的通用形式：

如果特定的敌手不能完成特定的攻破，则对给定任务的一个密码学方案是安全的。

需要强调这一定义不对敌手的策略进行任何假设。这是一个重要的区别：假设敌手的能力（例如，能够进行选择明文攻击而不是唯密文攻击），但是不对敌手如何使用其能力进行假设。这个称为“任意敌手原则”，安全必须确保抵御任意拥有特定计算能力的敌手。这一原则是重要的，因为敌手在攻击中将采用的策略是不可能预先知道的（且历史已经证明这种企图

注定失败)。

数学世界以及现实世界。安全定义本质上提供了对现实世界的问题数学描述。如果数学定义不能合适地对现实世界建模，则定义可能是无用的。例如，如果假定的敌手能力非常弱(但实际上敌手有更强的能力)，或者密码方案的攻破来自不可预见的真实攻击(如同前面对加密方案的回答)，那么还是没有得到“真正的安全”，即使是使用了“数学上安全的”构造方案。简言之，安全的定义必须准确地对现实世界建模，以便使用数学语言描述安全承诺。

事实上，通常被广泛接受的定义在某些新应用中是不合适的。作为一个明显的例子，有些加密方案被证明为安全的(相对一些前面讨论的定义而言)，且在智能卡中实现了。归因于智能卡的物理属性，敌手可能监控到在加密方案运行时智能卡的能量使用情况(例如，使用的能量随时间的变化情况)，而且，这些信息可以用来确定密钥。这既不是安全定义的错误，也不是满足安全定义的方案的安全证明的错误；问题出在安全定义和现实世界的实现(方案在智能卡上的实现)的不匹配。

这并不是说定义(或者证明)是无用的，定义以及满足定义的方案可能在某些情况下仍然是合适的，如当加密在终端主机上进行，它的能量使用情况不可能被敌手监控到。而且，达到智能卡安全加密的方案需要进一步提炼定义，以便于将能量分析包含在考虑中。或者，开发可能对抗能量分析的硬件对策，使得原来的定义(以及原来的方案)对智能卡来说还是合适的。关键点是：使用定义你至少知道你站在哪里，即使定义没有准确的建模应用该方案的特定场合。相比而言，没有定义时连什么错了都不清楚。

在数学模型和需要建模的现实之间存在不匹配的可能，它不是只出现在密码学中，而是出现在整个科学中。举一个来自计算机科学的例子，考虑数学证明的含义，存在一些定义清楚的问题但计算机不能解决^⑦。这立刻得到一个问題：“计算机能解决的问题”是什么意思。特别地，只有当对“什么是计算机”的数学定义存在的时候，才能提供数学证明。问题是计算是一个真实世界的过程，有很多不同的计算方式。为了真正相信“不可解的问题”是真正不可解的，必须确信对计算的数学定义的确反映了计算的真实世界过程。我们并不知道何时能够达到这个目标？

这个内在的困难被阿兰图灵注意到，他研究了计算机能解决什么问题 and 不能解决什么问题。他的论文[140]中写到(方括号中的文本替换了原文使得读起来更清楚)：

没有企图表明[我们定义的可被计算机解决的问题]包括自然被认为可以计算的[那些问题]。所有可给出的论据其局限在于基本上求助于直觉。真正的问题是“什么是[计算]中可能执行的过程？”

我将使用三种论据。

- (1) 直觉。
- (2) 两种定义的等价证明(如果新定义更具直觉上的吸引力)。
- (3) 给出一大类[可使用给定计算定义来解决的]问题例子。

在某种情况下，图灵面对的困难和密码学家面对的是一样的。他发展了一种数学计算模型，但是需要确信模型是合适的。同样地，密码学家定义了安全概念，需要确信其定义意味着对现实世界的有意义的的安全保证。和图灵一样，他们可能使用下面的工具来证实：

^⑦ 学习过计算理论课程读者将熟悉这类问题是存在的这一事实(如停机问题)。

(1) 求助于直觉：这是第一个工具，当思考一个新安全定义的时候，观察这个定义是否意味着直觉上希望拥有的安全属性。这是最小要求，因为初始直觉通常导致非常弱的安全想法（正如在对加密的讨论中看到的）。

(2) 等价性证明：通常论证新的安全定义和一个旧的（或更强）的、更熟悉的或者直觉上更有吸引力的定义等价。

(3) 例子：一个证明安全定义是充分、有用的方法是：显示熟悉的、不同的现实世界攻击可被定义排除掉。

此外，依赖时间的检验以及随着时间的推移，研究者和实践者的钻研和调查将会证明定义的合理性。

1.4.2 原则 2——精确假设的依赖

大部分现代密码学的构造方案不可能证明为无条件安全。实际上，这类证明需要解决计算机复杂性理论中的难题，这不像是现在能办到的。这种不幸的状态导致一种有特色的结果：安全性依赖于某种假设。第二种现代密码学的原则是：假设必须被精确地陈述。这三个原因：

(1) 假设的正确性：假设是没有证明的陈述，但是据推测是正确的。为了强调某些假设的可信程度，有必要对假设进行研究。假设检查得越仔细，且测试后未见反例，则假设的可信程度就越高。此外，对假设的研究能提供其正确性的支持证据，这些证据可从其他一些被广泛相信的假设推出。如果所依赖的假设没有精确的陈述和展现，它不可能被研究和（潜在性的）被驳斥。因此，提高假设的可信程度的一个先决条件是：拥有对被假设内容的精确陈述。

(2) 对多个方案的比较：通常在密码学中，可能面对两个方案，这两个方案都是可证明满足某些安全定义，但是基于不同的安全假设。假设两个方案是效率相等的，应该倾向于哪个方案？如果第一个方案基于的假设弱于第二个（即第二个假设推出第一个假设），那么第一个方案更可取，因为有可能发现第二个假设是错的，而第一个假设保持正确。如果被两个方案使用的假设是不可比的，那么通常的规则是选择那个其假设被更彻底地研究过的方案，或者假设更加简单的方案（原因在前面段落已经陈述）。

(3) 对安全证明的帮助：前面曾经提过，将对原则 3 进行更深入地讨论。现代密码学构造方法的介绍会伴随着安全证明。若方案的安全性不能被无条件地证明，即必须依赖某个假设，那么对“如果假设是正确的，则构造方案是安全的”的数学证明必须建立在精确的假设陈述之上。

通常有人假设构造方案本身是安全的。若安全性是明确定义的，则精确的假设也是明确定义的（构造方案的安全性证明是容易的）。不能这样做有几个原因：首先，如前所述，测试了多年的假设比一个新假设更可取，新的假设仅仅是为了证明给定构造方案的安全性而引入的。其次，通常倾向于陈述更简单的假设，因为这种假设更容易研究和反驳。所以比方说，某些数学问题难解决的假设是更容易学习和使用的，比假设一个加密方案满足复杂的（可能是不太自然的）安全定义要好。当简单的假设被长时间研究并且没有发现反驳意见，就会更自信地说它是正确的。另一个依靠“低层次”假设（而不是假设某个构造方案是安全的）的优势是：低层次假设的特点是能在多个构造方案中共享。如果特定的假设实例最终是错误的，（在任何基于该假设的高层次构造方案中）它可以简单地被不同的假设实例替换掉。

上述方法贯穿本书。例如，第 3 章和第 4 章显示如何（用多种方法）达到安全通信，假

定一个称为“伪随机函数”的原语存在。在这些章节中，完全没有说明如何构造这种原语。第 5 章讨论如何在实践中创建伪随机函数，并且在第 6 章中展示伪随机函数甚至可以从低层次原语构造。

1.4.3 原则 3——严格的安全证明

前面讨论的两个原则很自然地导致目前这个。现代密码学强调（对所建议方案的）严格安全证明的重要性。使用准确的定义和精确的假设，意味着这种安全证明是可能的。但是，需要证明的主要原因是构造方案和协议的安全性不可能用检查软件的方法来检查。例如，加密和解密“可行”，且密文看上去是乱码，并不意味着老练的敌手不能攻破方案。没有证明不存在特定能力的敌手可以攻破方案，则只好通过直觉。经验表明：密码学和计算机安全中依靠直觉将是灾难性的。有数不清的例子表明：没有证明的方案被攻破，有时候很快，有时候是方案提出或者部署后几年。

另一个安全证明如此重要的原因是与潜在的损害相关联的，如果使用不安全的系统，则导致损害。虽然软件缺陷有时候代价很高，但是当银行的加密方案或者认证机制被攻破后，其潜在的损害将是巨大的。最后，虽然软件中存在很多缺陷，但软件基本上还可用，因为通常用户不会试图让他们的软件崩溃。相比而言，攻击者使用相当复杂和诡异的方法（利用构造方案中的特殊属性）来攻击安全方案，有攻破系统的明确目标。所以，虽然正确性证明通常在计算机科学中是理想的，但是在密码学和计算机安全领域却是绝对必须的。需要强调上述观察不是猜测，而是多年来经验所证明的结论。

规约方法。大部分现代密码学中的证明使用所谓的规约方法。给定一个如下形式的定理

“若给定假设 X 是正确的，根据给定的定义，构造方案 Y 是安全的”。

证明通常展示：如何将假设 X 规约到攻破构造方案 Y。具体来说，证明中通常（通过构造的论据）展示：如何使用敌手攻破构造方案 Y 的方法导致一个与假设 X 的冲突。我们将在 3.1.3 节详细介绍。

总结——严格方法与临时方法的比较。以上三种原则组成了严格的现代密码学方法，与经典密码学中的临时方法不同。临时方法可能缺少上面三种原则中的某一个，也常常全部忽视三个。不幸的是，那些希望获得“快速和直接”解决方案的人（或者没有意识到严格方法的人），仍然在设计和应用临时方案。希望这本书帮助人们意识到严格方法的重要性，并且在开发新的、数学上安全的方案时取得成功。

参考文献和扩展阅读材料介绍

本章研究了少量已知的历史上的加密术。读者如果想了解更多历史和数学方面的知识，建议阅读 Stinson 的文献[138]，或者 Trappe 与 Washington 的文献[139]。这些方案在历史（特别是在战争史）中扮演着重要的角色（在 Kahn^[81]的书中有介绍）。

讨论古典非严格方法（以古典加密术为例）与给予精确定义和证明的严格方法之间的区别。Shannon^[127]是第一个采用后一种方法的人。现代密码学，基于（可计算的）假设和定义、证明，是从 Goldwasser 和 Micali^[69]的开创性论文开始的。我们将在第 3 章学习。现代密码学方法的全面论述可参见 Goldreich 的著作《Foundations of Cryptography》^[64,65]。本书的论

述在很多地方受到这本书的影响，特别是第 6 章。

练 习

- 1.1 请解密在“单字母替换”一节中最后部分提供的那段密文。
- 1.2 提供对一个Gen、Enc和Dec算法的形式化的定义，对单字母替换加密和 Vigenère 加密。
- 1.3 考虑 Vigenère 加密的改进版本，即在使用多个移位加密的地方，使用多个单字母替换加密。也就是说，密钥由 t 个字母的随机置换组成，明文字符在位置 $i, t+i, 2t+i$ 等处使用第 i 个置换加密。说明如何攻破这个版本的加密方法。
- 1.4 若企图阻止对 Vigenère 加密方法的 Kasiski 攻击，提出了下面的修改建议。给定周期 t ，明文分解成多个块，每个块的大小为 t 。回顾一下：在每个块中，Vigenère 加密方法是通过使用第 i 个密钥（使用移位加密）来加密第 i 个字符。令密钥为 $k_1, \dots, k_t$ ，这意味着在每个块中的第 i 个字符是通过加上 k_i 模 26 来加密的。在这个建议的修改方案中：加密第 j 个块中的 i 个字符通过加上 $k_i + j$ 模 26 来加密。
 - (1) 说明这种加密可以完成。
 - (2) 描述上面修改方案对 Kasiski 攻击的影响。
 - (3) 设计一种替代方法确定这个方案的周期。
- 1.5 说明移位加密、替换加密、Vigenère 加密都很容易被已知明文攻击攻破。对这些加密方法而言，需要多少明文就可以发现整个密钥？
- 1.6 说明移位加密、替换加密、Vigenère 加密都很容易被选择明文攻击攻破。为了完全发现密钥，需要多少明文被加密？和上一个问题进行比较。

第2章 完善保密加密

上一章介绍了一些古典加密方案以及如何用很少的计算量去完全破解这些方案。本章讨论另外一个极端，研究可证明安全的加密方案，甚至可抵御有无限计算能力的攻击者，这种方案称为完善保密加密。本章介绍在什么条件下才能达到完善保密加密，并解释其原因。

从某种意义上说，本章内容属于“经典密码学”多过“现代密码学”。除了这里介绍的内容是在密码学革命（20世纪70年代中期和80年代）之前形成的之外，本章中研究的构造方法仅仅依靠1.4节中的第一个和第三个原则。也就是说，给出精确的数学定义和严格的证明，不必依赖任何未经证明的假设。这显然是有利的。但是，这种方法有其内在的局限性。因此，除了作为一个良好的基础来理解现代密码学的原则之外，本章的结论也证实了所采用的三个前述原则。

本章假定读者熟悉基本的概率知识，相关概念参见附录A.3。

2.1 定义和基本属性

首先简要回顾一下上一章中介绍的一些语法。一个加密方案可由三个算法来定义，Gen、Enc和Dec，以及明文空间 $\mathcal{M}$ 的说明， $|\mathcal{M}| > 1^{\textcircled{1}}$ 。密钥产生算法Gen是一种概率算法输出密钥 K ， K 根据一定的概率分布来选取。用 $\mathcal{K}$ 表示密钥空间，即由Gen输出的所有可能的密钥集合，而且 $\mathcal{K}$ 必须是有穷的。加密算法Enc输入一个密钥 $k \in \mathcal{K}$ 和一个明文 $m \in \mathcal{M}$ ，然后输出一个密文 c ，我们定义它为 $\text{Enc}_k(m)$ 。加密算法可能是概率的，以至于 $\text{Enc}_k(m)$ 多次运行时输出不同的密文。为强调这一点，用 $c \leftarrow \text{Enc}_k(m)$ 来表示明文 m 用密钥 k 加密得到密文 c 的概率过程。（如果Enc是确定的，记为 $c := \text{Enc}_k(m)$ 加以强调。）对于所有可能的 $k \in \mathcal{K}$ 和 $m \in \mathcal{M}$ ，用 $\mathcal{C}$ 表示所有可能的由 $\text{Enc}_k(m)$ 输出的密文集合。（以及所有随机选择的Enc，如果它是随机的话。）解密算法Dec通过输入密钥 $k \in \mathcal{K}$ 和密文 $c \in \mathcal{C}$ ，输出明文 $m \in \mathcal{M}$ 。本书假设加密方案都是完全正确的，也就是说所有的 $k \in \mathcal{K}$ 、 $m \in \mathcal{M}$ 和任何由 $\text{Enc}_k(m)$ 输出的密文 c ， $\text{Dec}_k(c) = m$ 的概率为1。不失一般性，这意味着可以假设Dec是确定的（因为 $\text{Dec}_k(c)$ 每次运行时的输出必须相同）。用 $m := \text{Dec}_k(c)$ 来表示用密钥 k 解密密文 c 的过程。

在下面的定义和定理中，提及 $\mathcal{K}$ 、 $\mathcal{M}$ 和 $\mathcal{C}$ 上的概率分布。 $\mathcal{K}$ 的分布是由运行Gen并取输出而得。（正如前面提到的，Gen几乎总是从 $\mathcal{K}$ 中均匀地选择一个密钥然后输出。通常这一假设不失一般性。）对于 $k \in \mathcal{K}$ ，用 $\Pr[K = k]$ 来表示Gen输出 k 的概率。（正式地， K 是用来表示密钥值的一个随机变量。）类似地，对于 $m \in \mathcal{M}$ ，用 $\Pr[M = m]$ 来表示明文为 m 的概率。事实上，明文是根据一些分布进行选择的（而不是固定的），这一事实意味着，至少从敌手的观点上，不同的消息有不同的发送概率。（若敌手知道发送的是什么消息，则不必解密什么，通信双方也不必使用加密。）举一个例子，敌手可能知道加密的明文是“attack tomorrow”或者“don't attack”

^① 若 $|\mathcal{M}| = 1$ ，则只有一个消息，通信无意义，更不用说加密了。

中的一个。更进一步，敌手可能知道（通过其他方式）明文攻击命令的概率是 0.7，不攻击命令的概率是 0.3。此时， $\Pr[M = \text{attack tomorrow}] = 0.7$ ， $\Pr[M = \text{don't attack}] = 0.3$ 。

$\mathcal{K}$ 和 $\mathcal{M}$ 的分布是独立的，也就是说密钥和明文是独立选择的。这是因为在知道明文前密钥已被选定（如通信双方共享）。此外， $\mathcal{K}$ 的分布是被加密方案本身确定（因为它由Gen确定），但是 $\mathcal{M}$ 的分布可能随着使用加密方案的通信方而变化。

对于 $c \in \mathcal{C}$ ，用 $\Pr[C = c]$ 表示密文为 c 的概率。对于给定的加密算法Enc， $\mathcal{C}$ 上的分布完全取决于 $\mathcal{K}$ 上和 $\mathcal{M}$ 上的分布。

定义。现在定义完善保密加密这个概念。直观上，想象一个敌手知道 $\mathcal{M}$ 的概率分布；也就是说，敌手知道将要发送的不同消息的可能性（如上面的例子）。敌手随后观察到一些已经由一方发送给另外一方的密文。理想情况下，观察这些密文应该对敌手的知识没有任何影响；换句话说， m 已发送的后验（posteriori）概率（给定所观察到的密文）应该与 m 将被发送的先验（priori）概率相同。任何 $m \in \mathcal{M}$ 都满足上述要求。此外，即使敌手有无限计算能力，上述要求依然满足。这意味着，密文没有泄漏任何明文信息，因此，敌手截获一个密文不能得到任何关于明文的信息。正式的描述：

定义 2.1 明文空间为 $\mathcal{M}$ 的加密方案(Gen, Enc, Dec)是完善保密加密，若对 $\mathcal{M}$ 上任意的概率分布，任何明文 $m \in \mathcal{M}$ 、任何密文 $c \in \mathcal{C}$ 且 $\Pr[C = c] > 0$ ，有

$$\Pr[M = m | C = c] = \Pr[M = m]$$

（条件 $\Pr[C = c] > 0$ 是从技术上避免出现零概率事件。）另外一种对定义 2.1 的解释是：当明文和密文上的分布是独立的，方案才是完善保密加密。

简化约定。本章只考虑 $\mathcal{M}$ 和 $\mathcal{C}$ 上的概率分布中，任何 $m \in \mathcal{M}$ 和 $c \in \mathcal{C}$ 的概率是非零的情况^②，这大大简化了表述，因为常常需要被 $\Pr[M = m]$ 或者 $\Pr[C = c]$ 除，如果它们可能为零则会出现分母为零的问题。同样，在定义 2.1 中有时需要用到 $C = c$ 和 $M = m$ 的条件。如果这两个事件概率为零也将导致同样问题。得益于这种简化约定，从现在开始，不再特别声明 $\Pr[M = m] > 0$ 或 $\Pr[C = c] > 0$ ，即便确实需要如此。

值得强调的是，该约定仅仅是为了简化说明，并不是一种基本的限制。特别值得一提的是，所有证明的定理同样适用于 $\mathcal{M}$ 和 $\mathcal{C}$ 上任意概率分布的情形（可以给一些明文或密文的概率赋值为 0）。参见练习 2.6。

等价公式。下面的引理给出了定义 2.1 的一个等价公式。

引理 2.2 明文空间为 $\mathcal{M}$ 的加密方案(Gen, Enc, Dec)是完善保密加密，当且仅当对于任意 $\mathcal{M}$ 上的概率分布，任意一个明文 $m \in \mathcal{M}$ ，和任意一个密文 $c \in \mathcal{C}$ ，有

$$\Pr[C = c | M = m] = \Pr[C = c]$$

证明（充分条件）选定一个 $\mathcal{M}$ 上的分布，对于任意的 $m \in \mathcal{M}$ 和 $c \in \mathcal{C}$ ，有

$$\Pr[C = c | M = m] = \Pr[C = c]$$

② 对 $k \in \mathcal{K}$ 而言这通常是满足的，因为 $\mathcal{K}$ 是由Gen输出的概率非零的密钥的集合。

在等式两边同时乘以 $\Pr[M = m]/\Pr[C = c]$, 得

$$\frac{\Pr[C = c | M = m] \cdot \Pr[M = m]}{\Pr[C = c]} = \Pr[M = m]$$

应用贝叶斯 (Bayes) 定理 (参见定理 A.8), 等式左边等于 $\Pr[M = m | C = c]$ 。因此, $\Pr[M = m | C = c] = \Pr[M = m]$, 该加密方案是完善保密加密方案。

必要条件的证明留做练习。 ■

注意: 上面的证明中假设 $m \in \mathcal{M}$ 和 $c \in \mathcal{C}$ 出现的概率都为非零概率 (根据前述简化约定), 因此 $\Pr[C = c]$ 和 $\Pr[M = m]$ 都可以做分母。

完美不可区分性 (Perfect indistinguishability)。现在应用引理 2.2 来得到一个完善保密加密的等价陈述: $\mathcal{C}$ 的概率分布独立于明文。也就是, 用 $\mathcal{C}(m)$ 表示加密 $m \in \mathcal{M}$ 时的密文分布 (密文分布取决于密钥的选择, 以及加密算法的随机性, 如果算法是概率的)。然后要求每一个 $m_0, m_1 \in \mathcal{M}$, $\mathcal{C}(m_0)$ 和 $\mathcal{C}(m_1)$ 的分布是相同的。这刚好以另外一种方式说明: 密文中不包含任何明文信息。我们称此为完美不可区分性, 因为它意味着不可能区分 m_1 的密文和 m_0 的密文 (因为在两种情况下, 密文分布是一样的)。

引理 2.3 明文空间为 $\mathcal{M}$ 的加密方案($\text{Gen}, \text{Enc}, \text{Dec}$)是完善保密加密, 当且仅当对于任意 $\mathcal{M}$ 上的概率分布, 每个 $m_0, m_1 \in \mathcal{M}$, 以及每个 $c \in \mathcal{C}$, 有

$$\Pr[C = c | M = m_0] = \Pr[C = c | M = m_1]$$

证明 假设此加密方案是完善保密加密, 并且选定明文 $m_0, m_1 \in \mathcal{M}$ 和密文 $c \in \mathcal{C}$ 。

由引理 2.2 可得:

$$\Pr[C = c | M = m_0] = \Pr[C = c] = \Pr[C = c | M = m_1]$$

由此完成必要条件的证明。

下面证明充分性: 假设对任意 $\mathcal{M}$ 上的概率分布, 每个 $m_0, m_1 \in \mathcal{M}$ 和每个 $c \in \mathcal{C}$, 满足 $\Pr[C = c | M = m_0] = \Pr[C = c | M = m_1]$ 。选定某个 $\mathcal{M}$ 上的分布, 对任意的 $m_0 \in \mathcal{M}$ 与 $c \in \mathcal{C}$, 定义 $p \stackrel{\text{def}}{=} \Pr[C = c | M = m_0]$, 由于对所有 m , 有 $\Pr[C = c | M = m] = \Pr[C = c | M = m_0] = p$, 于是有

$$\begin{aligned} \Pr[C = c] &= \sum_{m \in \mathcal{M}} \Pr[C = c | M = m] \cdot \Pr[M = m] \\ &= \sum_{m \in \mathcal{M}} p \cdot \Pr[M = m] \\ &= p \cdot \sum_{m \in \mathcal{M}} \Pr[M = m] \\ &= p \\ &= \Pr[C = c | M = m_0] \end{aligned}$$

因为 m_0 是任意选择的, 所以对所有 $c \in \mathcal{C}$ 和 $m \in \mathcal{M}$, 都有 $\Pr[C = c] = \Pr[C = c | M = m]$ 成立。由引理 2.2, 可以得出此加密方案是完善保密的。 ■

敌手不可区分性 (Adversarial indistinguishability): 本节介绍完善保密加密的另一个等

价定义。这个定义基于一个涉及敌手 $\mathcal{A}$ 的实验，并且形式化定义 $\mathcal{A}$ 不可达到的能力，即 $\mathcal{A}$ 不能区分出密文是来自哪个明文的加密。因而叫做敌手不可区分性。这个定义将作为起点引出下一章中计算安全的概念。纵观全书，将经常用实验来定义安全性。这个“实验”本质上是敌手和虚构的测试者之间的游戏，敌手试图破解一个密码方案，而测试者希望知道敌手是否成功。

定义一个实验 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}$ ，因为实验只考虑加密的秘密密钥和窃听敌手（称敌手为窃听者是因为其只能获得密文 c 并且试图从中确定一些有关明文的信息）。这个实验面向任意加密方案 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ 定义，明文空间为 $\mathcal{M}$ ，任意敌手为 $\mathcal{A}$ 。用 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}$ 表示一个给定 Π 和 $\mathcal{A}$ 的实验。实验定义如下：

窃听不可区分实验 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}$:

- (1) 敌手 $\mathcal{A}$ 输出一对信息 $m_0, m_1 \in \mathcal{M}$ 。
- (2) 由 Gen 产生一个随机密钥 k ，并且从 $\{0, 1\}$ 中随机选择一个比特 b ($b \leftarrow \{0, 1\}$)。（选择由虚构实体完成，该实体与 $\mathcal{A}$ 一起运行实验。）然后，一个密文 $c \leftarrow \text{Enc}_k(m_b)$ 被计算出来并交给 $\mathcal{A}$ 。
- (3) $\mathcal{A}$ 输出一个比特 b' 。
- (4) 如果 $b' = b$ ，定义实验的输出为 1，否则为 0。如果输出为 1，用 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}} = 1$ 表示，此时称 $\mathcal{A}$ 成功。

应该认为敌手 $\mathcal{A}$ 会设法去猜测实验中选定的值 b ，当猜测值 b' 正确时 $\mathcal{A}$ 就取得成功。可以观察到， $\mathcal{A}$ 在实验中仅仅通过随机猜测 b' ，便总是能以 $1/2$ 的概率取得成功。问题是 $\mathcal{A}$ 能否使猜测成功的概率更大？于是给出完善保密加密的另一种定义：如果没有敌手 $\mathcal{A}$ 能以大于 $1/2$ 的概率成功，则这种加密方案是完善保密加密。值得强调的是，本章中敌手 $\mathcal{A}$ 的计算能力是没有限制的。

定义 2.4 明文空间为 $\mathcal{M}$ 的加密方案 $(\text{Gen}, \text{Enc}, \text{Dec})$ 为完善保密加密，当对于所有敌手都满足

$$\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}} = 1] = \frac{1}{2}$$

下面的命题说明定义 2.4 与定义 2.1 是等价的，命题的证明留做练习。

命题 2.5 明文空间为 $\mathcal{M}$ 的加密方案 $(\text{Gen}, \text{Enc}, \text{Dec})$ ，当且仅当其为根据定义 2.1 的完善保密加密，则它是根据定义 2.4 的完善保密加密。

2.2 一次一密（Vernam 加密）

1917 年，Vernam 创造了一种完善保密加密的加密方法，现在称为“一次一密”（one-time pad），然而在当时并没有证明其是完善保密加密的（事实上，当时并没有完善保密加密的概念）。约 25 年以后，香农（Shannon）提出了完善保密加密的概念，并且证明“一次一密”能达到这一安全水平。

令 $a \oplus b$ 表示把二进制串 a 和二进制串 b 按比特位异或（也就是说，如果 $a = a_1, \dots, a_l$ ， $b = b_1, \dots, b_l$ ，则 $a \oplus b = a_1 \oplus b_1, \dots, a_l \oplus b_l$ ）。“一次一密”方案定义如下：

(1) 令整数 $l > 0$, 设明文空间 $\mathcal{M}$, 密钥空间 $\mathcal{K}$ 和密文空间 $\mathcal{C}$ 都等于 $\{0, 1\}^l$ (也就是说长度为 l 的二进制比特串的集合)。

(2) 密钥产生算法 Gen 从 $\mathcal{K} = \{0, 1\}^l$ 中依据均匀分布选择一个二进制比特串 (2^l 个串空间中任何一个被选中的概率都为 2^{-l})。

(3) 加密算法 Enc: 给定一个密钥 $k \in \{0, 1\}^l$ 和一个明文 $m \in \{0, 1\}^l$, 输出 $c := k \oplus m$ 。

(4) 解密算法 Dec: 给定一个密钥 $k \in \{0, 1\}^l$ 和一个密文 $c \in \{0, 1\}^l$, 输出 $m := k \oplus c$ 。

在讨论“一次一密”的安全性前, 我们注意到每个 k 和 m 都满足 $\text{Dec}_k(\text{Enc}_k(m)) = k \oplus k \oplus m = m$, 所以“一次一密”是一个正确的加密方案。

直观上, “一次一密”是完善保密加密, 因为给定密文 c , 敌手没有办法知道来自哪个原始明文 m 。为弄清原因, 注意到对所有可能 m 都存在一个密钥 k 满足 $c = \text{Enc}_k(m)$; 也就是说, $k = m \oplus c$ 。此外, 密钥根据均匀概率进行选择 (不被敌手知道), 所以没有一个密钥比其他密钥选中的可能性高。综上所述, 由于所有明文被加密的机会均等, 因而 c 没有泄露任何被加密明文 m 的信息。下面正式证明这一直觉:

定理 2.6 “一次一密”是完善保密加密。

证明 选定明文空间 $\mathcal{M}$ 上的分布, 选定任意一个 $m \in \mathcal{M}, c \in \mathcal{C}$ 。“一次一密”有

$$\begin{aligned}\Pr[C = c \mid M = m] &= \Pr[M \oplus K = c \mid M = m] \\ &= \Pr[m \oplus K = c] = \Pr[K = m \oplus c] = \frac{1}{2^l}\end{aligned}$$

由于这对所有分布和所有 m 都成立, 所以对于任意 $\mathcal{M}$ 上的概率分布, 每个 $m_0, m_1 \in \mathcal{M}$ 和每个 $c \in \mathcal{C}$, 满足

$$\Pr[C = c \mid M = m_0] = \frac{1}{2^l} = \Pr[C = c \mid M = m_1]$$

根据引理 2.3, 这意味着一次一密是完善保密加密。 ■

由此得出结论完善保密加密是可以实现的。令人遗憾的是, “一次一密”有很多缺陷, 最突出的是需要密钥和明文有一样的长度。首先也是最重要的是, 这意味着必须安全地存储一个很长的密钥, 这在实践中问题较大且通常是不可能的。另外, 其应用受到限制, 例如当希望发送很长消息的时候 (很难保证安全地存储一个很长的密钥), 或者当事先并不知道明文消息的长度上限的时候 (不可能共享一个没有长度上限的密钥)。并且, “一次一密”顾名思义, 密钥只用一次的时候才是“安全”的。尽管并没有定义当多个消息被加密时的安全, 但是很容易看出加密多于一次后会泄漏很多信息。特别地, 如果明文信息 m, m' 用同样一个密钥 k 进行加密, 敌手得到 $c = m \oplus k$ 和 $c' = m' \oplus k$ 从而可以计算出:

$$\begin{aligned}c \oplus c' &= (m \oplus k) \oplus (m' \oplus k) \\ &= m \oplus m'\end{aligned}$$

这样就知道了两条消息异或的结果。尽管这看上去好像不很重要, 但是已经足够说明其加密两条消息时不是完善保密加密。此外, 如果明文消息对应英语文本, 则给两个足够长消息的异或值, 就可能通过频率分析 (前一章有介绍, 虽然更为复杂) 得到明文消息。

2.3 完善保密的局限

本节介绍“一次一密”加密方案的上述限制是内在固有的。具体而言，证明所有完善保密加密方案的密钥空间至少要和明文空间一样大。如果密钥空间由固定长度的密钥组成，明文空间由固定长度的明文组成，则意味着密钥至少要和明文一样长。因此，长密钥的问题并不是“一次一密”专有，而是所有完善保密加密的内在问题。（另一个限制是密钥只能使用一次，也是完善保密加密的内在问题；参见练习 2.9。）

定理 2.7 设 $(\text{Gen}, \text{Enc}, \text{Dec})$ 是明文空间为 $\mathcal{M}$ 的一个完善保密加密方案， $\mathcal{K}$ 的密钥空间由 Gen 决定，则 $|\mathcal{K}| \geq |\mathcal{M}|$ 。

证明 下面证明如果 $|\mathcal{K}| < |\mathcal{M}|$ ，则这个加密方案不是完善保密加密。假设 $|\mathcal{K}| < |\mathcal{M}|$ ，考虑 $\mathcal{M}$ 为均匀分布且设密文 $c \in \mathcal{C}$ 有非零概率， $\mathcal{M}(c)$ 是所有密文 c 解密而得可能明文的集合，也就是

$$\mathcal{M}(c) \stackrel{\text{def}}{=} \{\hat{m} \mid \hat{m} = \text{Dec}_{\hat{k}}(c), \text{ 对于部分 } \hat{k} \in \mathcal{K}\}$$

显然有 $|\mathcal{M}(c)| \leq |\mathcal{K}|$ ，因为对于每个明文消息 $\hat{m} \in \mathcal{M}(c)$ ，至少可以确定一个密钥 $\hat{k} \in \mathcal{K}$ 使得 $\hat{m} = \text{Dec}_{\hat{k}}(c)$ 。（我们在前面曾假设 Dec 是确定的。）由于假设 $|\mathcal{K}| < |\mathcal{M}|$ ，这就意味着有一些 $m' \in \mathcal{M}$ 出现 $m' \notin \mathcal{M}(c)$ 。但是

$$\Pr[M = m' \mid C = c] = 0 \neq \Pr[M = m']$$

所以这个方案不是完善保密加密。 ■

完善保密加密的更低代价？ 上面的定理显示出完善保密加密的内在局限性。即便如此，常有个人或者组织宣称自己开发了一种全新的无法破解的加密方案，并且达到“一次一密”那样的安全等级而不需使用长密钥。上面的证明说明这样的宣称不可能是事实；这些声称者要么根本不懂密码学，要么是在公开撒谎。

2.4 *香农定理

在香农开创性的关于完善保密加密的工作中，提出了完善保密加密方案的一种性质。这种特性为：假设 $|\mathcal{K}| = |\mathcal{M}| = |\mathcal{C}|$ ，密钥产生算法 Gen 必须从所有可能密钥组成的集合中均匀地选择一个秘密密钥（就像“一次一密”），这样对于所有的明文和密文都存在唯一的密钥使明文映射到密文（还是就像“一次一密”那样）。进一步而言，这个定理是一个强大的验证（或者否定）完善保密加密方案的工具。证明后会进一步讨论这一点。

如前所述，假定 $\mathcal{M}$ 和 $\mathcal{C}$ 的概率分布满足：所有的 $m \in \mathcal{M}$ 和 $c \in \mathcal{C}$ 都为非零概率。这个定理考虑一种特殊情况： $|\mathcal{K}| = |\mathcal{M}| = |\mathcal{C}|$ ，即明文、密钥、密文集合的大小相等。由 $|\mathcal{K}| \geq |\mathcal{M}|$ ，易知 $|\mathcal{C}|$ 必须至少和 $|\mathcal{M}|$ 一样大，否则一定存在两个明文映射到同一个密文（造成解密的不确定）。因此，从某种意义上说， $|\mathcal{K}| = |\mathcal{M}| = |\mathcal{C}|$ 的情形“效率最高”。现在可以开始陈述这个定理。

定理 2.8（香农定理） 设加密方案 $(\text{Gen}, \text{Enc}, \text{Dec})$ 的明文空间为 $\mathcal{M}$ ，且 $|\mathcal{K}| = |\mathcal{M}| = |\mathcal{C}|$ ，则当且仅当下列条件成立时，此方案是完善保密加密。

(1) 由Gen产生的任意密钥 $k \in \mathcal{K}$ 的概率都是 $1/|\mathcal{K}|$ 。

(2) 对任意明文 $m \in \mathcal{M}$ 和任意密文 $c \in \mathcal{C}$ ，只存在唯一的密钥 $k \in \mathcal{K}$ 使得 $\text{Enc}_k(m)$ 输出 c 。

证明 证明背后的直觉逻辑如下。首先，如果加密方案满足条件(2)，则给定密文 c ，它可能从任意的明文 m 加密而来（这是由于对于任何明文 m 都存在一个密钥 k 映射到 c ）。同时，只有一个密钥映射每一个明文 m 到密文 c ，又由条件(1)知选取所有密钥的概率相同，完善保密加密可以用“一次一密”的方法来证明。以上为充分条件的证明，下面证明必要条件。如果 $|\mathcal{M}| = |\mathcal{K}| = |\mathcal{C}|$ 则一定只有一个密钥映射每个明文 m 到每个密文 c 。（否则，要么有些明文 m 不能映射到某个给定的 c ，要么某些 m 通过多个密钥映射到 c ，导致另一个明文 m' 不能映射到 c ，这些都和完美保密加密相矛盾。）有鉴于此，密钥必须等概率选取，否则某些明文的可能性会高于其他明文，这与完美保密加密相悖。正式证明如下。

设 $(\text{Gen}, \text{Enc}, \text{Dec})$ 为定理中的加密方案。简明起见，假设 Enc 为确定算法。首先证明如果 $(\text{Gen}, \text{Enc}, \text{Dec})$ 是完美保密加密，那么条件(1)和条件(2)成立。根据定理 2.7，不难发现对于每个 $m \in \mathcal{M}$ 和 $c \in \mathcal{C}$ 至少存在一个密钥 $k \in \mathcal{K}$ 满足 $\text{Enc}_k(m) = c$ 。（否则， $\Pr[M = m \mid C = c] = 0 \neq \Pr[M = m]$ 。）选定 m ，考虑集合 $\{\text{Enc}_k(m)\}_{k \in \mathcal{K}}$ ，根据刚才的证明， $|\{\text{Enc}_k(m)\}_{k \in \mathcal{K}}| \geq |\mathcal{C}|$ 。（因为任意 $c \in \mathcal{C}$ 都存在一个 $k \in \mathcal{K}$ 满足 $\text{Enc}_k(m) = c$ 。）另外，很明显有 $|\{\text{Enc}_k(m)\}_{k \in \mathcal{K}}| \leq |\mathcal{C}|$ 。得出结论

$$|\{\text{Enc}_k(m)\}_{k \in \mathcal{K}}| = |\mathcal{C}|$$

既然 $|\mathcal{K}| = |\mathcal{C}|$ ，于是 $|\{\text{Enc}_k(m)\}_{k \in \mathcal{K}}| = |\mathcal{K}|$ 。这意味着没有两个不同的密钥 $k_1, k_2 \in \mathcal{K}$ 满足 $\text{Enc}_{k_1}(m) = \text{Enc}_{k_2}(m)$ 。由于 m 是任意选择的，故对于任意 m 和 c ，都存在至少一个密钥 $k \in \mathcal{K}$ 满足 $\text{Enc}_k(m) = c$ 。综上所述（即存在至少一个密钥，且至多一个密钥），得到条件(2)。

下面证明对于任意 $k \in \mathcal{K}$ 有 $\Pr[K = k] = 1/|\mathcal{K}|$ 。设 $n = |\mathcal{K}|$ ， $\mathcal{M} = \{m_1, \dots, m_n\}$ （其中， $|\mathcal{M}| = |\mathcal{K}| = n$ ），并且选定密文 c 。标注密钥为 $k_1, \dots, k_n$ 以至于对于任意 i （ $1 \leq i \leq n$ ）都有 $\text{Enc}_{k_i}(m_i) = c$ 成立。这种标注可以实现是因为对任意密文 c 和 m_i ，都存在一个唯一的密钥 k_i ，满足 $\text{Enc}_{k_i}(m_i) = c$ 。此外对于不同的明文 m_i, m_j ，其密钥也不同（否则可能会出现解密结果不确定）。由完美保密加密，得到对任意 i ，有

$$\begin{aligned} \Pr[M = m_i] &= \Pr[M = m_i \mid C = c] \\ &= \frac{\Pr[C = c \mid M = m_i] \cdot \Pr[M = m_i]}{\Pr[C = c]} \\ &= \frac{\Pr[K = k_i] \cdot \Pr[M = m_i]}{\Pr[C = c]} \end{aligned}$$

其中第二个等式是由贝叶斯定理得来的，第三个等式由前面的标注得到的（即 k_i 是 m_i 映射到 c 的唯一密钥）。

综上所述，对于任意 i ，

$$\Pr[K = k_i] = \Pr[C = c]$$

因此，对于所有 i 和 j ， $\Pr[K = k_i] = \Pr[C = c] = \Pr[K = k_j]$ ，所有密钥等概率选取。得出结论：密钥选择服从均匀分布。也就是说，对任意密钥 k ，满足 $\Pr[K = k_i] = 1/|\mathcal{K}|$ ，得证。

下面证明充分条件。假设所有密钥出现的概率满足 $1/|\mathcal{K}|$ ，并且对于任意 $m \in \mathcal{M}$ 和任意 $c \in \mathcal{C}$ 都有唯一密钥 $k \in \mathcal{K}$ 满足 $\text{Enc}_k(m) = c$ 。这意味着可立刻得出：对于所有 m 和 c ，有

$$\Pr[C = c \mid M = m] = \frac{1}{|\mathcal{K}|}$$

与 $\mathcal{M}$ 上的概率分布无关。因此，对于任意 $\mathcal{M}$ 上的概率分布，任意 $m, m' \in \mathcal{M}$ ，任意 $c \in \mathcal{C}$ 都满足

$$\Pr[C = c \mid M = m] = \frac{1}{|\mathcal{K}|} = \Pr[C = c \mid M = m']$$

所以根据引理 2.3 可知，这个加密方案是完善保密加密。 ■

香农定理的应用：定理 2.8 从本质上给出了完善保密加密的完整特征。另外，由于条件(1)和条件(2)与明文空间 $\mathcal{M}$ 上的概率分布没有任何关系，这个定理意味着：如果存在一个加密方案对明文空间 $\mathcal{M}$ 上的特定概率分布满足完善保密加密，则此加密方案在一般情况下也能满足完善保密加密（这里一般情况指明文 $\mathcal{M}$ 上的所有概率分布）。最后，香农定理在证明一个给定方案是否是完善保密加密上有很大的用处。条件(1)很容易确认，条件(2)可以在不分析任何概率的情况下来证明或否定（这一点和定义 2.1 不同）。例如，“一次一密”的完善保密性（定理 2.6）用香农定理证明很就很简单。但是，定理 2.8 只在 $|\mathcal{K}| = |\mathcal{M}| = |\mathcal{C}|$ 时满足，所以在应用定理时要特别小心。

2.5 小 结

这里将结束完善保密加密的介绍，本章主要思想是：完善保密加密是可以达到的，也就是说存在这样的加密方案使得密文完全不会泄漏明文的信息，即使攻击者有无限制的计算能力。但是，这些方案的局限是密钥必须至少和明文一样长。因此，在实践中完善保密加密很少用到。传言在冷战时期连接白宫和克里姆林宫的“红色电话”是用“一次一密”来保护的。当然，美国政府与苏联之间进行超长随机密钥的交换并不是很困难，所以“一次一密”实践上是可行的。然而，在大多数情况下（尤其是商业模式下），这种长密钥的局限令“一次一密”以及其他完善保密加密方案无法使用。

参考文献和扩展阅读材料介绍

“完善保密加密”最初是由香农提出并进行研究的^[127]。此外，为了介绍这个定义，他证明了“一次一密”（最初由 Vernam^[141]提出）是完善保密加密，并且也证明了完善保密加密的特征（及其局限性）。Stinson^[138]给出了关于完善保密加密的更多讨论。

本章简要地介绍了完善保密的加密方案。还有其他一些密码学问题同样也可以用“完美”安全来解决。一个典型的例子就是消息鉴别问题，目标是防止敌手在传输数据时（以一种不能被检测到的方式）修改数据，这个问题将在第 4 章深入讨论。对完美安全的消息认证有兴趣的读者可以参考 Stinson^[136]、Simmons^[134]的综述，或者 Stinson 教课书的第一版^[137]第 10 章了解更多信息。

练习

2.1 证明定理 2.2 的必要条件。

2.2 给出证明或者提出反例：

所有完善保密加密方案都满足对任意明文空间 $\mathcal{M}$ ，任意 $m, m' \in \mathcal{M}$ 和任意 $c \in \mathcal{C}$ ，都满足

$$\Pr[M = m \mid C = c] = \Pr[M = m' \mid C = c]$$

2.3 当使用密钥为 $k = 0^\ell$ 的“一次一密”(Vernam's 密码)时，有 $\text{Enc}_k(m) = k \oplus m = m$ ，明文其实没有加密。因而建议在加密时使用 $k \neq 0^\ell$ 来改善“一次一密”。(即随机均匀地从长为 ℓ 的非零密钥集中选取密钥。)这样是否可以改进“一次一密”？特别是它还是完善保密加密吗？证明你的回答。如果你的回答是肯定的，解释为什么“一次一密”不能用这种方式来描述。如果你的回答是否定的，用事实说明用 0^ℓ 加密并没有改变明文。

2.4 本题研究移位密码、单字母替换密码和 Vigenère 密码在何种条件下是完善保密加密。

(1) 证明只有给单个字符加密时，移位密码才是完善保密加密。

(2) 单字母替换密码是完善保密加密的最大明文空间 $\mathcal{M}$ 是什么？($\mathcal{M}$ 不必仅包括有效的英文单词。)

(3) 说明如何使用 Vigenère 加密方法给长为 t 的单词加密以达到完善保密加密(注意：密钥长度可以选择)。证明你的回答。

结合前面章节介绍的攻击案例。

2.5 证明或者提出反例：任意密钥空间大小与明文空间大小相等，且密钥从密钥空间均匀选择的加密方案是完善保密加密。

2.6 加密方案 $(\text{Gen}, \text{Enc}, \text{Dec})$ 对于任意 $\mathcal{M}$ 上的分布，每一个非零概率 $m \in \mathcal{M}$ 都满足定义 2.1 (利用本章中的简化约定)。证明此方案在明文空间 $\mathcal{M}$ 上所有分布都满足此定义(即包括可能出现零概率明文的明文空间)。推断，此方案对于任意明文空间 $\mathcal{M}' \subset \mathcal{M}$ 仍然满足完善保密加密。

2.7 证明由定义 2.1 可以推出定义 2.4。

提示：利用习题 2.6，当明文分布为任何两个明文上的均匀分布，依旧是完善保密加密。

(特别是这两个消息都是由实验中的 $\mathcal{A}$ 输出的。)然后，利用引理 2.3。

2.8 证明命题 2.5 的必要条件。也就是证明定义 2.4 推出定义 2.1。

提示：如果加密方案 Π 根据定义 2.1 不是完善保密加密，则引理 2.3 表明存在明文 $m_0, m_1 \in \mathcal{M}$ 和 $c \in \mathcal{C}$ ，满足 $\Pr[C = c \mid M = m_0] \neq \Pr[C = c \mid M = m_1]$ 。利用 m_0, m_1 构造的敌手 $\mathcal{A}$ ，其成功的概率为 $\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}} = 1] > \frac{1}{2}$ 。

2.9 考虑下面对加密两个消息的完善保密加密定义。一个明文空间为 $\mathcal{M}$ 的加密方案 $(\text{Gen}, \text{Enc}, \text{Dec})$ 是两消息完善保密加密，当对 $\mathcal{M}$ 上的所有分布，任意 $m, m' \in \mathcal{M}$ ，以及任意 $c, c' \in \mathcal{C}$ 满足 $\Pr[C = c \wedge C' = c'] > 0$ ，有

$$\Pr[M = m \wedge M' = m' \mid C = c \wedge C' = c'] = \Pr[M = m \wedge M' = m']$$

其中 m 和 m' 是从相同 $\mathcal{M}$ 上的分布中独立选择的。证明不存在加密方案满足这个定义。

提示：利用 $m \neq m'$ ，但是 $c = c'$ 。

- 2.10 考虑下面对两消息加密的完善保密加密定义。明文空间为 $\mathcal{M}$ 的加密方案($\text{Gen}, \text{Enc}, \text{Dec}$)是双消息完善保密加密，当对于明文空间 $\mathcal{M}$ 上的所有分布，任意 $m, m' \in \mathcal{M}$ ，以及任意 $c, c' \in \mathcal{C}$ 满足 $c \neq c'$ 且 $\Pr[C = c \wedge C' = c'] > 0$ ，有

$$\begin{aligned}\Pr[M = m \wedge M' = m' \mid C = c \wedge C' = c'] \\ = \Pr[M = m \wedge M' = m' \mid M \neq M']\end{aligned}$$

其中， m 和 m' 是从 $\mathcal{M}$ 上的同一分布中独立选择的。提出一个可证明满足这个定义的加密方案。密钥长度与明文长度相比如何？

提示：提出的加密方案可不要求效率。

- 2.11 证明可以假设密钥产生算法 Gen 是从密钥空间 $\mathcal{K}$ 上均匀选择的。

提示：设 $\mathcal{K}$ 表示随机算法 Gen 产生的所有随机带的集合。

- 2.12 假设只要求明文空间 $\mathcal{M}$ 上的加密算法($\text{Gen}, \text{Enc}, \text{Dec}$)满足下列条件：对所有 $m \in \mathcal{M}$ ， $\text{Dec}_k(\text{Enc}_k(m)) = m$ 的概率至少是 2^{-t} （这个概率是通过选择密钥 k ，以及加密解密算法的随机性决定的）。完善保密加密（如定义 2.1 所述）在当 $|\mathcal{K}| < |\mathcal{M}|$ ，其中 $t \geq 1$ 时可以达到。你能找到并证明 $\mathcal{K}$ 的更小的界吗？

- 2.13 证明一个类似于定理 2.7 中的“几乎完美”安全的情况。设 $\varepsilon < 1$ 是一个常量，仅要求对明文空间 $\mathcal{M}$ 上的所有分布，任意 $m \in \mathcal{M}$ 和任意 $c \in \mathcal{C}$ ，有

$$|\Pr[M = m \mid C = c] - \Pr[M = m]| < \varepsilon$$

找到并证明对任何满足这一定义的加密方案，相对于 $\mathcal{M}$ 的 $\mathcal{K}$ 的更小的界。

第二部分 对称密钥（对称）密码学

第3章 对称密钥加密以及伪随机性

本章将学习“伪随机性”的概念，即“看上去”是完全随机的（满足精确的定义），尽管它其实不是。理解如何用它来打破上一章给出的完善保密加密的约束条件。特别是本章会介绍一个使用短密钥（长度有几百比特）的加密方案用来安全地加密很长的消息（假设总共有几个吉字节）。这样的方案能够避开完善保密的内在限制，能够实现稍弱但已足够的计算安全。在开始讨论对称密钥加密之前，首先考察密码学中的一般计算方法。这种计算方法将在本书其他部分使用，是现代密码学的基础。

3.1 密码学的计算方法

在前两章中，已经学习了什么是经典密码学。从对历史上使用的密码方案的一个简短回顾开始，重点关注它们是如何被攻破的，以及从这些攻击中，可以学到什么。在第2章中，继续讲解密码学方案，这些密码学方案能够被证明是安全的（根据一些特殊的安全定义），甚至假设敌手拥有无限计算能力。这样的方案叫做“信息理论安全”，或者是“完美安全”，因为它们的安全性是基于敌手没有足够的“信息”来成功地完成攻击，而不管敌手的计算能力^①。特别地，如前所述，在“完善保密”的加密方案中，密文不能含有任何关于明文的信息（假设密钥是未知的）。

信息理论安全与计算安全形成鲜明对比，后者是大多数现代密码学构造方法的目标。若限制在对称密钥加密的情形（尽管这里的讨论可以应用在更加通用的情况），现代加密方案的特点是，只要给足够的时间和计算能力，它们都能被攻破，所以它们不满足定义2.1。尽管如此，在某种假设下，即使使用现有的最快的超级计算机，为了攻破这些加密方案需要的计算量所耗运行时间将会超过人的一生。对于所有实用目的，这个级别的安全就足够了。

计算安全比信息理论安全要稍弱一些。计算安全当前仍然依赖不能被证明的假设，然而实现信息理论安全不需要假设^②。姑且认为计算安全可满足所有的实际用途，为什么要放弃实现完美安全的想法？2.3节的结论给出了一个为什么现代密码学已经选择走这条路线的原因。该节说明完善保密的加密方案使得密钥的长度有严格的下界限制；也就是说，密钥的长度必须和所有被该密钥加密的消息的总长度相等。当需要信息理论安全时，其他的密码学任务也

① 术语“信息”有严格的数学含义，但是在这里以非正式的方式来使用它。

② 理论上，这些假设可能有一天都会被推翻。但是不幸的是，现在的知识状况要求做出一些假设，来证明任何密码学构造方案中的计算安全。那些熟悉 P 和 NP 问题的读者，值得留意：任何对密码学构造方法的计算安全的无条件证明，都需要证明 $P \neq NP$ 。

拥有着类似的消极结论。因此，尽管数学上是吸引人的，但为了获得实用的密码学方案，有必要对完美安全进行妥协。需要强调的是，尽管不能获得完美安全，但这个并不意味着要抛开严格的数学方法。相反，定义和证明仍然是必须的，唯一的区别是，现在考虑的是稍弱一些，但是仍然有意义的安全定义。

3.1.1 计算安全的基本思想

Kerckhoffs 以他的“密码学设计应该是公开的”原则而闻名于世。但是，他实际上阐述了 6 条原则，下面的原则与本节的讨论非常相关：

一种[密码术]必须在实践上无法破译，如果不是数学上无法攻破的话。

尽管他在那个时候不太可能用现在的方式来陈述，这个 Kerckhoffs 原则从根本上说明了，使用一个完善保密的加密方案是不必要的，取而代之，使用一种不能在“合理的时间”内，以任何“合理的成功概率”攻破的方案就足够了（用 Kerckhoffs 的话说，这是一个“实践上不可破译”的方案）。用更具体的术语，使用这样一种加密方案就足够了：该方案能够（理论上）被攻破，但是在 200 年内，使用最快的可用的超级计算机，不能以大于 10^{-30} 的概率攻破。本节提出一个构架，来形式化地陈述密码学方案是“实践上不可破译”。

基于计算的方法包含了两种放宽的完美安全概念：

(1) 仅仅在对抗“有效的”敌手时，安全性存在，“有效的”是指在可行的时间里运行。

(2) 敌手潜在的成功概率是非常小的（小到以至于不关心它是否会真的发生）。

为了获得一个有意义的理论，需要精确地定义上面所提到的内容。有两个普遍的方法可以做到：具体方法和渐进方法。下面一一解释。

具体方法。具体方法是，通过明确限定任意敌手在最多某个特定时间内的最大成功概率，对一个给定密码学方案的安全性进行量化。也就是，令 t, ϵ 为正数常量， $\epsilon \leq 1$ 。则一个对安全的具体定义，大体地说，将会是以下的形式：

一个方案为 (t, ϵ) 安全，如果每个运行时间最多为 t 的敌手以最多为 ϵ 的概率成功攻破该方案。

（当然，上面提供的仅仅只是一个通用模板，为了让上面的陈述有意义，需要准确地定义什么是“攻破”方案。）作为一个例子，一个人可能希望使用这样的方案，该方案能够保证在使用最快的可用的超级计算机，且最多运行 200 年的情况下，没有敌手能够以大于 10^{-30} 的概率成功攻破该方案。或者，以 CPU 周期作为单位来测量运行时间可能会更方便，使用这样一种密码学方案，在最多运行 2^{80} 个 CPU 周期的情况下，没有敌手能够以高于 2^{-64} 的概率攻破该方案。

感觉一下 t, ϵ 的值是有益的，这些值在现代密码方案中是有代表性的。

例 3.1 现代对称密钥加密方案一般被认为给予了几乎最优的安全性：当密钥的长度为 n ，一个敌手运行时间为 t （假设用 CPU 周期测量），能够成功攻破该方案的概率最多为 $t/2^n$ 。（稍后将会看到为什么这个确实是最优的。）对 $t = 2^{60}$ 这个量级，计算在现在几乎不可能达到。在 1 兆赫的计算机（每秒执行 10^9 次周期）上运行 2^{60} 个 CPU 周期需要 $2^{60}/10^9$ 秒，大约 35 年。并行地使用多台超级计算机可能会将这个数据降低到几年。

一个普遍的密钥长度值，可能为 $n = 128$ 。在 2^{60} 和 2^{128} 之间的差别是一个包含了 21 个十

进制位的乘法因子 2^{68} 。为了对这个数有多大找点感觉，请注意，自从宇宙大爆炸到现在，根据物理学家的估计，秒数为 2^{58} 。

每百年发生一次的事件能够大致地被估计为事件在任意指定的一秒内发生的概率为 2^{-30} 。在任意指定的一秒内发生概率为 2^{-60} 的事件是前面提到概率的 2^{30} 分之一，可能被期待每 1000 亿年大概发生一次。因此，一个谨慎的参数选择应该为 $t = 2^{80}$ 以及 $\varepsilon = 2^{-48}$ （意味着密钥必须至少有128比特长）。◇

具体方法在实践中是有用的，因为具体方法保证了上面这些有实际数据的安全，这些都是密码方案的用户感兴趣的。但是，在解释具体的安全保证时，必须要小心。举个例子，当声称没有敌手能够在 5 年的时间，以大于 ε 的概率攻破一个指定的方案，必须清楚：这里假定什么类型的计算能力（如台式个人计算机、超级计算机、联网的数百台计算机）？这里是否考虑了未来计算能力的提高（通过摩尔定理，计算能力每隔 18 个月便会增加一倍）？这里假定的是使用“现成的”算法，还是对攻击优化的专门软件？更进一步说，这样的—个保证几乎没有提及敌手运行 2 年的成功概率（除了概率最大为 ε 外），并且没有提及敌手运行 10 年的成功概率的相关信息。

当使用一个具体安全方法时，方案能够达到 (t, ε) 安全，但是永远不会只是安全（单独出现）。更加确切地说， t, ε 的范围应该是多少，才能够认为一个 (t, ε) 安全方案是“安全”的？对此，没有明确的答案，因为一个安全保证可能能够满足普通用户，但是当用于加密机密的政府文件时，可能就无法满足了。

渐进方法。渐进方法是本书采用的方法。该方法来源于复杂性理论，将敌手的运行时间以及成功的概率视为某个参数的函数，而不是具体的数值。特别是，一个密码方案将包含一个安全参数，即整数 n 。当诚实的双方初始化方案（即生成密钥）时，它们选择某个 n 值作为安全参数；假设这个值被任何攻击该方案的敌手知道。那么敌手的运行时间（以及诚实方的运行时间）以及敌手的成功概率将都被看作 n 的函数。于是：

(1) 将“可行的策略”或者“有效的算法”的概念，和在以 n 为参数的多项式时间内运行的概率算法等同看待。（有时使用 PPT 来代表概率多项式时间。）这意味着，对于一些常量 a, c ，安全参数为 n ，该算法运行时间为 $a \cdot n^c$ 。要求诚实方在多项式时间内运行，并且仅关心对抗多项式时间的敌手。需要强调的是，敌手尽管要求在多项式时间内运行，但可能比诚实方的计算能力更强（并且运行更长时间）。而且，一个需要超多项式时间的攻击策略不被认为是具有现实意义的威胁（所以基本上被忽略）。

(2) 将“小的成功概率”的概念，和成功概率小于任何以 n 为参数的多项式倒数等同看待，这意味着对于每个常量 c ，当 n 的值足够大，敌手成功的概率小于 n^{-c} （参见定义 3.4）。比任何多项式的倒数增长得都慢的函数被称为可忽略的（negligible）。

因此，对渐进安全的定义一般采用下面的形式：

如果每个 PPT 敌手以可忽略的概率成功攻破一个方案，那么该方案是安全的。

尽管从理论上看是很清晰的（因为能够确实地说出一个方案是否安全），但是要理解渐进方法仅仅只是在 n 足够大的情况下才“保证安全”，下面的例子会说明这一点。

例 3.2 假设我们有一个方案是安全的，则它可能是这样的，一个运行了 n^3 分钟的敌手能

够成功“攻破该方案”的概率为 $2^{40} \cdot 2^{-n}$ （这是一个可忽略的关于 n 的函数）。当 $n \leq 40$ ，意味着一个运行 40^3 分钟（大约 6 周）的敌手能够攻破该方案的概率为 1，所以那样的 n 值不是很有用。甚至当 $n = 50$ ，一个运行 50^3 分钟（大约 3 个月）的敌手能够攻破该方案的概率约为 $1/1000$ ，这是不能被接受的。从另一个方面，当 $n = 500$ 时，一个运行时间超过200年的敌手攻破该方案的概率大概仅仅为 2^{-500} 。◇

前面的例子指出，可以将一个更大的安全参数看作提供了“更高的”安全级别。就大部分情况而言，安全参数决定了诚实方使用的密钥长度，因此有个类似的概念，即密钥越长，安全程度越高。通过使用一个更大的安全参数值来“增强安全性”的能力有着很重要的实用意义，因为它能够使诚实方抵御敌手计算能力的增强以及算法的进步。下面的例子将充分反映这一点是如何在实际中体现的。

例 3.3 让我们来看一下，在实际中更快的计算机的可用性对安全的影响。假设有一个密码学方案，诚实方被要求运行 $10^6 \cdot n^2$ 个时钟周期，同时，运行 $10^8 \cdot n^4$ 个周期的敌手能够成功“攻破”该方案的概率为 $2^{20} \cdot 2^{-n}$ （该例子中的数字都是经过设计的，使得计算更简单，并不等同于任何一种存在的密码方案）。

假设所有的各方都是用 1 兆赫的计算机，并且 $n = 50$ 。则诚实方运行 $10^6 \cdot 2500$ 个周期，或2.5秒，同时，一个运行 $10^8 \cdot (50)^4$ 个周期，或者大概一周的敌手，能够攻破该方案的概率只有 2^{-30} 。

现在假设一台 16 兆赫的计算机是可用的，并且所有各方都升级更新。诚实方能够将 n 增长到100（需要生成一个新的密钥），运行时间仍然被改进到0.625秒（即 $10^6 \cdot 100^2$ 个周期，速度为 $16 \cdot 10^9$ 周期/秒）。相比之下，敌手现在不得不运行 10^7 秒，或者超过16周，来实现成功概率 2^{-80} 。一个运行速度更快的计算机的影响就是使得敌手的工作更加困难。◇

渐进方法的优点就是不需要依赖于任何特定的假设，比方说，敌手使用机器的类型（这是计算复杂性理论中丘奇—图灵论题的一个结果，从根本上阐述了所有有足够能力的计算设备之间的相对速度是多项式相关的）。另一个方面，正如上面的例子证明的，在实际中，准确地理解一个特殊的渐进安全方案意味着什么水平的具体安全是有必要的。这是因为，诚实方必须选择一个具体的 n 值来使用，并且不能依赖于对“足够大的 n 值”的保证。决定安全参数值的任务很复杂，依赖于考虑之中的方案以及其他考虑。幸运的是，将渐进安全的保证转换为一个具体安全的保证往往相对简单。

从这里开始，将只使用渐进方法。尽管如此，正如上面的例子说明的那样，本书中所有的结论都能够转换为具体的安全结论。

一个技术注释。正如我们提到的，将敌手和诚实方的运行时间看作是 n 的函数。为了和算法以及复杂性理论中的标准约定保持一致，即一个算法的运行时间被衡量为一个输入长度的函数，必要的时候把安全参数表示为一进制形式 1^n （即 n 个1的字符串），提供给敌手和诚实方。

松弛的必要性。如前所述，计算安全在完美安全的概念中引入了两种松弛：首先，安全性仅仅抵抗有效的（即多项式时间的）敌手；其次，允许小的（即可忽略的）成功概率。这两种对概念的松弛对实现实践中的密码方案非常必要，特别是避开了完善保密加密的消极结

论。现在非正式地讨论其原因。假设有一个加密方案，密钥空间 $\mathcal{K}$ 的大小比消息空间 $\mathcal{M}$ 小得多（根据前一章所述，这意味着该方案不能够提供完善保密）。无论该加密方案是如何构造的，都有两种攻击可以应用，这两种攻击在两个相反的极端：

(1) 给定一份密文 c ，一个敌手通过使用所有的密钥 $k \in \mathcal{K}$ ，能够破译 c 。这样将会给出一列 c 对应的所有可能消息的清单。因为该清单不可能含有所有的 $\mathcal{M}$ （因为 $|\mathcal{K}| < |\mathcal{M}|$ ），这样会泄露已加密消息的一些信息。

再者，假设敌手执行了已知明文攻击，并且掌握了密文 $c_1, \dots, c_\ell$ ，分别对应着消息 $m_1, \dots, m_\ell$ 。该敌手能够反复地尝试使用所有可能的密钥，解密每个密文，直到找到一个密钥 k ，对于所有的 i ，满足 $Dec_k(c_i) = m_i$ 。很大概率上，这个密钥是唯一的，在这种情况下，该敌手已经找到了诚实方正在使用的密钥^③。因此，随后使用这个密钥就变得不安全了。

(2) 再次考虑这种情况，敌手掌握了密文 $c_1, \dots, c_\ell$ ，分别对应着消息 $m_1, \dots, m_\ell$ 。该敌手能够随机猜一个密钥 $k \in \mathcal{K}$ ，然后检查看看对于所有的 i ，是否 $Dec_k(c_i) = m_i$ 。如果满足，再次期望 k 有很大概率就是诚实方正在使用的密钥。

这里，敌手在必要的恒定时间内运行，成功的概率非零（尽管非常小），约为 $1/|\mathcal{K}|$ 。

如果希望用单个短密钥来加密很多消息，要实现安全性，只有限制敌手的运行时间（以至于敌手没有时间来执行蛮力搜索），并且不认为非常小的成功概率是一个攻破（所以第二种“攻击”被排除）。因此，前面提到的两种放宽是必要的。上面的讨论得出的一个额外结果，就是在任何加密方案中，密钥空间必须足够大，以至于敌手不能够遍历。更正式地，密钥空间的规模必须为安全参数的超多项式。

3.1.2 有效的算法和可忽略的成功概率

上节概述了本书使用的渐进安全方法。那些未接触过算法或者复杂性理论的读者可能对“多项式时间”，“概率（或随机）算法”，或者“可忽略概率”这些概念不适应，对渐进方法感到困惑。本节将更加详细和更加正式地重述这种（渐进）方法。那些对上节内容已很了解的读者，建议直接跳到3.1.3节，必要时再回来查阅。

有效的计算。前面已经定义有效的计算是指能够在“概率多项式时间”（PPT）内执行的计算。一个算法 A 被认为在多项式时间内运行是指：如果存在一个多项式 $p(\cdot)$ ，使得对于每个输入 $x \in \{0, 1\}^*$ ， $A(x)$ 的计算最多在 $p(|x|)$ 个步骤内终止（这里 $|x|$ 表示的是字符串 x 的长度）。一个概率算法具有“抛硬币”的能力，这是一种比喻，说明该算法可以获得一个随机源，该随机源产生无偏差的随机比特，其中每个比特都是相互独立的，为1的概率为 $1/2$ ，为0的概率也是 $1/2$ 。同样的，一个随机算法可视为除了输入，还赋予了一个随机带上的均匀分布的“足够长”的比特字符串。当考虑一个概率多项式时间算法，该算法运行时间为 p ，输入长度为 n ，一个长度为 $p(n)$ 的随机字符串将是足够的，因为该算法在被分配的时间内，只能使用 $p(n)$ 个随机比特。

那些对复杂性理论或者算法熟悉的读者将会意识到，将有效的计算和（概率）多项式时间计算等同起来的想法对密码学来说，并不是唯一的。（概率）多项式时间算法最主要

^③ 技术上说，密钥可能实际上不唯一（举个例子，可能相同一个密钥被不止一个字符串表示）。尽管如此，如果一个等价的密钥将被找到，效果是相同的。

的优点就是给出了一类在组成上是封闭的算法，意思是说，一个多项式时间的算法 A ，将另外的多项式时间算法 A' 作为一个子函数来运行，则 A 也能够多项式时间内运行。除了这个有用的事实，限制敌手在多项式时间内运行并没有其他内在的特殊原因，并且本书基本上所有的结论，都将公式化地表示为敌手的运行时间为 $n^{O(\log n)}$ （诚实方仍然运行在多项式时间内）。

在继续讨论之前，要解决一个问题：为什么要考虑概率多项式时间算法而不是确定性多项式时间算法。对此，有两个主要的原因。首先，随机性对密码学来说是必要的（例如，为了选择随机的密钥等），所以诚实方必须是概率的。给定了这种情况，很自然地考虑到攻击方也是概率的。考虑概率算法的第二个原因是，“抛硬币”的能力可能提供了额外的力量。因为使用有效计算的概念来作为现实中的敌手模型，让这个类型尽可能地扩大是很重要的（在保证是现实的情况下）。

额外提醒一下：概率多项式时间的敌手是否比确定性多项式时间的敌手能力更强大，这个问题还没有解决。事实上，最近的复杂性理论中的结论指出随机性没有起到帮助作用。尽管如此，这个结论没有影响将敌手建模成概率算法的做法，并且这只能提供更强的保证，也就是说，任何安全方案若能抵抗概率多项式时间敌手，则也能抵抗确定性多项式时间的敌手。

生成随机性。前面已将各方建模成概率多项式时间算法，如前所述，密码系统只有在随机性可用时，才是可能的（如果秘密密钥不能够随机生成，则一个敌手就能够依照诚实方相同的步骤，来获得他们的“秘密密钥”。回想 Kerckhoffs 准则，假设这个过程是已知的）。有了这个前提，某些人可能怀疑是否有可能在计算机上实现“扔硬币”来得到概率计算。

实际中，很多方法能够获得“随机比特”。一个解决方案就是使用硬件随机数发生器，它能够根据特定的物理现象，比如热/电噪声或者是放射衰变，来生成随机比特流。另外一种可能性是，使用软件随机数发生器，它能够根据不可预料的行为，比如击键之间的间隔时间、鼠标的移动、硬盘访问时间等，来生成随机比特流。一些现代操作系统提供了这种类型的函数。在这两种情况下，依赖的不可预料事件（自然的或者是依赖用户的）不可能直接产生均匀分布的比特，所以对初始比特流的进一步处理是需要的。对此，技术上较复杂，一般很难理解，并且也超出了本书的范围。

必须非常小心地选择随机比特，因为使用设计拙劣或者不恰当的随机数发生器往往会让一个好的密码系统在攻击面前变得脆弱。特别地，要使用一个为“密码学用途”而设计的随机数发生器，而不是一个为“通用目的”而设计的随机数发生器，后者可能对某些应用来说可以，但是对密码学应用来说是不行的。作为一个具体的例子，在 C 编程语言中，使用 `random()` 函数是个很差的想法，因为它根本就不随机。同样地，当前时间（即使精确到毫秒）并不是非常随机，不能作为一个秘密密钥的基础。

可忽略的成功概率。现代密码学允许：能以非常小的概率被攻破的方案，仍被认为是“安全的”。考虑多项式运行时间是可行的，类似地，认为概率为多项式倒数时是显著的。因此，对某个（正）多项式 p ，如果一个敌手能够成功地攻破该方案的概率为 $1/p(n)$ ，则该方案被认为是不安全的。但是，对任意多项式 p ，如果攻破该方案的概率渐进小于 $1/p(n)$ ，则认为该方案是安全的。这是因为敌手成功的概率太小，被认为是无趣的。这样的概率叫做可忽略的，定义如下。

定义 3.4 函数 f 为可忽略的, 如果对于每个多项式 $p(\cdot)$, 存在一个 N , 使得对于所有的整数 $n > N$, 都满足 $f(n) < \frac{1}{p(n)}$ 。

一个和上面等价的公式化的表述是, 对所有常量 c , 存在一个 N , 使得对于所有的 $n > N$, 满足 $f(n) < n^{-c}$ 。为了方便记忆, 上面的表述也可以陈述如下: 对于任意多项式 $p(\cdot)$ 以及所有足够大的 n 值, 满足 $f(n) < \frac{1}{p(n)}$ 。这当然是等价的陈述。通常将一个可忽略函数表示为 negl 。

例 3.5 函数 2^{-n} , $2^{-\sqrt{n}}$ 以及 $n^{-\log n}$ 都是可忽略的。但是, 它们接近零的速度是非常不同的。为了说明此点, 下面计算什么样的 n 值, 会使得每一个函数的值都小于 10^{-6} 。

(1) $2^{20} = 1048576$, 因此对于 $n \geq 20$, 有 $2^{-n} < 10^{-6}$ 。

(2) $2^{\sqrt{400}} = 1048576$, 因此对于 $n \geq 400$, 有 $2^{-\sqrt{n}} < 10^{-6}$ 。

(3) $32^5 = 33554432$, 因此对于 $n \geq 32$, 有 $n^{-\log n} < 10^{-6}$ 。

从上面的示例可能产生一个印象, 就是 $n^{-\log n}$ 比 $2^{-\sqrt{n}}$ 更快地接近零。但这并不正确; 对于所有的 $n > 65536$, 满足 $2^{-\sqrt{n}} < n^{-\log n}$ 。尽管如此, 这还是不能说明当 n 的值为成百上千时, 敌手成功的概率 $n^{-\log n}$ 会比概率 $2^{-\sqrt{n}}$ 更高。◇

可忽略的成功概率的一个技术上的优势是, 它们也遵循某个闭包的特性。下面是个简单的练习。

命题 3.6 令 negl_1 和 negl_2 为可忽略函数。则

(1) 函数 negl_3 被定义为 $\text{negl}_3(n) = \text{negl}_1(n) + \text{negl}_2(n)$, 则该函数是可忽略的。

(2) 对于任何正多项式 p , 函数 $\text{negl}_4(n)$ 被定义为 $\text{negl}_4(n) = p(n) \cdot \text{negl}_1(n)$, 则该函数是可忽略的。

上面命题的第(2)部分, 意味着如果在某个实验中, 一个特定的事件发生的概率可以忽略, 那么该实验重复多项式次, 该事件发生的概率还是可忽略的。举个例子, 如果抛硬币 n 次, 都是“正面”的概率可忽略。这个意味着, 如果抛 n 个硬币多项式次, 则任何一次实验中, n 个“正面”的概率也是可以忽略的。(正式地, 该结论已经被使用一致限来证明了, 在命题中 A.7 陈述。)

以可忽略的概率发生的事件, 对所有实践中的用途来说(至少对于足够大的 n 值), 能够被安全地忽略, 理解这一点是很重要的。其重要性值得反复强调:

以可忽略的概率发生的事件, 因为几乎不可能发生, 所以对所有实践用途, 它们可被忽略。因此, 一个攻破密码方案的事件若发生的概率是可忽略的, 则是不重要的。

为了避免读者对敌手在某个微小(但是非零)概率下, 能够攻破一个给定方案的事实感觉不适应, 这里给出一个例子, 在某个微小(但是非零)概率下, 当执行该安全方案时, 诚实方被小行星撞到。更温和地解释, 但是意思相同: 某个诚实方的硬盘以非零的概率失效, 从而清除了秘密密钥。因此, 担心发生概率足够小的事件是没有意义的。

对渐进安全的总结。任何安全定义由两部分组成: 对该方案的“攻破”的定义, 和对敌

手能力的详细说明。敌手的能力和很多问题（比如，在加密情况下，假设是唯密文攻击，还是选择明文攻击）相关。但是，考虑到敌手的计算能力，从现在开始，将敌手建模成有效的和概率多项式时间的（意味着只有“可行的”攻击策略被考虑）。定义往往被公式化，以至于以可忽略的概率发生攻破不被认为是重要的。因此，任何安全定义的一般框架如下：

如果每个概率多项式时间的敌手 A 执行某个特定类型的攻击，在这次攻击（成功也被很好地定义过）中， A 成功的概率是可忽略的，那么这个方案是安全的。

这样的定义是渐进的，因为对于小的 n 值，敌手成功的概率很高是有可能的。为了说明更多细节，在上面的陈述中，将使用到上面陈述的对“可忽略”的完全定义：

如果每个概率多项式时间的敌手 A 执行某个特定类型的攻击，对于每个多项式 $p(\cdot)$ ，存在一个整数 N ，对 $n > N$ ， A 在这次攻击中成功的概率小于 $\frac{1}{p(n)}$ ，则这个方案是安全的。

注意，对 $n \leq N$ 时，没有保证。

3.1.3 规约证明

如前所述，如果给予足够的时间，一个计算安全（而不是完美安全）的密码方案往往能够被攻破。为了无条件地证明某方案是计算安全的，需要证明攻破该方案所需要时间的下界。特别地，需要证明该方案不能被任何多项式时间算法攻破。不幸的是，当前的现状是还不能证明该类型的下界。事实上，对任何现代加密方案的无条件的证明，需要突破看起来遥不可及的复杂性理论中的结论^④。这看上去别无选择，只能简单地假设给定的方案是安全的。正如在 1.4 节中讨论的那样，但是，这是个非常不理想的方法，历史证明这个方法很危险。

不用假设给定的密码学构造是安全的，取而代之的策略是，假设某个低层次难题很难解决，然后证明基于这个假设，构造方案是安全的。在 1.4.2 节中，已经很详细地解释了为什么这个方法更合适，所以在这里，不再重复这些论述了。

只要构造所依赖的问题是困难的，则给定的密码学构造方案就是安全的，对这一逻辑关系进行证明的一般方式是，陈述一个显式规约，说明如何将任何概率不可忽略的且成功“攻破”该构造方案的有效敌手 A ，转换为成功解决该难题的有效算法 A' 。（事实上，这是在本书中唯一使用的证明方法。）因为这个方法是如此重要，所以详细地浏览一遍证明步骤的概要。从假设开始，假设某个难题 X 不能通过任何多项式时间算法以不是可忽略的概率被解决（从某种准确定义的角度来说）。想要证明某个密码构造 Π 是安全的（同样，从某种准确定义的角度来说）。做法如下：

(1) 指定某个有效的（即概率多项式时间）敌手 A 攻击 Π 。将敌手的成功概率表示成为 $\epsilon(n)$ 。

(2) 构造一个叫做“规约”的有效算法 A' ，该算法将敌手 A 作为子程序来使用，试图解决难题 X （如图 3.1 所示）。这里很重要的一点是， A' 对 A 是“如何”工作的一无所知； A' 知道的唯一的事情就是 A 想要攻击 Π 。所以，指定难题 X 的某个输入实例 x ，算法 A' 将会对 A 模拟出一个 Π 的实例，满足：

①就 A 而言，它和 Π 交互。更正式地说，当 A 被作为子程序被 A' 运行时， A 的分布应该和当 A 自身和 Π 交互时的分布是相同的（或至少是很接近的）。

④ 对那些熟悉 P 和 NP 问题的读者，我们标注：对任意消息长度大于密钥长度的加密方案的安全性的无条件证明意味着要证明 $P \neq NP$ 。

②如果 A 成功“攻破了”由 A' 模拟的 Π 的实例，则这将允许 A' 解决给出的难题实例 x ，成功的概率至少为多项式的倒数，即 $1/p(n)$ 。

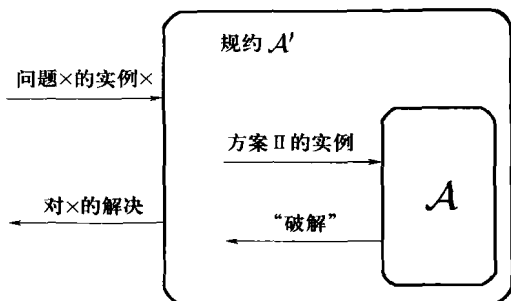


图 3.1 通过规约对安全性的证明的高层次概况

(3) 总起来考虑，(2)①和(2)②意味着，如果 $\epsilon(n)$ 不是可忽略的，则 A' 解决难题 X 的概率为不可忽略的概率 $\epsilon(n)/p(n)$ 。因为 A' 是有效的，将 PPT 敌手 A 作为子程序来运行，意味着存在一个有效的算法能够以不可忽略的概率解决 X ，与原假设矛盾。

(4) 因而结论是，给定一个关于 X 的假设，不存在有效的敌手 A 能够以不可忽略的概率成功攻破 Π 。换句话说， Π 是计算上安全的。

当下一节给出这种证明的例子时，这一点将会更加清楚。

3.2 定义计算安全的加密

上一节介绍的背景已准备好定义对称密钥加密的计算安全。首先，重新定义对称密钥加密的语法；这个和在第 2 章中介绍的语法本质上是相同的，唯一不同是现在要明确考虑安全参数。现在，令消息空间默认为所有（有限长度的）二进制字符串的集合 $\{0,1\}^*$ 。

定义 3.7 一个对称密钥加密方案是概率多项式时间算法($\text{Gen}, \text{Enc}, \text{Dec}$)的三元组：

(1) 密钥生成算法 Gen 的输入为安全参数 1^n ，输出密钥 k ；记为 $k \leftarrow \text{Gen}(1^n)$ （要强调 Gen 是一个随机算法）。不失一般性，假设任何由 $\text{Gen}(1^n)$ 输出的密钥 k ，都满足 $|k| \geq n$ 。

(2) 加密算法 Enc 将密钥 k 和明文消息 $m \in \{0,1\}^*$ 作为输入，并且输出一个密文 $c^\circledast$ 。因为 Enc 可能是随机化的，记为 $c \leftarrow \text{Enc}_k(m)$ 。

(3) 解密算法 Dec 将密钥 k 和密文 c 作为输入，输出一份消息 m 。假设 Dec 是确定性的，所以记为 $m := \text{Dec}_k(c)$ 。

需要满足的是：对每个 n 、每个由 $\text{Gen}(1^n)$ 输出的密钥 k 、每个 $m \in \{0,1\}^*$ ，都满足 $\text{Dec}_k(\text{Enc}_k(m)) = m^\circledast$ 。

⑤ 作为一个技术条件， Enc 允许在以 $|k| + |m|$ （即输入的总长度）为参数的多项式时间内运行。如果仅仅包含 $|m|$ ，那么加密单个比特将不会占用多项式时间，如果仅仅包含 $|k|$ ，则不得不对要加密的消息长度设置一个先验限制。虽然有必要，从这里开始忽略该技术细节。

⑥ 给定这些，我们对 Dec 是确定的这一假设是不失一般性的。

如果 $(\text{Gen}, \text{Enc}, \text{Dec})$ 满足：对每个由 $\text{Gen}(1^n)$ 输出的密钥 k ，算法 Enc_k 只对消息 $m \in \{0, 1\}^{\ell(n)}$ 有定义，则假设 $(\text{Gen}, \text{Enc}, \text{Dec})$ 是一个消息长度为定长 $\ell(n)$ 的对称密钥加密方案。

需要强调在本章和下一章， $\text{Gen}(1^n)$ 都是均匀随机地选择 $k \leftarrow \{0, 1\}^*$ 。

3.2.1 安全的基本定义

定义对称密钥加密的安全性的标准方式有很多种，主要的不同在于：对攻击中敌手的能力的假设。本节从说明最基本的安全概念开始，即抵抗唯密文攻击这种弱安全，敌手只能观察到“单个”密文，在本章后面，会考虑更强的安全定义。

定义的动机。正如在第 1 章讨论的，任何对安全的定义由两个不同的部分组成：对假定的敌手能力的详细说明，对什么是“攻破”方案的描述。在定义开始之前，考虑一种情况，一个窃听敌手观察到单个消息，或者被给予希望“破解”的单个密文。这类敌手较弱，但是这正是前面章节所考虑的敌手类型（后面章节中会遇到更强的敌手）。当然，正如在前一节中解释的那样，目前只对那种有计算能力限制和运行时间被限制在多项式时间内的敌手感兴趣。

尽管对敌手能力做出了两个重要假设（即仅仅是窃听，在多项式时间内运行），但是这里对敌手的策略没有做出任何假设。这一点对获得有意义的安全概念是很关键的，因为预测所有可能的策略是不可能的。因此，要预防所有可被敌手执行的策略，这些敌手包含在已定义敌手类型中。

定义对加密算法的“攻破”并不简单，已经在 1.4.1 节和前面一章中详细地讨论过这个问题。因此，这里只需要回忆一下，定义背后的思想是，敌手应该不能从密文中得到关于明文的任何信息。对“语义安全”定义的直接形式化正是这个概念，这是第一个被建议的加密安全的定义。不幸的是，语义安全的定义复杂并且困难。幸运的是，有个等价的定义，即“不可区分性”，它更简单。因为定义是等价的，所以可以从更简单的不可区分性的定义着手，确信获得的安全保证正是想从语义安全中所期望的。（参见 3.2.2 节对这一点的进一步讨论。）

这里给出的不可区分性的定义从语法上说，和在定义 2.4 中给出的完善保密的定义几乎是相同的。（这提供了进一步的定义不可区分性的动机。）回忆定义 2.4 中，考虑一个实验 $\text{PrivK}_{A, \Pi}^{\text{eav}}$ ，一个敌手 A 输出两个消息 m_0, m_1 ，然后敌手被给予其中一个消息加密后的密文，该消息是随机选择的并使用一个随机生成的密钥。定义陈述为：如果没有敌手 A 能够以任何不同于 $1/2$ 的概率判断出消息 m_0, m_1 中是哪一个被加密的（ $1/2$ 即随机猜测的概率），则方案 Π 是安全的。

这里保持 $\text{PrivK}_{A, \Pi}^{\text{eav}}$ 基本上不变（除了某些下面讨论的技术区别以外），引入对定义的两个关键改变：

- (1) 这里仅仅考虑在多项式时间内运行的敌手，尽管定义 2.4 中考虑无计算能力限制的敌手。
- (2) 这里要求敌手可能判断出加密的消息的概率大于 $1/2$ 的程度是可忽略的。

如上节的大量讨论，上面的条件放宽组成了计算安全的核心元素。

至于在实验 $\text{PrivK}_{A, \Pi}^{\text{eav}}$ 中的不同，一个区别纯粹是语法上的，而另一个区别是因为技术问题。最突出的区别是，现在用一个安全参数 n 来将实验参数化。用 n 的函数来衡量敌手 A 的运行时间和成功的概率。用 $\text{PrivK}_{A, \Pi}^{\text{eav}}(n)$ 来表示给定安全参数值的实验，并且写为

$$\Pr[\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}} = 1] \quad (3.1)$$

来表示在实验 $\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}}(n)$ 中， $\mathcal{A}$ 输出1的概率。值得注意的是当 $\mathcal{A}$ 和 Π 固定时，等式(3.1)是 n 的函数。

实验 $\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}}$ 中的第二个区别是，这里需要敌手输出两个长度相等的消息 m_0, m_1 。从理论的角度讲，该限制是有必要的，因为这里要求加密方法应该能够用来加密任意长度的消息。如果愿意放弃这个要求，就可以不要这个限制，如同完善保密的情况中所做的那样（参见练习 3.3 以及 3.4）。该限制对在实际应用中的大部分加密方案都是适用的，这些加密方案中不同长度的消息导致了不同长度的密文，所以，如果允许输出不同长度的消息，则敌手能够很轻易地分辨出哪一个消息是经过加密的。

大部分实际中的加密方案都没有隐藏加密消息的长度。为了防止一个消息的长度可能会泄露一些敏感信息（比如，当消息指出一个雇员薪水是几位数时），必须在加密之前，填充输入使之达到某个固定长度。在后面将不再重申这一点。

窃听者存在情况下的不可区分性。现在给出正式的定义，从上面概述的实验开始。该实验的定义对任何对称密钥加密方案 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ ，任何敌手 $\mathcal{A}$ ，以及对任何安全参数 n 的值都适用：

窃听不可区分实验 $\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}}(n)$ ：

- (1) 给定输入 1^n 给敌手 $\mathcal{A}$ ， $\mathcal{A}$ 输出一对长度相等的消息 m_0, m_1 。
- (2) 运行 $\text{Gen}(1^n)$ 生成一个密钥 k ，选择一个随机比特 b ， $b \leftarrow \{0, 1\}$ 。一个密文 $c \leftarrow \text{Enc}_k(m_b)$ 被计算并且给 $\mathcal{A}$ 。把 c 叫做挑战密文。
- (3) $\mathcal{A}$ 输出一个比特 b' 。
- (4) 该实验的输出被定义为：如果 $b = b'$ ，则为1，否则为0。如果 $\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}}(n) = 1$ ，则称 $\mathcal{A}$ 成功。

这里除了要求长度是相等的以外，加密消息 m_0 和 m_1 的长度是没有限制的。（当然，因为敌手被限制在多项式时间内运行， m_0 和 m_1 的长度为以 n 为参数的多项式。）如果 Π 是一个定长方案，消息长度为 $\ell(n)$ ，那么上面的实验要被修改，需要 $m_0, m_1 \in \{0, 1\}^{\ell(n)}$ 。

不可区分性的定义说明了：如果在上面的实验中，任何 PPT 敌手的成功概率的最大值高于 $1/2$ 的部分可忽略，则一个加密方案是安全的。（注意，可以 $1/2$ 的概率成功是很容易的，因为可以通过输出一个随机比特 b' ，困难的地方是要比这个做到更好。）现在为下面的定义做好了准备。

定义 3.8 如果对于所有的概率多项式时间敌手 $\mathcal{A}$ ，存在一个可忽略函数 negl 使得：

$$\Pr[\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$$

则一个对称密钥加密方案 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ 具备在窃听者存在的情况下不可区分的加密，其中概率的来源是 $\mathcal{A}$ 的随机性以及实验的随机性（选择密钥、随机比特 b ，以及在加密过程中使用到的任何随机性）。

该定义是对“所有的”概率多项式时间的敌手定量分析，意味着对所有“可行的”策略（可行的策略和那些能够在多项式时间执行的策略是等价的）而言，都是安全的。敌手的输入

被限制为（单个）密文，以及敌手和发送方或接收方没有更多的交互这一事实，隐含着敌手仅仅只有窃听的能力。（稍后会看到，允许更多的交互将会导致更强的敌手。）现在，该定义简短地说明任何敌手 $\mathcal{A}$ 将成功猜出哪一个是加密消息的概率的最大值超出一个单纯的猜测（猜对的概率为 $1/2$ ）的部分是可忽略的。

注意，敌手是被允许选择消息 m_0 和 m_1 。因此，即使知道 c 是这些明文消息中的某个消息加密的密文，仍然无法判断出是从哪一个加密来的。这是个非常强的保证，有很重要的实际意义。举个例子，考虑一个场景，敌手知道被加密的消息要么是“今天攻击”，或者是“不攻击”。很明显，不希望敌手知道是哪一个消息被加密，尽管已经知道密文是来自两种消息中的某一个。

一个等价的公式化表述。定义 3.8 阐述了一个窃听敌手不能做到：判断是哪一个明文被加密的概率明显大于随机猜测成功的概率。一个公式化该定义的等价方式是：陈述每个敌手，在观察到 m_0 的密文或者 m_1 的密文（对任何长度相等的 m_0 和 m_1 ）时，其行为是相同的。因为 $\mathcal{A}$ 只输出单个比特，“行为相同”意味着在每种情况下，输出1的概率几乎相等。为将其形式化，同上定义 $\text{PrivK}_{\mathcal{A},n}^{\text{eav}}(n,b)$ ，唯一不同是使用一个固定比特 b （而不是随机的选择）。此外，用 $\text{output}(\text{PrivK}_{\mathcal{A},n}^{\text{eav}}(n,b))$ 来表示 $\mathcal{A}$ 在 $\text{PrivK}_{\mathcal{A},n}^{\text{eav}}(n,b)$ 中的输出比特 b' 。接下来的定义从根本上说明了 $\mathcal{A}$ 无法判断是在实验 $\text{PrivK}_{\mathcal{A},n}^{\text{eav}}(n,0)$ 中运行还是在 $\text{PrivK}_{\mathcal{A},n}^{\text{eav}}(n,1)$ 中运行。

定义 3.9 如果对所有概率多项式时间的敌手 $\mathcal{A}$ ，存在一个可忽略函数 negl 使得：

$$|\Pr[\text{output}(\text{PrivK}_{\mathcal{A},n}^{\text{eav}}(n,0)) = 1] - \Pr[\text{output}(\text{PrivK}_{\mathcal{A},n}^{\text{eav}}(n,1)) = 1]| \leq \text{negl}(n)$$

则对称密钥加密方案 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ 具有窃听者存在的情况下不可区分性。

该定义和定义 3.8 等价的事实留作练习。

3.2.2 *定义的属性

通过要求敌手无法从密文中掌握任何关于明文的部分消息，给出了安全加密的定义。但是，不可区分性的实际定义看起来非常不同。如前所述，定义 3.8 实际上和“语义安全”是等价的，语义安全将部分消息不能被得到这一直觉概念进行了形式化地描述。这里将不证明等价性，但将证明两个声称（Claim）展示了不可区分性意味着语义安全的弱版本。为了比较，（从根本上）表述语义安全的完整定义。读者可以参考文献[65]的 5.2 章得到更深入的讨论以及完整的等价性证明。

不可区分性意味着一个随机选择的明文中没有单个比特能够被猜出的概率显著大于 $1/2$ 。下面，我们用 m^i 来表示 m 中的第 i 个比特，如果 $i > |m|$ ，令 $m^i = 0$ 。

声称 3.10 令 $(\text{Gen}, \text{Enc}, \text{Dec})$ 为一个对称密钥加密方案，在窃听者存在的情况下，它具备不可区分的加密，那么对于所有的概率多项式时间敌手 $\mathcal{A}$ 和所有 i ，存在一个可忽略函数 negl ，使得

$$\Pr[\mathcal{A}(1^n, \text{Enc}_k(m)) = m^i] \leq \frac{1}{2} + \text{negl}(n)$$

其中 m 是从 $\{0,1\}^n$ 中均匀随机选择的，并且概率来源于 $\mathcal{A}$ 的随机性、 m 和 k 选择的随机性以及

在加密过程中使用到的任何随机性。

证明 证明背后的思想是给出 $\text{Enc}_k(m)$, 如果猜出 m 的第 i 个比特是可能的, 则区分明文消息 m_0 和 m_1 的密文也是可能的, 其中 m_0 的第 i 个比特等于 0, m_1 的第 i 个比特等于 1。特别地, 给定一个密文 c , 尝试计算出对应明文的第 i 个比特。如果这个计算表明第 i 个比特为 0, 则猜测是 m_0 被加密; 如果表明第 i 个比特为 1, 则猜测是 m_1 被加密。正式的描述是, 给定 $\text{Enc}_k(m)$, 对某个函数 $\varepsilon(n)$, 如果存在一个敌手 $\mathcal{A}$ 能够在概率至少为 $1/2 + \varepsilon(n)$ 的情况下, 猜出 m 的第 i 个比特, 则存在一个敌手能够在不可区分性实验 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ 中成功的概率为 $1/2 + \varepsilon(n)$ 。如果 Π 是不可区分的加密, 则 $\varepsilon(\cdot)$ 必须是可忽略的。

具体而言, 令 $\mathcal{A}$ 是一个概率多项式时间敌手, 定义 $\varepsilon(\cdot)$ 如下:

$$\varepsilon(n) \stackrel{\text{def}}{=} \Pr[\mathcal{A}(1^n, \text{Enc}_k(m)) = m^i] - \frac{1}{2}$$

其中 m 是从 $\{0, 1\}^n$ 中均匀随机选择的。从这里开始, 为了视觉上更清晰, 不再明确指出输入 1^n 给 $\mathcal{A}$ 。令 $n \geq i$, 令 I_0^n 为所有长度为 n , 且第 i 个比特为 0 的字符串的集合, I_1^n 为所有长度为 n , 且第 i 个比特为 1 的字符串的集合。有

$$\Pr[\mathcal{A}(\text{Enc}_k(m)) = m^i] = \frac{1}{2} \cdot \Pr[\mathcal{A}(\text{Enc}_k(m_0)) = 0] + \frac{1}{2} \cdot \Pr[\mathcal{A}(\text{Enc}_k(m_1)) = 1]$$

其中, m_0 是从 I_0^n 中随机选择的, m_1 是从 I_1^n 中随机选择的。(上面的条件满足, 因为 I_0^n 和 I_1^n 每一个都正好包含 $\{0, 1\}^n$ 的一半。因此, m 在每个集合中的概率正好是 $1/2$ 。)

现在, 考虑下面的敌手 $\mathcal{A}'$, 窃听单个消息的加密:

敌手 $\mathcal{A}'$:

(1) 输入 1^n ($n \geq i$), 从指定的集合中均匀随机地选择 $m_0 \leftarrow I_0^n$ 和 $m_1 \leftarrow I_1^n$ 。输出 m_0, m_1 。

(2) 当接收到密文 c 时, 输入 c 给敌手 $\mathcal{A}$ 。如果 $\mathcal{A}$ 输出 0, 输出 $b' = 0$, 如果 $\mathcal{A}$ 输出 1, 输出 $b' = 1$ 。

因为 $\mathcal{A}$ 在多项式时间运行, 所以 $\mathcal{A}'$ 在多项式时间内运行。

使用实验 $\text{PrivK}_{\mathcal{A}', \Pi}^{\text{eav}}(n)$ 的定义 ($n \geq i$), 注意当且仅当 $\mathcal{A}$ 收到 $\text{Enc}_k(m_b)$ 时, 输出 b , 有 $b' = b$ 。所以

$$\begin{aligned} \Pr[\text{PrivK}_{\mathcal{A}', \Pi}^{\text{eav}}(n) = 1] &= \Pr[\mathcal{A}(\text{Enc}_k(m_b)) = b] \\ &= \frac{1}{2} \cdot \Pr[\mathcal{A}(\text{Enc}_k(m_0)) = 0] + \frac{1}{2} \cdot \Pr[\mathcal{A}(\text{Enc}_k(m_1)) = 1] \\ &= \Pr[\mathcal{A}(\text{Enc}_k(m)) = m^i] \\ &= \frac{1}{2} + \varepsilon(n) \end{aligned}$$

通过假设 $(\text{Gen}, \text{Enc}, \text{Dec})$ 在窃听者存在的条件下, 具备不可区分加密, 则 $\varepsilon(\cdot)$ 必须是可忽略的。(注意, 当 $n < i$ 时, 无论发生什么都不重要, 因为这里只关心渐进行为。)得证。 ■

粗略地说, 给定密文, PPT 敌手不能推测关于明文消息的任何函数, 而且不管发送消息的先验分布如何, 该条件都满足。形式化地定义这个要求并不简单。例如要计算明文消息 m 的第 i 个比特 (如同上面所考虑的), 若 m 从集合 I_0^n 中均匀地选取 (而不是从 $\{0, 1\}^n$ 中均匀地

选取), 则将非常容易计算。这里需要定义的是对某个函数 f , 如果一个接收到密文 $c = \text{Enc}_k(m)$ 的敌手能够计算出 $f(m)$, 则存在一个敌手能够在不给密文的(仅仅知道 m 的先验分布)情况下, 以同样的概率正确地计算出 $f(m)$ 。

在下一个声称中, 当 m 均匀随机地从某个集合 $S \subseteq \{0, 1\}^n$ 中选择时, 也满足上面的这些讨论。即如果明文是邮件信息, 则令 S 是有正确邮件头的英语消息的集合。由于考虑渐进情形, 故着手于无穷集合 $S \subseteq \{0, 1\}^*$ 。对于安全参数 n , 从 $S_n \stackrel{\text{def}}{=} S \cap \{0, 1\}^n$ 中均匀选出一个明文消息, 该集合默认为是非空明文消息的集合(即 S 的子集, S 中长度为 n 的字符串的集合)。作为一个技术上的条件, 需要假设能够有效且均匀地从 S_n 中采样字符串; 也就是说, 存在某个概率多项式时间算法, 对于输入 1^n , 输出 S_n 中一个均匀分布的元素。这样, 认为集合 S 是“有效可采样的”。这里将函数 f 限制为能够在多项式时间内计算。

声称 3.11 令 $(\text{Gen}, \text{Enc}, \text{Dec})$ 为一个对称密钥加密方案, 具备在窃听者存在的情况下不可区分加密, 那么对任意概率多项式时间敌手 $\mathcal{A}$, 存在一个概率多项式时间算法 $\mathcal{A}'$, 使得对所有的多项式时间可计算函数 f , 以及所有有效可采样的集合 S , 存在一个可忽略函数 negl , 满足:

$$|\Pr[\mathcal{A}(1^n, \text{Enc}_k(m)) = f(m)] - \Pr[\mathcal{A}'(1^n) = f(m)]| \leq \text{negl}(n)$$

其中 m 是从 $S_n \stackrel{\text{def}}{=} S \cap \{0, 1\}^n$ 中均匀随机地选取, 并且概率计算来源于对 m 和密钥 k 的选择、任何 $\mathcal{A}$ 、 $\mathcal{A}'$ 以及加密过程用到的随机性。

证明(概要) 仅仅陈述一个非正式的对该声称的证明概要。假设 $(\text{Gen}, \text{Enc}, \text{Dec})$ 具备不可区分性的加密。这就意味着, 对任何 $m \in \{0, 1\}^n$, 没有概率多项式时间的敌手 $\mathcal{A}$ 能够区分 $\text{Enc}_k(m)$ 和 $\text{Enc}_k(1^n)$ 。那么给定 $\text{Enc}_k(m)$, $\mathcal{A}$ 成功计算出 $f(m)$ 的概率, 与给定 $\text{Enc}_k(1^n)$, $\mathcal{A}$ 成功计算出 $f(m)$ 的概率, 两者几乎是相等的。否则, $\mathcal{A}$ 就能被用来区分 $\text{Enc}_k(m)$ 和 $\text{Enc}_k(1^n)$ 。(区分器很容易构造: 均匀随机地选择 $m \leftarrow S_n$, 这里假设 S 是有效可采样的, 输出 $m_0 = m, m_1 = 1^n$ 。当给定一个密文 c , c 要么是 m 的密文, 要么是 1^n 的密文, 以 c 为输入来调用 $\mathcal{A}$, 当且仅当 $\mathcal{A}$ 输出 $f(m)$ 时输出0。当给定 m 的密文的概率与 1^n 的密文的概率存在非可忽略的差别时, $\mathcal{A}$ 输出 $f(m)$, 则描述的区分器违背了定义3.9)。

上面的观察产生了下面的算法 $\mathcal{A}'$, 该算法不接收 $c = \text{Enc}_k(m)$, 但是同样能计算 $f(m)$: 输入安全参数 1^n , $\mathcal{A}'$ 选择随机密钥 k , $c \leftarrow \text{Enc}_k(1^n)$ 作为输入调用 $\mathcal{A}$, $\mathcal{A}$ 的输出作为 $\mathcal{A}'$ 的输出。以上, 当 $\mathcal{A}$ 作为一个子程序被 $\mathcal{A}'$ 调用时, 输出 $f(m)$ 的概率几乎等于收到 $\text{Enc}_k(m)$ 的概率。因此, $\mathcal{A}'$ 实现了该声称需要的属性。 ■

语义安全。对语义安全的完整定义, 比声称 3.11 证明的属性更一般。这里考虑了明文消息的任意分布, 也考虑了任意“外部的”关于明文的信息, 这些信息有可能通过其他的方式被“泄露”给敌手(比如, 因为相同的消息 m 也被用作其他用途)。同上, 用 f 来表示敌手试图计算出的明文函数。不考虑一个特定的集合 S (以及一个在 S 的子集中的均匀分布), 而是考虑一个任意分布 $X = (X_1, X_2, \dots)$, 对安全参数 n , 明文是根据分布 X_n 来选择的。要求 X 是有效可采样的, 这意味着存在一个 PPT 算法, 对输入 1^n , 输出一个元素, 该元素根据分布 X_n 来选择。另外, 所有的 n 和所有在 X_n 中的字符串都有相同的长度。最后, 除了 m 的加密之外, 对某个任意函数 h , 通过给予敌手 $h(m)$, 来模拟“外部的”关于 m 信息被“泄露”给敌手。即:

定义 3.12 一个对称密钥加密方案($\text{Gen}, \text{Enc}, \text{Dec}$)。如果对于每个概率多项式时间算法 $\mathcal{A}$, 存在一个概率多项式时间算法 $\mathcal{A}'$, 使得对所有的有效可采样的分布 $X = (X_1, \dots)$ 以及所有多项式时间可计算的函数 f 和 h , 存在一个可忽略函数 negl 满足:

$$|\Pr[\mathcal{A}(1^n, \text{Enc}_k(m), h(m)) = f(m)] - \Pr[\mathcal{A}'(1^n, h(m)) = f(m)]| \leq \text{negl}(n)$$

则称其为在窃听者存在的情况下是语义安全的。其中 m 是根据 X_n 分布来选择的, 概率计算来源于对 m 和密钥 k 的选择、任何 $\mathcal{A}$ 、 $\mathcal{A}'$ 以及加密过程用到的随机性。

给予敌手 $\mathcal{A}$ 密文 $\text{Enc}_k(m)$ 和历史函数 $h(m)$, 这里后一个函数表示的是敌手可能掌握的关于明文 m 的“外部的”知识。敌手 $\mathcal{A}$ 试图去猜测 $f(m)$ 的值。算法 $\mathcal{A}'$ 也试图去猜测 $f(m)$ 的值, 但是仅仅只给予 $h(m)$ 。安全需求陈述的是, 当给予密文时, $\mathcal{A}$ 猜测 $f(m)$ 的成功概率和某个不给予密文的算法 $\mathcal{A}'$ 本质上是一样的。因此, 密文 $\text{Enc}_k(m)$ 没有泄露任何关于 $f(m)$ 的信息。

定义 3.12 组成了一个非常强且令人信服的对安全保证的形式化描述, 这个安全保证由加密方案提供。可论证地, 这个比不可区分性 (仅仅考虑两个明文, 没有提到外部知识或者任意分布) 更令人信服。但是, 从不可区分性定义着手更容易 (比如, 证明一个指定方案是安全的)。幸运的是, 已证明两个定义是等价的:

定理 3.13 一个对称密钥加密方案具备窃听者存在情况下不可区分性加密, 当且仅当该方案窃听者存在的情况下是语义安全的。

往后, 类似的等价性适用于本章所有关于不可区分性的定义。因而通常使用不可区分性作为工作定义, 而达到的安全保证是语义安全的。

3.3 伪随机性

前面已经定义什么是加密方案的安全性, 读者可能期望即刻开始构造一个安全加密方案。但是, 在此之前, 需要引入伪随机性的概念。该概念在密码学中扮演了一个很重要的角色, 特别是在对称密钥加密中。不严格地说, 一个伪随机的字符串 (均为比特字符串) 是看起来很像均匀分布的字符串, 只要这个“看”的实体是在多项式时间内运行。如同不可区分性可视为完善保密在计算上的松弛, 伪随机是真正的随机性在计算上的松弛。

从技术上来说, 一个重要的概念是没有固定的字符串能够被认为是“伪随机的” (类似地, 认为任何固定的字符串是“随机的”没有意义)。伪随机性实际上指的是字符串的分布, 当声明一个长度为 ℓ 的字符串的分布 $\mathcal{D}$ 是伪随机的, 就意味着长度为 ℓ 的字符串的均匀分布和分布 $\mathcal{D}$ 是不可区分的。(严格地说, 在渐进的情形中, 实际上是分布 $\mathcal{D} = \{\mathcal{D}_n\}$ 的一个序列的伪随机性, 这里 $\mathcal{D}_n$ 和安全参数有关。在当前讨论中忽略这一点。) 更精确地, 对任何多项式时间算法来说, 分辨出它是分布 $\mathcal{D}$ 的字符串采样, 还是一个均匀随机选择的 ℓ 比特长的字符串, 是不可行的。

即使给出上面的讨论, 这一概念经常滥用, 把一个根据均匀分布采样的字符串叫做“随机字符串”, 并且把一个根据伪随机分布 $\mathcal{D}$ 采样的字符串叫做“伪随机字符串”。这里只是一种有用的速记法。

在继续之前，先提供一些直观的感受，为什么伪随机性能帮助构造安全的对称密钥加密方案。从一个简单的层面来讲，如果一个密文看起来随机，则很清楚的是，没有敌手能够从密文中推测任何关于明文的信息。在某种程度上，这是位于“一次一密”的完善保密性之后的准确直觉。如果是这样的话，该密文是均匀分布的（假设密钥未知），所以没有泄漏关于明文的任何信息。（当然，这样的陈述只是直觉上有吸引力，不能构成一个正式的论证。）“一次一密”是计算一个随机字符串（密钥）和明文的异或。如果一个伪随机字符串被使用，在一个多项式时间敌手看来，应该无法觉察到任何区别。因此，对多项式时间敌手来说，安全应该仍然满足。

如下所述，该想法可以实现。使用一个伪随机字符串而不是真正的随机字符串的好处在于，一个长的伪随机字符串能够通过一个相对短的随机种子（或者密钥）来生成。因此，一个短的密钥能用来加密一个长消息，这个完善保密不能办到的事情，现在可以办到了。

伪随机发生器。非正式地说，如上讨论，如果没有多项式时间的区分器能够区分一个根据 $\mathcal{D}$ 采样的字符串和一个均匀随机选择的字符串，则一个分布 $\mathcal{D}$ 是伪随机的。和定义 3.9 相似，可形式化为，当给予一个真正的随机字符串，以及一个伪随机字符串时，要求每个多项式时间算法输出1的概率相等（该输出比特被理解成为算法的“猜测”）。一个“伪随机发生器”是一个确定算法，接受一个短的真正随机的种子，然后将其扩展成为一个长的伪随机字符串。在下面的定义中，令 n 为种子的长度，要输入给发生器，并且令 $\ell(n)$ 为输出长度。很明显，该发生器只有在 $\ell(n) > n$ 时，才会有意义（否则，它不会产生任何新的“随机性”）。

定义 3.14 令 $\ell(\cdot)$ 为多项式，令 G 为确定多项式时间算法，该算法满足：对于任何输入 $s \in \{0, 1\}^n$ ，算法 G 输出一个长度为 $\ell(n)$ 的字符串。如果满足下面两个条件，则称 G 是一个伪随机发生器：

- (1)（扩展性：）对每个 n 来说，满足 $\ell(n) > n$ 。
- (2)（伪随机性：）对所有的概率多项式时间的区分器 $\mathcal{D}$ 来说，存在一个可忽略函数 negl ，满足

$$|\Pr[\mathcal{D}(r) = 1] - \Pr[\mathcal{D}(G(s)) = 1]| \leq \text{negl}(n)$$

其中 r 是从 $\{0, 1\}^{\ell(n)}$ 中均匀随机选择的，种子 s 是从 $\{0, 1\}^n$ 中均匀随机选择的，并且概率来源为 $\mathcal{D}$ 的随机性和对 r, s 的选择。

函数 $\ell(\cdot)$ 被称为 G 的扩展系数。

讨论。需要强调，一个伪随机发生器的输出实际上和随机输出相差很多。为了看清这一点，考虑当 $\ell(n) = 2n$ 的情况，此时 G 的输入变成原来的两倍。对 $\{0, 1\}^{2n}$ 的均匀分布， 2^{2n} 中每个可能的字符串被选择的概率正好为 2^{-2n} 。相反，考虑由 G 生成的分布。因为 G 接收长度为 n 的输入，不同的可能字符串的个数的范围最多为 2^n 。因此，一个长度为 $2n$ 的随机字符串在 G 的范围中的概率为 $2^n/2^{2n} = 2^{-n}$ （用所有在 G 的范围中的字符串的总数，除以长度为 $2n$ 的字符串的数量）。即大部分长度为 $2n$ 的字符串不会作为 G 的输出。

这特别意味着如果给定的时间无限制，区分随机字符串和伪随机字符串是很容易的。考虑下面指数时间的区分器 $\mathcal{D}$ 的如下工作：对某个输入字符串 w ，当且仅当存在一个 $s \in \{0, 1\}^n$ ，满足 $G(s) = w$ 时，区分器 $\mathcal{D}$ 输出1。（通过搜索所有的 $\{0, 1\}^n$ ，对每个

$s \in \{0, 1\}^n$ 计算 $G(s)$ 来执行该计算。该计算能够执行是因为 G 的工作描述是已知的；只有它的种子是未知的。) 现在，如果 w 是由 G 生成的，那么它满足 D 输出 1 的概率为 1。相反地，如果 w 是均匀分布在 $\{0, 1\}^{2^n}$ 中的，则存在一个 s 使得 $G(s) = w$ 的概率最多为 2^{-n} ，在这种情况下， D 输出 1 的概率最多为 2^{-n} 。因此我们有

$$|\Pr[D(r) = 1] - \Pr[D(G(s)) = 1]| \geq 1 - 2^{-n}.$$

这个结果很大。该类型的攻击被叫做蛮力攻击，因为它试过了所有的可能种子。这类“攻击”的优势在于对所有的发生器都适用，和它们是如何工作的无关。

以上说明 G 产生的分布不是均匀的，但多项式时间的区分器没有时间来执行上面的步骤。如果 G 是一个伪随机发生器，则它保证了不存在任何多项式时间的步骤能够区分随机字符串和伪随机字符串。这意味着伪随机字符串和真正随机的字符串一样，只要种子是保密的，且只考虑多项式时间的观察者（敌手）。

种子和它的长度。一个伪随机发生器的种子必须被均匀地随机选取，且对区分器来说是完全保密的。从上面的蛮力攻击可看到另外一个关键点是， s 必须足够的长，以至于没有“有效的算法”有时间去遍历所有可能的种子。技术上，所有算法假设都是在多项式时间内运行，因此当 n 足够大时，算法不可能搜索到所有 2^n 个可能的种子。但是实际上，种子必须被认为具有某个具体的长度。鉴于此， s 至少必须足够长，以至于尝试所有的可能种子是不可能的。

伪随机发生器的存在性。一个首要问题应该是满足定义 3.14 的实体是否真的存在。不幸的是，目前不知道如何证明伪随机发生器的存在性。尽管如此，本书相信伪随机发生器真的存在，并且这种相信基于一个事实：伪随机发生器能够在相当弱的假设下被构造出来（从可证明的意义上），这个相当弱的假设是单向函数是存在的。这将在第 6 章更详细地讨论。就当前而言，有充分的理由说：存在某个已经研究了很久的问题，该问题还没有已知的有效算法，并且该问题被广泛地认为在多项式时间内是无法解决的。这类问题的一个例子是大数分解问题：即找到一个给定整数的素因子。单向函数，从而伪随机发生器能够在这些难题真的很“困难”的假设下被构造出来。

实际中，相信是伪随机发生器的各种构造方法是已知的。事实上，正如稍后将在本章和第 5 章看到的，这些构造方法是存在的，并且被认为满足甚至更强的要求。

3.4 构造安全加密方案

3.4.1 一个安全的定长加密方案

现在已准备好去构造一个定长的加密方案，该方案在有窃听者存在的条件下，不可区分加密。这里构造的加密方案和“一次一密”加密方案很相似（参见 2.2 节），除了这里使用的是一个“伪随机字符串”而不是一个随机字符串作为“填充”。由于一个伪随机字符串对任何多项式时间敌手来说“看起来随机”，故该加密方案能被证明是计算安全的。

加密方案。令 G 是一个扩展因子为 ℓ 的伪随机发生器（即 $|G(s)| = \ell(|s|)$ ）。回想一个加密方案被三个算法定义：一个密钥生成算法 Gen ，一个加密算法 Enc 以及一个解密算法 Dec 。加密过程是，密钥（作为一个种子）应用到一个伪随机发生器来获得一个长的填充，然后和

明文消息作异或处理。该方案在构造方法 3.15 中正式地描述，如图 3.2 所示。

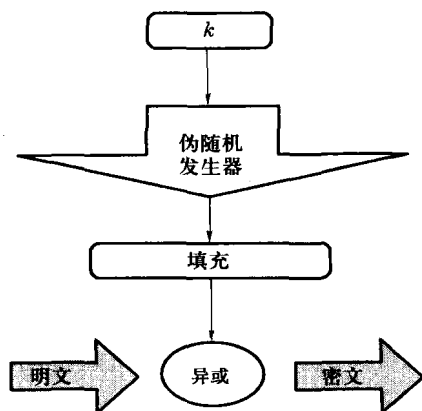


图 3.2 使用伪随机发生器的加密

构造方法 3.15

令 G 是一个带扩展因子 ℓ 的伪随机发生器。定义一个消息长度为 ℓ 的对称密钥加密方案如下：

- Gen: 输入 1^n ，均匀随机地选择 $k \leftarrow \{0, 1\}^n$ ，并将其作为密钥输出。
- Enc: 输入一个密钥 $k \in \{0, 1\}^n$ ，以及一个消息 $m \in \{0, 1\}^{\ell(n)}$ ，输出密文 $c := G(k) \oplus m$ 。
- Dec: 输入一个密钥 $k \in \{0, 1\}^n$ ，以及一个密文 $c \in \{0, 1\}^{\ell(n)}$ ，输出明文消息 $m := G(k) \oplus c$ 。

一个使用任何伪随机发生器的对称密钥加密方案

现在证明在窃听者存在的情况下，若假设 G 是伪随机发生器，则给定的加密方案具备不可区分性。注意这里的声称不是无条件的，从而，将加密方案的安全规约到伪随机发生器 G 的属性。这是一个在 3.1.3 节中描述的非常重要的证明技术，证明后将进一步说明。

定理 3.16 若 G 是一个伪随机发生器，则构造方法 3.15 是一个在窃听者存在的情况下，具备不可区分加密的定长对称密钥加密方案。

证明 令 Π 表示构造方法 3.15。需要说明如果存在一个概率多项式时间的敌手 $\mathcal{A}$ ，对其而言定义 3.8 不满足，则可以构造一个概率多项式时间算法将真正的随机字符串和 G 的输出区分开。这背后的直觉是，如果 Π 使用一个真正的随机字符串来代替伪随机字符串 $G(k)$ ，则得到的方案将会和“一次一密”加密方案相同， $\mathcal{A}$ 将不能够以大于 $1/2$ 的概率正确猜出是哪一个消息被加密。所以，如果定义 3.8 不满足，则 $\mathcal{A}$ 必能（隐含地）区分真正的随机字符串和 G 的输出。下面的规约将使之更清晰。

令 $\mathcal{A}$ 为一个概率多项式时间的敌手，定义 ε 为

$$\varepsilon(n) \stackrel{\text{def}}{=} \Pr [\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eva}}(n) = 1] - \frac{1}{2} \quad (3.2)$$

使用 $\mathcal{A}$ 来构造一个伪随机发生器 G 的区分器 D ，使得 D “成功”的概率为 $\varepsilon(n)$ 。该区分器

被给定一个字符串 w 作为输入，并且它的目标是判断 w 是均匀随机的选出（即 w 是“随机字符串”），还是 w 通过选择一个随机 k ，然后通过 $w = G(k)$ 计算出的（即 w 是“伪随机字符串”）。 D 用下面描述的方式来模拟 $\mathcal{A}$ 的窃听实验，并且观察 $\mathcal{A}$ 是否成功。如果 $\mathcal{A}$ 成功，则 D 猜测 w 是一个伪随机字符串；如果 $\mathcal{A}$ 没有成功，则 D 猜测 w 是一个随机字符串。具体如下：

区分器 D ：

D 被指定了一个字符串 $w \in \{0, 1\}^{\ell(n)}$ 作为输入。（假设 n 能够被 $\ell(n)$ 确定。）

- (1) 运行 $\mathcal{A}(1^n)$ 来获得一对消息 $m_0, m_1 \in \{0, 1\}^{\ell(n)}$ 。
- (2) 随机选择一个比特 $b \leftarrow \{0, 1\}$ 。令 $c := w \oplus m_b$ 。
- (3) 把 c 给 $\mathcal{A}$ ，得到输出 b' 。如果 $b' = b$ ，则输出1，否则输出0。

在分析区分器 D 的行为之前，定义一个修改的加密方案 $\tilde{\Pi} = (\widetilde{\text{Gen}}, \widetilde{\text{Enc}}, \widetilde{\text{Dec}})$ 正好是“一次一密”加密方案，唯一不同的是现在包含一个安全参数，该安全参数确定了被加密消息的长度。即 $\widetilde{\text{Gen}}(1^n)$ 输出一个长度为 $\ell(n)$ 的完全随机的密钥 k ，并且使用密钥 $k \in \{0, 1\}^{\ell(n)}$ 加密消息 $m \in \{0, 1\}^{\ell(n)}$ ，得到密文 $c := k \oplus m$ 。（解密照常，但对接下来的讨论不重要。）根据“一次一密”的完善保密性，有

$$\Pr \left[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{eav}}(n) = 1 \right] = \frac{1}{2} \quad (3.3)$$

在对 D 的行为的分析中，主要的观察如下：

(1) 如果 w 均匀随机地从 $\{0, 1\}^{\ell(n)}$ 中选择，则当 $\mathcal{A}$ 作为一个子程序被 D 运行的时候， $\mathcal{A}$ 所见内容（称为视图）的分布，与 $\mathcal{A}$ 在实验 $\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{eav}}(n)$ 中的所见内容的（视图）分布是相同的。这是因为 $\mathcal{A}$ 被给定一个密文 $c = w \oplus m_b$ ，其中 $w \in \{0, 1\}^{\ell(n)}$ 是一个完全随机的字符串。因此，结果就是均匀随机选择 $w \leftarrow \{0, 1\}^{\ell(n)}$ ，

$$\Pr [D(w) = 1] = \Pr \left[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{eav}} = 1 \right] = \frac{1}{2}$$

其中第二个等式由式 (3.3) 而得。

(2) 如果 w 等于 $G(k)$ ， $k \leftarrow \{0, 1\}^n$ 是随机均匀地选择，则当 $\mathcal{A}$ 作为一个子程序被 D 运行时的视图分布，和 $\mathcal{A}$ 在实验 $\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{eav}}(n)$ 中的视图分布是相同的。这是因为 $\mathcal{A}$ 被指定了一个密文 $c = w \oplus m_b$ ，其中 $w = G(k)$ ，这里 $k \leftarrow \{0, 1\}^n$ 是一个均匀分布的值。因此，当 $w = G(k)$ ， $k \leftarrow \{0, 1\}^n$ 是均匀随机地选择，有

$$\Pr [D(w) = 1] = \Pr [D(G(k)) = 1] = \Pr \left[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{eav}}(n) = 1 \right] = \frac{1}{2} + \varepsilon(n)$$

其中第二个等式是依据 ε 的定义。

因此有

$$|\Pr [D(w) = 1] - \Pr [D(G(k)) = 1]| = \varepsilon(n)$$

这里， w 是从 $\{0, 1\}^{\ell(n)}$ 中均匀随机选择的，并且 k 是从 $\{0, 1\}^n$ 中均匀随机选择的。假设 G 是伪随机发生器，那么 ε 必须是可忽略的。通过 ε 的定义（参见式 (3.2)），意味着 Π 在窃听者存在的情况下，具备不可区分的加密，证毕。 ■

很容易在证明的细节中迷失，也很容易感到疑惑，与“一次一密”相比，得到了什么；

毕竟，“一次一密”也是通过异或一个 ℓ 比特的消息和一个 ℓ 比特的字符串来加密。该构造方法的关键是， ℓ 比特的字符串 $G(k)$ 可比密钥 k 长很多。特别地，使用上面的加密方案加密一个几兆字节长的文件，仅使用 128 比特的密钥是可能的。这和定理 2.7 的陈述完全相反：完善保密的加密方案中，密钥必须至少和被加密的消息一样长。基于计算的方法得到的比完善保密更多。

规约——讨论。构造 3.15 不能无条件地证明是安全的。相反，在假设 G 是一个伪随机发生器的情况下，可证明是安全的。将构造的安全性规约到某个基础原语的方法是非常重要的，原因如下。首先，正如已经注意到的，目前还不知道如何证明满足定义 3.8 的加密方案的无条件存在性，并且这样的证明是目前不可能达到的。鉴于此，将高级构造方案的安全性规约到一个低级原语有很多好处（正如在 1.4.2 节讨论的那样）。好处之一就是，通常设计一个低级原语比高级构造方案要简单；相似地，与高级定义相比，满足低级定义更容易让人信服。这并不意味着构造一个伪随机发生器是“容易的”，构造发生器仅仅是比从最开始构造一个加密方案容易。（当然，在现在的情况下，加密方案几乎什么都不做，除了一个异或操作，该异或操作将伪随机发生器的输出和消息做异或，所以，这个说法并不是真正正确。但是，后面将看到更复杂的构造方法，在这些情况下，将这些任务规约到一个更简单任务的能力是非常重要的。）

3.4.2 处理变长消息

上一节的构造方法存在一个缺点，就是仅仅只对定长的消息加密。（即对每个给定的安全参数 n ，只有长度为 $\ell(n)$ 的消息能被加密。）该缺陷能够通过使用一个在构造方法 3.15 中的输出长度可变的伪随机发生器来轻松解决。

输出长度可变的伪随机发生器。在某些应用中，不能事先知道需要多少比特的伪随机序列。因此，实际上需要这样一个伪随机发生器，该发生器能够输出一个任何希望长度的伪随机字符串。具体而言，希望 G 接受两个输入：种子 s 以及输出长度 ℓ （该长度 ℓ 以一进制的形式给出，因为安全参数是一进制的）； G 应该输出一个长度为 ℓ 的伪随机字符串。正式的定义如下：

定义 3.17 一个确定的多项式时间算法 G ，如果满足以下三个条件，则 G 是一个输出长度可变的伪随机发生器，如果满足以下条件：

- (1) 令 s 为一个字符串，整数 $\ell > 0$ 。则 $G(s, 1^\ell)$ 输出一个长度为 ℓ 的字符串。
- (2) 对所有的 s, ℓ, ℓ' ， $\ell < \ell'$ ，字符串 $G(s, 1^\ell)$ 是 $G(s, 1^{\ell'})$ 的前缀^⑦。
- (3) 定义 $G_\ell(s) \stackrel{\text{def}}{=} G(s, 1^{\ell(|s|)})$ 。则对于每个多项式 $\ell(\cdot)$ ，有 G_ℓ 是一个扩展因子为 ℓ 的伪随机发生器。

任何标准的伪随机发生器（参见定义 3.14）能够被转化成为一个输出长度可变的伪随机发生器，参见 6.4.2 节。

^⑦ 需要该条件是基于技术理由，为了证明构造方法 3.15 在使用变长发生器时，对变长消息加密的安全性。

给出上面的定义后，很自然地修改构造方法 3.15：使用密钥 k 加密一个消息是通过计算密文 $c := G(k, 1^{|m|}) \oplus m$ 来完成的；用密钥 k 解密一个消息是通过计算消息 $m := G(k, 1^{|c|}) \oplus c$ 来完成的。将满足定义 3.8 的方案的证明留作练习。证明过程参照定理 3.16，除了某些技术上的细微差别。

3.4.3 流密码和多个加密

在密码学的文献资料中，在前两节陈述的加密方案的类型经常被称为“流密码”。这是因为加密的执行是通过首先生成一个伪随机比特流，然后将该比特流和明文做异或运算。不幸的是，这里有一点困惑，就是术语“流密码”指的是生成比特流的算法（即伪随机发生器 G ），还是整个加密方案。这是一个很关键的问题，因为使用伪随机发生器的方式直接决定了指定的加密方案是否安全。本书认为，术语流密码最好是指生成伪随机流的算法，因此，一个“安全的”流密码应该满足一个输出长度可变的伪随机发生器^⑥的定义。使用这个术语，一个流密码本质上不是一个加密方案，而是一种构造加密方案的工具。在下面讨论多个加密问题时，这个讨论的重要性就会显而易见了。

实际应用中的流密码。实践中的流密码构造方法有很多种，并且这些一般都是非常快的。一个流行的例子就是流密码 RC4，当合理使用时（参见后面），RC4 被广泛认为是安全的。实际中，流密码的安全性不是很好理解，特别是和分组密码（稍后在本章进行介绍）进行比较时。事实证明：不存在一种标准的、流行的流密码被使用多年，且安全性没有出现问题。例如，“简单的”RC4（在某一点上被认为是安全的，仍然被广泛地部署）已知存在一些重要的弱点。其中之一，由 RC4 生成的输出流的前几个字节已经被证明是有偏离的。尽管看起来这不是很严重，但是已经被证实该缺陷能够被用来轻易地攻破在 802.11 无线网络中使用的 WEP 加密协议。（WEP 是保护无限通信的标准化协议。WEP 标准已经升级来修补该问题。）如果 RC4 被使用，输出的字符串最前面的 1024 比特应该被丢弃。

线性反馈移位寄存器（LFSRs）在历史上曾经是很流行的流密码。但是它们被证明非常不安全（在某种程度上，给定足够多字节的输出，密钥能够完全还原），所以应该永远不再使用。

一般主张使用分组密码来构造安全加密方案。对除了大部分资源受限的环境之外的所有场合，分组密码都是足够高效的，看起来比现有的流密码要安全得多。而且，流密码可从分组密码简单地构造出来，正如下面 3.6.4 节所描述的。该方法的缺陷是，和一个专门的流密码相比，其效率较低。

多个加密的安全性。定义 3.8 以及到目前为止所有的讨论，已经处理了敌手收到单个密文的情况。在现实中，通信双方发送多个密文，一个窃听者将会看到很多密文。因而很重要的一点是，即使在这样的情况下，也要确保加密方案的安全性。

首先给出一个安全定义。如定义 3.8，首先引入一个合适的实验，该实验为了加密方案 Π 而定义，一个敌手 $\mathcal{A}$ ，以及一个安全参数 n 。

多消息窃听实验 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{mult}}(n)$:

⑥ 很快将引入分组密码的概念。在那些内容中，该术语指的是工具本身，而不是如何使用以便实现安全加密。因此类似地，更愿意使用术语“流密码”。

- (1) 敌手 $\mathcal{A}$ 被给定输入 1^n , 输出一对消息向量 $\vec{M}_0 = (m_0^1, \dots, m_0^t)$ 以及 $\vec{M}_1 = (m_1^1, \dots, m_1^t)$, 对于所有 i , 满足 $|m_0^i| = |m_1^i|$.
- (2) 通过运行 $\text{Gen}(1^n)$ 生成一个密钥 k 和选择一个随机比特 $b \leftarrow \{0, 1\}$. 对于所有 i , 计算密文 $c^i \leftarrow \text{Enc}_k(m_b^i)$, 并且将密文向量 $\vec{C} = (c^1, \dots, c^t)$ 给 $\mathcal{A}$.
- (3) $\mathcal{A}$ 输出一个比特 b' .
- (4) 如果 $b' = b$, 该实验的输出为1; 否则输出0.

除了现在所指的是上面的实验, 对多消息的安全定义和单个消息的安全定义其实是相同的。

定义 3.18 一个对称密钥加密方案 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$, 如果对所有的概率多项式时间敌手 $\mathcal{A}$, 存在一个可忽略函数 negl , 满足

$$\Pr \left[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{mult}}(n) = 1 \right] \leq \frac{1}{2} + \text{negl}$$

则称其具备窃听者存在的情况下不可区分多次加密, 其中概率来源于 $\mathcal{A}$ 使用的随机性, 以及在实验中使用的随机性 (选择密钥、随机比特以及加密本身)。

一个关键的观察是: 单个加密的安全 (定义 3.8) 并不意味着在多次加密下的安全。下面的命题给出正式的论述。

命题 3.19 存在这样的对称密钥加密方案, 满足窃听者存在情况下不可区分的加密, 但不满足窃听者存在情况下不可区分的多次加密。

证明 没必要寻找一个加密方案来满足该命题。特别地, 在定理 3.16 中已经证明了构造方法 3.15 的单次加密的安全性, 该构造用于多次加密时, 是不安全的。对此不必觉得意外, 因为前面已经看到只有在使用一次时, “一次一密”才是安全的, 对构造方法 3.15 而言也是相同的道理。

具体而言, 考虑下面的攻击加密方案 (在某种意义上, 通过 $\text{PrivK}^{\text{mult}}$ 来定义) 敌手 $\mathcal{A}$: $\mathcal{A}$ 输出向量 $\vec{M}_0 = (0^n, 0^n)$ 以及 $\vec{M}_1 = (0^n, 1^n)$ 。也就是, 第一个向量包含了两个明文, 每个明文都是长度为 n 的全零字符串。相比之下, 在第二个向量中, 第一个明文是全零, 第二个明文是全1。现在, 令 $\vec{C} = (c^1, c^2)$ 为 $\mathcal{A}$ 接收到的密文向量。如果 $c^1 = c^2$, 则 $\mathcal{A}$ 输出0; 否则, $\mathcal{A}$ 输出1。

现在分析 $\mathcal{A}$ 成功猜测 b 的概率。关键之处是构造方法 3.15 是确定的, 所以如果相同消息被多次加密, 则会导致每次都是相同密文。现在, 如果 $b = 0$, 则每次都是相同消息被加密 (因为 $m_0^1 = m_0^2$); 因此, $c^1 = c^2$, $\mathcal{A}$ 在这种情况下, 总是输出0。另一方面, 如果 $b = 1$, 则每次加密不同的消息 (因为 $m_0^1 \neq m_0^2$), 并且 $c^1 \neq c^2$; 这里, $\mathcal{A}$ 总是输出1。因此, $\mathcal{A}$ 输出 $b' = b$ 的概率为1, 根据定义 3.18, 该加密方案并不安全。 ■

命题 3.19 的证明可能看起来是人为的, 但事实并非如此。相同的消息被重发能够提供重要的信息, 在历史上对密码分析起到很大作用。

概率加密的必要性。在命题 3.19 的证明中, 已经说明了构造方法 3.15 对多次加密并不安全。证明中用到的该构造方法的唯一特征是, 加密一个消息总是产生相同的密文, 所以实际上获得的是, 任何确定方案对多次加密来说, 都是不安全的。这足够重要以至于作为定理来

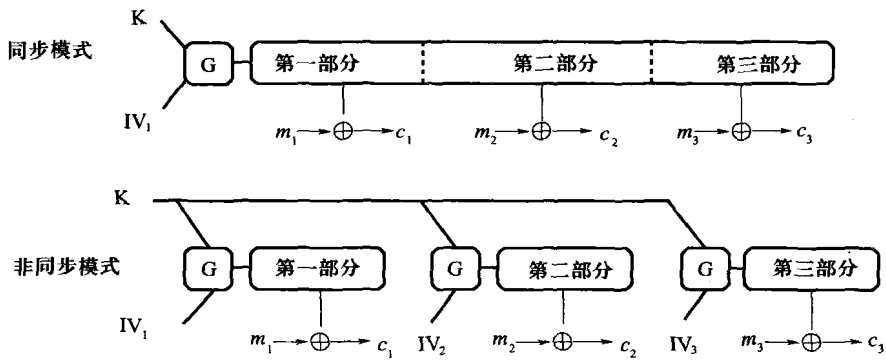
陈述。

定理 3.20 令 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ 为一个加密方案， Enc 是密钥和消息的一个确定函数，则 Π 不具备窃听者存在情况下不可区分多次加密。

这意味着根据定义 3.18 来构造一个安全的加密方案，至少必须保证当相同的消息被多次加密时，每次都产生不同的密文。初看上去这好像是一个不可能完成的任务，因为解密必须总能恢复消息。但是，稍后将会解释如何实现它。

使用流密码的多次加密中的一个普遍的错误。不幸的是，对密码学构造方法的不正确的实现是很频繁的。一个普遍的错误就是，使用一个流密码（在构造方法 3.15 中的初级形式）来加密多个明文。例如，这个错误出现在 Microsoft Word 以及 Excel 中执行加密的情况下；参见参考文献[147]。在实际中，这样的错误可能是毁灭性的。需要强调这不仅仅是“理论上的人工制品”，因为加密相同的消息两次产生相同消息。即使相同的消息从来不被加密两次，各种攻击也是可能的，正如 2.2 节中提到的。

使用流密码的安全多次加密。在实际中，一般有两种流密码/伪随机发生器的使用方式进行安全地加密多个明文（见如图 3.3）。



(1) 同步模式：在该模式中，通信双方使用流密码输出流的不同部分来加密每个消息。该模式是“同步的”，因为双方需要知道输出流的哪个部分已经被使用，以便于防止重用，因为（已经说明了）重用会不安全。

该模式对于通信双方在单个会话中是非常有用的。在该情形中，一方使用流的第一部分来发送它的第一个消息。另一方获得密文，解密，然后使用流的接下来的一部分来加密它的回复。因为流的每一部分仅仅使用一次，将所有双方发送的消息连接起来，看作单个长明文是可能的。该方案的安全性依据定理 3.16（变长消息情形）。

该模式并不适合所有的应用，因为在加密时，要求双方维护状态（要分清接下来使用流中哪一部分）。由于使用流的不同部分，尽管该方法是确定的，但其安全性和定理 3.20 并不冲突（因为它并不满足定义 3.7 中的语法要求）。

(2) 非同步模式：在这种模式下，加密是双方相互独立地执行的，并且双方不需要维护状态。但是，为了实现安全性，必须重点强调伪随机发生器的概念。这里一个伪随机发生器有两个输入：一个种子 s 和一个长度为 n 的初始向量 IV 。简言之，要求即使当 IV 已知时（但

是 s 保密), $G(s, IV)$ 依然是伪随机的。而且, 对应于两个随机选择的初始向量 IV_1 和 IV_2 , 流 $G(s, IV_1)$ 以及 $G(s, IV_2)$ 应该保持随机性, 即使是它们连同各自的 IV 一起被观察到。

给出一个上面的发生器, 加密能够被定义为

$$\text{Enc}_k(m) := \langle IV, G(k, IV) \oplus m \rangle$$

其中 $IV \leftarrow \{0, 1\}^n$ 是随机选择的。(为了简化, 这里主要关注对定长消息的加密。)对每次加密, IV 是全新选出的(即独立且均匀随机地选择), 并且因此每个流是伪随机的, 即使前面的流已知。注意到 IV 是作为密文的一部分来发送的, 用于接收方的解密; 即给定 (IV, c) , 接收方能够计算出 $m := c \oplus G(k, IV)$ 。

实际中的很多流密码被假定具备上面非正式地简要描述的“广义伪随机性”的特点, 并且能够被应用在非同步模式。但是注意, 一个标准的伪随机发生器可能不具备这个特点, 该假设是一个强假设。事实上, 这样一个发生器“几乎是”一个伪随机函数; 参见 3.6.1 节以及练习 3.20。

3.5 选择明文攻击 (CPA) 的安全性

到目前为止, 已经考虑了一个相对弱的敌手, 仅仅只能在通信方之间被动地窃听。(其实该定义中 $\text{PrivK}^{\text{eav}}$ 允许敌手选择要加密的明文。尽管如此, 除了这个能力外, 敌手是完全被动的。)在本节中, 正式引入一个更强大的攻击类型, 叫做选择明文攻击 (CPA)。在 CPA 攻击下的安全定义和定义 3.8 中的相同, 除了敌手的能力被加强了。

选择明文攻击背后的基本想法是, 敌手 $\mathcal{A}$ 被允许适应性地选择多消息来请求加密。这一点被形式化为: 允许 $\mathcal{A}$ 和“加密预言机”自由交互, 加密预言机被视为一个“黑盒子”, 将使用密钥 k ($\mathcal{A}$ 不知道 k) 加密 $\mathcal{A}$ 选择的消息。根据计算机科学中的标准表示方法, 这里用 $\mathcal{A}^{\mathcal{O}(\cdot)}$ 来表示 $\mathcal{A}$ 访问预言机 $\mathcal{O}$ 的计算过程, 因此在这种情况下, 用 $\mathcal{A}^{\text{Enc}_k(\cdot)}$ 来表示 $\mathcal{A}$ 访问使用密钥 k 的加密预言机 $\mathcal{O}$ 的计算。当 $\mathcal{A}$ 询问预言机并提供一个明文消息 m 作为输入时, 预言机返回一个密文 $c \leftarrow \text{Enc}_k(m)$ 作为回复。当 Enc 是随机的, 则预言机每次回答一个询问时, 都会保证新的随机性。

安全性的定义要求: 即使 $\mathcal{A}$ 具备访问加密预言机能力, $\mathcal{A}$ 也不能区分两个任意消息的加密。这里先提出定义, 然后将讨论该定义模拟真实世界的什么攻击。

首先定义一个对任意对称密钥加密方案 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$, 任意敌手 $\mathcal{A}$, 以及任意安全参数 n 的实验。

CPA 不可区分实验 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$:

- (1) 一个密钥 k 是通过运行 $\text{Gen}(1^n)$ 来生成的。
- (2) 输入 1^n 给敌手 $\mathcal{A}$, 敌手 $\mathcal{A}$ 可以访问预言机 $\text{Enc}_k(\cdot)$, 输出一对长度相等的消息 m_0, m_1 。
- (3) 选择一个随机比特 $b \leftarrow \{0, 1\}$, 计算出一个密文 $c \leftarrow \text{Enc}_k(m_b)$, 交给 $\mathcal{A}$ 。 c 叫做挑战密文。
- (4) 敌手 $\mathcal{A}$ 继续访问预言机 $\text{Enc}_k(\cdot)$, 输出一个比特 b' 。
- (5) 如果 $b = b'$, 该实验的输出被定义为1, 否则定义为0。(若 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1$, 则认为

$\mathcal{A}$ 成功。)

定义 3.21 一个对称密钥加密方案 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ ，如果对所有的概率多项式敌手 $\mathcal{A}$ ，存在一个可忽略函数 negl ，使得

$$\Pr \left[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{CPA}}(n) = 1 \right] \leq \frac{1}{2} + \text{negl}(n)$$

则是选择明文攻击（CPA）条件下的不可区分加密，这里概率来源于 $\mathcal{A}$ 使用到的随机性，以及在实验中使用到的随机性。

任何方案如果是选择明文攻击条件下的不可区分加密，则也是窃听者存在的情况下不可区分加密。这是因为 $\text{PrivK}^{\text{eav}}$ 是 $\text{PrivK}^{\text{CPA}}$ 的一种特殊情况，即敌手没有使用预言机。

初看上去，可能会觉得定义 3.21 是不可能达到的，特别是考虑到敌手输出 (m_0, m_1) ，然后接收到挑战密文 $c \leftarrow \text{Enc}_k(m_b)$ 。因为敌手 $\mathcal{A}$ 能够访问预言机 $\text{Enc}_k(\cdot)$ ，它能够要求该预言机加密消息 m_0 以及 m_1 ，并且因此获得 $c_0 \leftarrow \text{Enc}_k(m_0)$ 和 $c_1 \leftarrow \text{Enc}_k(m_1)$ 。敌手 $\mathcal{A}$ 能够比较 c_0 和 c_1 得到 c ，即如果 $c = c_0$ ，则 $\mathcal{A}$ 知道 $b = 0$ ；如果 $c = c_1$ ，则知道 $b = 1$ 。问题是为什么该策略不能使 $\mathcal{A}$ 以 1 的概率确定 b ？

答案就是，如果加密方案是确定的（因为这意味着每次同一个消息加密，会获得相同的密文），这样的攻击确实有效。因此，和多次加密的安全一样，任何确定加密方案都不能抵御选择明文攻击。换句话说，任何 CPA 安全的加密方案必须是概率性的。必须将随机性作为加密过程中的一部分，来确保相同消息的加密可能是不同的^⑨。

现实世界中的选择明文攻击。定义 3.21 至少和早期的定义 3.8 一样强，所以从这个更新的定义着手，不会损失安全性。但是，通常使用一个太强的定义会付出一些代价，因为可能会导致使用效率更低的方案（即使存在满足“现实世界的应用”的更有效的方案）。因此，应该询问自己：选择明文攻击是否代表了一个真正值得关注的现实中的敌对威胁。

事实上在很多场景中，选择明文攻击（以某种形式）是一种现实的威胁。首先通过查看一些二战时期的军事历史可说明这一点。在 1942 年 5 月，美国海军密码专家已经发现，日本正计划对位于中太平洋的中途岛发动攻击。他们通过截获了一份通信消息才掌握到这个信息，该消息包含了一份密文片段“AF”，并且他们相信这对应着明文“中途岛”。他们尝试让华盛顿的指挥者相信的确如此，但不幸的是，尝试无效；通常认为中途岛不可能成为攻击目标。海军密码专家策划了接下来的计划。他们命令在中途岛的美军军队发送一个明文消息，说他们的淡水供给不足。日本截获了该消息，立刻报告给其上级：“AF”淡水不足。海军密码专家于是确信“AF”的确就是中途岛，美军迅速派遣了三架运输机飞抵该位置。结果就是中途岛被解救了，日军受到重创。据说，这一场战争就是美国和日本在太平洋战场的转折点。（参见文献[91]和[146]查阅更多信息。）

这里，海军密码专家完成了一次经典的选择明文攻击。他们能够要求日本（尽管是一种迂回方式）来加密“中途岛”这个词，来推测另外一份之前截获的密文的信息。如果日军的

^⑨ 正如我们已经看到的，如果加密过程在连续加密之间保持状态（正如在流密码的同步模式中那样），则随机性（由抛硬币决定）可能不是必须的。按照定义 3.7，我们一般只考虑无状态的方案（更合意）。

加密方案能够抵御选择明文攻击，美军密码专家的策略将无法成功（并且历史可能变得非常不同）。需要强调，日本并没有打算成为美军的“加密预言机”，也没有分析 CPA 安全的必要性，他们不太可能意识到 CPA 安全是必要的^⑩。

注意，不要认为选择明文攻击仅仅只是投机取巧的结果。在很多情况中，一个敌手能够影响诚实方加密的内容（对敌手来说，完全掌握加密的内容是不多见的）。考虑下面的例子：当前很多服务器以一种安全的方式（即使用加密）进行相互通信。但是，这些服务器发送给对方的消息是基于收到的内部和外部请求，这些请求可能是恶意用户选择的。这些敌手能够影响服务器加密的明文消息，且有的时候影响程度非常大。这样的系统必须通过使用一个可以抵御选择明文攻击的加密方案来保护，因此强烈建议总是使用这种类型的加密方案。

多次加密的 CPA 安全。把定义 3.21 扩展到多次加密的情况是很简单的，与定义 3.8 扩展到定义 3.18 相同。即定义一个实验，除了 $\mathcal{A}$ 输出一对明文向量以外，该实验和 $\text{PrivK}^{\text{CPA}}$ 完全相同。要求不存在多项式时间的敌手 $\mathcal{A}$ ，这种敌手在该实验中成功的概率大于 $1/2$ 的部分是不可忽略的。

重要的是，单次加密的 CPA 安全即意味着多次加密的 CPA 安全。（这和窃听敌手的情况形成对比；参见命题 3.19。）这里陈述该声称，而不给出证明，在 10.2.2 节公钥情形中会给出证明。

命题 3.22 任何在选择明文攻击条件下的不可区分加密的对称密钥加密方案，也是在选择明文攻击条件下的不可区分“多次”加密方案。

这是 CPA 安全的重要技术优势。它足够用来证明一个方案对单次加密是 CPA 安全的，然后我们“免费”获得了它对于多次加密也是 CPA 安全的。

固定长度和任意长度消息的比较。从 CPA 安全的定义着手的另外一个优势是，它允许在处理定长的加密方案时，不会过多失去一般性。特别地，给定任何 CPA 安全的定长加密方案 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ ，容易构造一个 CPA 安全的任意长度的加密方案 $\Pi' = (\text{Gen}', \text{Enc}', \text{Dec}')$ 。为了简化，假设 Π 加密 1 比特长的消息（当 Π 加密某个任意长度 $\ell(n)$ 的消息，这里所述可以自然而然地进行相应扩展）。令 Gen 和 Gen' 是相同的。对任意消息 m （有某个任意长度 ℓ ），以下面的方式定义 Enc'_k ：

$$\text{Enc}'_k(m) = \text{Enc}_k(m_1), \dots, \text{Enc}_k(m_\ell)$$

其中 $m = m_1 \dots m_\ell$ ，对所有 i ， $m_i \in \{0, 1\}$ 。解密照常进行。于是，当且仅当 Π 是 CPA 安全时， Π' 是 CPA 安全，证明可从命题 3.22 得出。

尽管如此，实际中可能存在比上面这种改变定长加密方案的方式更有效的方法来加密任意长度的消息。在 3.6.4 节中来介绍其他加密任意长度消息的方法。

3.6 CPA 安全的加密方案创建

本节将构造能够抵御选择明文攻击的加密方案，从引入伪随机函数这个重要概念开始。

^⑩ 问问自己是否预先考虑到这样的攻击。

3.6.1 伪随机函数

如前所述,伪随机发生器能够被用来获得在窃听者存在情况下的安全性。伪随机性的概念在获得抵御选择明文攻击的安全性中,有重要的作用。但是,现在不考虑伪随机字符串,而是考虑伪随机函数。我们将特别关注将 n 比特字符串映射到 n 比特字符串的伪随机函数。正如早前对伪随机性的讨论,任何固定函数 $f: \{0,1\}^n \rightarrow \{0,1\}^n$ 是伪随机的说法是没有意义的(同样的道理,一个固定的函数是随机的说法几乎没有意义)。因此,必须在技术上说明这是函数分布的伪随机性。一个简单的方式是考虑带密钥的函数,定义如下。

一个带密钥的函数 F 是双输入函数, $F: \{0,1\}^* \times \{0,1\}^* \rightarrow \{0,1\}^*$, 其中第一个输入叫做密钥,用 k 来表示,第二个输入就叫输入。一般地,密钥 k 被选择之后就固定了,于是关注单输入函数 $F_k: \{0,1\}^* \rightarrow \{0,1\}^*$, 用 $F_k(x) \stackrel{\text{def}}{=} F(k,x)$ 来定义。为了简化,通常将假设 F 是保留长度的,意味着 F 中的密钥,输入以及输出的长度都是相等的。也就是说,假设函数 F 仅当密钥 k 和输入 x 有相同长度时才被定义,即 $|F_k(x)| = |x| = |k|$ 。所以,通过固定密钥 $k \in \{0,1\}^n$,可获得一个将 n 比特字符串映射到 n 比特字符串的函数 $F_k(\cdot)$ 。如果存在一个确定的多项式时间算法能够在给定输入 k 和 x 的情况下,计算出 $F(k,x)$,则称 F 是有效的(efficient)。这里将只关注有效的函数 F 。

假定选择一个随机密钥 $k \leftarrow \{0,1\}^n$,并考虑产生的单输入函数 F_k ,一个带密钥的函数 F 导致一种函数的分布。从直觉上来说,如果函数 F_k (对随机选择的密钥 k 而言),无法和一个从拥有同样定义域和值域的所有函数的集合中均匀随机选取的函数相区分;也就是说,如果没有多项式时间的敌手能够区分(从某种意义上来说,将马上非常严格的定义)是和 F_k (随机选择的密钥 k)交互,还是和 f 交互(这里 f 是从所有将 n 比特字符串映射到 n 比特字符串的函数中随机选择的),则称 F 是伪随机的。

因为随机选择一个函数的概念同随机选择一个字符串的概念比较起来,显得比较陌生,所以值得在这个概念上花更多的时间。从数学的观点来讲,考虑一个将 n 比特字符串映射到 n 比特字符串所有函数的集合 Func_n 。该集合是有限的(稍后将看到),因此,随机选择一个将 n 比特字符串映射到 n 比特字符串的函数,正好对应着均匀随机地从集合中选择一个元素。集合 Func_n 的规模有多大?通过对其每个定义域中的点给出值,一个函数 f 能够被完全确定下来。事实上,能够将任何函数(在有限定义域中)看作一个大的查找表,在该表中标签为 x 的那一行存储了 $f(x)$ 。对于 $f \in \text{Func}_n$, f 的查找表中有 2^n 行(每一行对应着定义域 $\{0,1\}^n$ 中的一个点),并且每一行包含一个 n 比特字符串(因为 f 的值域是 $\{0,1\}^n$)。任何这样的表能够正好用 $n \cdot 2^n$ 比特来表示。此外, Func_n 中的函数都是和这种形式的查找表一一对应的,也就是意味着它们和所有长度为 $n \cdot 2^n$ 的字符串是一一对应的。既然长度为 $n \cdot 2^n$ 的字符串有 $2^{n \cdot 2^n}$ 个,这就是 Func_n 的大小。

将一个函数看成一个查找表提供了另外一种有用的方式来思考均匀随机地选择一个函数 $f \in \text{Func}_n$,这正好等价于在 f 的查找表中均匀随机地选择每一行。也就是说, $f(x)$ 和 $f(y)$ 的值($x \neq y$)是完全独立且均匀分布的。

回到关于伪随机函数的讨论,还记得目的是希望构造一个带密钥的函数 F ,使得能够区分 F_k ($k \leftarrow \{0,1\}^n$ 是均匀随机选择的)和 f ($f \leftarrow \text{Func}_n$ 是均匀随机选择的)。前者是从(最多) 2^n 个不同的函数的分布中选择出来的,但是后者是从 Func_n 中所有 $2^{n \cdot 2^n}$ 个函数的分布中选择出来的。尽管如此,这些函数的“行为”对任意多项式时间区分器来说,必须看起来都是

相同的。

第一种试图将伪随机函数的概念进行形式化将会使用和定义 3.14 一样的方式进行。也就是说，可以要求每个多项式时间的区分器 D ，接收到一个伪随机函数 F_k 的描述，输出 1 的概率“几乎”和它接收到一个随机函数 f 的描述，输出 1 的概率相同。但是，该定义是不合理的，因为一个随机函数的描述具有指数长度（即查找表的长度，为 $n \cdot 2^n$ ），然而 D 被限制在多项式内运行。所以， D 甚至将没有足够的时间来检查它的整个输入。

因此，实际的定义允许 D 访问函数（ F_k 或者 f ）的预言机。对任何点 x ， D 被允许询问预言机，并且对预言机返回的点 x 的函数值作出响应。在选择明文攻击的定义中，提供敌手访问加密预言机的能力，把该预言机当作黑盒子。但这里该预言机计算一个确定函数，因此，当对相同的输入询问两次时，总是返回相同的值。现在已经准备好给出正式的定义。（尽管该定义要求 F 是长度保留的，这仅仅是为了使假设简化，而不是必要的。）

定义 3.23 令 $F: \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ 是有效的、长度保留的、带密钥的函数。如果对所有多项式时间区分器 D ，存在一个可忽略函数 negl ，满足：

$$|\Pr[D^{F_k(\cdot)}(1^n) = 1] - \Pr[D^{f(\cdot)}(1^n) = 1]| \leq \text{negl}(n),$$

则称 F 是一个伪随机函数，其中 $k \leftarrow \{0, 1\}^n$ 是均匀随机选择的，并且 f 是从将 n 比特字符串映射到 n 比特字符串的函数集合中均匀随机选择出来的。

注意 D 是和预言机自由交互的，因此它能够适应性地询问，并根据前一个接收到的输出来选择下一个输入。但是，因为 D 在多项式时间内运行，所以它只能执行多项式时间数量的询问。同时注意，伪随机函数必须拥有随机函数固有的高效率的特点。例如，即使 x 和 x' 仅仅只有单个比特不同， $F_k(x)$ 以及 $F_k(x')$ 的输出必须（概率计算主要来源于 k 的随机选择）看上去完全不相关。这预示伪随机函数对构造安全加密方案是有用的。

该定义中的关键点是，区分器 D 不知道密钥 k 。如果已知 k ，那么要求 F_k 是随机的将没有意义，因为给定 k ，区分 F_k 的预言机和 f 的预言机是很容易的。区分器向预言机询问点 0^n 来获得答案 y ，然后将 y 和结果 $y' = F_k(0^n)$ （通过使用已知的密码 k 计算出来）相比较。一个 F_k 的预言机将总是返回 $y = y'$ ，然而随机函数的预言机返回 $y = y'$ 的概率仅为 2^{-n} 。在实际中，这意味着一旦 k 被泄露，则 F_k 的伪随机性就不再满足。具体而言，考虑伪随机函数 F ，给定 F_k 的预言机供访问（这里 k 是随机的），找到一个输入 x 满足 $F_k(x) = 0^n$ 是困难的（因为对真正的随机函数 f 而言，找到这样的输入是很困难的）。但是如果 k 是已知的，则找到这样的一个输入可能很简单（现实中常常如此）。

伪随机函数的存在性。和伪随机发生器一样，很重要的一点是要弄清楚伪随机函数是否存在，如果存在，应该在怎样的假设下存在。实际中，一个非常有效的原语叫做分组密码，被使用且被广泛地认为可作为伪随机函数。在 3.6.3 节中将进一步讨论，并且第 5 章给出更深入地对分组密码的讨论。从理论角度看，已经知道，当且仅当伪随机发生器存在，则伪随机函数存在。因此，伪随机函数能够基于任何困难问题来构造，这些困难问题就是可以构造伪随机发生器的问题，在第 6 章中将详细讨论。基于困难问题的伪随机函数的存在代表了现代密码学的一个令人吃惊的真正迷人的贡献。

在密码学中使用伪随机函数。在很多不同的密码学构造方案中，伪随机函数成为一个非

常有用的构造模块。下面使用它来获得 CPA 安全的加密，并在第 4 章中，用它来构造消息鉴别码。伪随机函数用途很大的一个原因是，使用了伪随机函数的构造方案在分析时会很清晰和精巧。也就是说，给定一个基于伪随机函数的方案，一般的分析方法是：首先证明在真正随机函数的假设下，该方案是安全的。这一步骤依赖于一个概率分析，与计算的限制或困难性无关。接下来，原始方案的安全性是通过证明来获得，这个证明是：当伪随机函数被使用时，如果一个敌手能够攻破该方案，则它必须能够将伪随机函数与真正的随机函数区分开来。

3.6.2 基于伪随机函数的 CPA 安全加密

本节关注构造一个 CPA 安全的定长加密方案。根据 3.5 节末的陈述，这意味着存在一个 CPA 安全的任意长度消息的加密方案。3.6.4 节将考虑更多有效的方式来处理任意长度的消息。

从伪随机函数构造一个安全的加密方案的简单想法是定义 $\text{Enc}_k(m) = F_k(m)$ ，期待这“没有泄露任何关于 m 的信息”（因为随机函数 f 的 $f(m)$ 只是一个简单的随机 n 比特字符串）。但是，该加密方法是确定性的，所以不是 CPA 安全的。具体而言，指定 $c = \text{Enc}_k(m_b)$ ，请求得到加密 $\text{Enc}_k(m_0)$ 和 $\text{Enc}_k(m_1)$ 是不可能的；因为 $\text{Enc}_k(\cdot) = F_k(\cdot)$ 是一个确定函数，其中有一个肯定会等于 c ，因而就泄露了 b 的值。

实际中的构造是概率性的。特别是对随机值（而不是明文消息）应用伪随机函数，然后将这个结果和明文消息异或来进行加密。（参见构造方法 3.24，如图 3.4 所示。）这可以再一次被看作伪随机“填充”和明文消息进行异或的一个实例，主要的不同是每次使用一个独立的伪随机填充。只要伪随机函数每次的输入都不同，这一点就能办到。当然，一个随机值 r 可能重复，并多次使用，这个将在证明中明确地被考虑到。

构造方法 3.24

令 F 是伪随机函数，定义一个消息长度为 n 的对称密钥加密方案如下：

- Gen: 输入 1^n ，均匀随机地选择 $k \leftarrow \{0,1\}^n$ ，并将其作为密钥输出。
- Enc: 输入一个密钥 $k \in \{0,1\}^n$ ，以及一个消息 $m \in \{0,1\}^n$ ，均匀随机地选择 $r \leftarrow \{0,1\}^n$ ，并且输出密文

$$c := \langle r, F_k(r) \oplus m \rangle.$$

- Dec: 输入一个密钥 $k \in \{0,1\}^n$ ，以及一个密文 $c = \langle r, s \rangle$ ，输出明文消息

$$m := F_k(r) \oplus s.$$

从任意伪随机函数构造 CPA 安全的加密方案

从直观上看安全性是满足的，因为 $F_k(r)$ 对一个观察密文 $\langle r, s \rangle$ 的敌手来说，看上去完全随机。因此，加密方案和“一次一密”是非常相似的，只要值 r 没有在之前的某次加密中使用（特别是当回答敌手的问询时，只要它没有被加密预言机使用）。此外，该“坏事件”（也就是值 r 的一次重复）发生的概率是可忽略的。

定理 3.25 如果 F 是伪随机函数，则构造方法 3.24 为消息长度为 n 的定长对称密钥加密方案，在选择明文攻击下，该构造方法具备不可区分加密。

证明 这里的证明遵照着一般处理伪随机函数的范例：首先是在理想的世界中分析方案

的安全性，一个真正的随机函数 f 取代了 F_k ，然后说明该方案在这种情况下是安全的。接下来声称，当 F_k 被使用时，如果该方案是不安全的，则意味着将 F_k 从一个真正随机函数中区分出来是可能的。

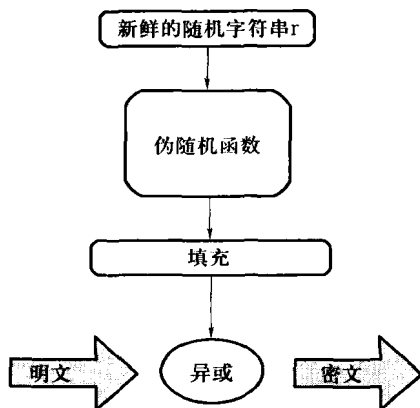


图 3.4 用伪随机函数的加密

令 $\tilde{\Pi} = (\widetilde{\text{Gen}}, \widetilde{\text{Enc}}, \widetilde{\text{Dec}})$ 为加密方案，除了真正的随机函数 f 取代了 F_k 之外，完全和构造方法 3.24 中的 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ 是相同的。也就是： $\widetilde{\text{Gen}}(1^n)$ 选择了一个随机函数 $f \leftarrow \text{Func}_n$ ，除了 f 取代了 F_k 外， $\widetilde{\text{Enc}}$ 加密和 Enc 是一样的。（这不是一个合法的加密方案，因为它不是有效的。尽管如此，这是一个为了证明而构造的脑力实验，为达到证明的目的而定义。）每个（甚至是无效的）敌手 $\mathcal{A}$ 最多向它的加密预言机询问 $q(n)$ 次，有

$$\Pr \left[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{cpa}}(n) = 1 \right] \leq \frac{1}{2} + \frac{q(n)}{2^n} \quad (3.4)$$

为了说明这一点，回想每次一个消息 m 被加密（要么通过加密预言机，要么是在实验 $\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{cpa}}(n)$ 中的挑战密文被计算出来），选择一个随机 $r \leftarrow \{0, 1\}^n$ ，并且密文被设置为等于 $\langle r, f(r) \oplus m \rangle$ 。令 r_c 表示生成挑战密文 $c = \langle r_c, f(r_c) \oplus m_b \rangle$ 时用到的随机字符串。有两种分情况：

(1) 值 r_c 被加密预言机使用来回答至少一个 $\mathcal{A}$ 的询问：在这种情况下， $\mathcal{A}$ 可能轻易地判断出哪一个消息被加密。这是因为无论何时加密预言机返回一个密文 $\langle r, s \rangle$ 来回应加密消息 m 的请求，敌手掌握了值 $f(r)$ （因为 $f(r) = s \oplus m$ ）。

因为 $\mathcal{A}$ 最多向预言机询问 $q(n)$ 次，并且每次预言机回答它的询问都是使用一个均匀随机选择的 r ，所以该事件发生的概率最多为 $q(n)/2^n$ （能够通过一致限来理解这一点）。

(2) 在加密预言机回答 $\mathcal{A}$ 的询问时，值 r_c 从来没有被用到：在这种情况下， $\mathcal{A}$ 在它和加密预言机的交互过程中，没有掌握到任何关于 $f(r_c)$ 值的信息（因为 f 是一个真正随机的函数）。这就意味着，据 $\mathcal{A}$ 所知， $f(r_c)$ 和 m_b 作异或操作后的值是完全随机的，因此在这种情况下， $\mathcal{A}$ 输出 $b' = b$ 的概率正好是 $1/2$ （和“一次一密”中的情况一样）。

令 Repeat 表示值 r_c 被加密预言机使用来回答至少一个 $\mathcal{A}$ 的询问的事件。上面已经说明了 Repeat 发生的概率最多为 $q(n)/2^n$ ，并且如果 Repeat 没有发生，则 $\mathcal{A}$ 在 $\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{cpa}}(n)$ 中成功的概率刚好是 $1/2$ 。因此，我们有：

$$\begin{aligned}
& \Pr[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{cpa}}(n) = 1] \\
&= \Pr[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{cpa}}(n) = 1 \wedge \text{Repeat}] + \Pr[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{cpa}}(n) = 1 \wedge \overline{\text{Repeat}}] \\
&\leq \Pr[\text{Repeat}] + \Pr[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{cpa}}(n) = 1 \mid \overline{\text{Repeat}}] \\
&\leq \frac{q(n)}{2^n} + \frac{1}{2},
\end{aligned}$$

得到等式(3.4)。

现在，固定某个 PPT 敌手 $\mathcal{A}$ ，定义函数 ε

$$\varepsilon(n) \stackrel{\text{def}}{=} \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1] - \frac{1}{2} \quad (3.5)$$

由 $\mathcal{A}$ 发出的预言机问询次数的上界值是它的运行时间。因为 $\mathcal{A}$ 在多项式时间内运行，所以它发出的预言机问询的次数的上界是某个多项式 $q(\cdot)$ 。关于这个 $\mathcal{A}$ ，等式(3.4)也满足；因此

$$\Pr[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{cpa}}(n) = 1] \leq \frac{1}{2} + \frac{q(n)}{2^n}$$

并且通过 ε 的定义，

$$\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1] = \frac{1}{2} + \varepsilon(n)$$

如果 ε 不可忽略，则两者之间的差别也是不可忽略的。直觉上表明，这样一个“差距”（如果存在的话）将可用来区分伪随机函数和一个真正的随机函数。

下面正式用规约法来证明。使用 $\mathcal{A}$ 来构造一个伪随机函数 F 的区分器 D 。区分器 D 可以使用某函数的预言机，其目标是判断出该函数是“伪随机的”（即 F_k ，随机选择的 $k \leftarrow \{0, 1\}^n$ ），还是“随机的”（ f ，随机选择的 $f \leftarrow \text{Func}_n$ ）。为了达到目的， D 使用下面的方式，来模拟 $\mathcal{A}$ 的 CPA 不可区分性实验，并且观察 $\mathcal{A}$ 成功与否。如果 $\mathcal{A}$ 成功了，则 D 猜测它的预言机一定是一个伪随机函数，然而，如果 $\mathcal{A}$ 没有成功，则 D 猜测它的预言机一定是一个随机函数，具体如下：

区分器 D ：

D 被指定了输入 1^n 以及可访问预言机 $\mathcal{O} : \{0, 1\}^n \rightarrow \{0, 1\}^n$ 。

(1) 运行 $\mathcal{A}(1^n)$ 。无论何时 $\mathcal{A}$ 向它的加密预言机发出关于消息 m 的问询，用下面的方法来回答该询问：

- ① 均匀随机地选择 $r \leftarrow \{0, 1\}^n$ 。
 - ② （向预言机）问询 $\mathcal{O}(r)$ ，获得相应 s' 。
 - ③ 返回密文 $\langle r, s' \oplus m \rangle$ 给 $\mathcal{A}$ 。
- (2) 当 $\mathcal{A}$ 输出消息 $m_0, m_1 \in \{0, 1\}^n$ 时，随机选择一个比特 $b \leftarrow \{0, 1\}$ ，然后：
- ① 均匀随机地选择 $r \leftarrow \{0, 1\}^n$ 。
 - ② （向预言机）问询 $\mathcal{O}(r)$ ，获得相应 s' 。
 - ③ 返回挑战密文 $\langle r, s' \oplus m_b \rangle$ 给 $\mathcal{A}$ 。

(3) 正如之前那样，继续回答任何 $\mathcal{A}$ 对加密预言机的问询。最终， $\mathcal{A}$ 输出一个比特 b' 。如果 $b' = b$ ，则输出 1，否则输出 0。

关键点如下：

(1) 如果 D 的预言机是一个伪随机函数，那么当 $\mathcal{A}$ 被 D 作为一个子程序运行时的视图分布，与在实验 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$ 中 $\mathcal{A}$ 的视图分布是相同的。因为密钥 k 是随机选择的，并且每次加密的完成都是通过选择一个随机的 r ，计算 $s' = F_k(r)$ ，然后设定密文等于 $\langle r, s' \oplus m \rangle$ ，与构造方法 3.24 一样。因此，

$$\Pr \left[D^{F_k(\cdot)}(1^n) = 1 \right] = \Pr \left[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1 \right]$$

其中，上面的 $k \leftarrow \{0, 1\}^n$ 是均匀随机选择的。

(2) 如果 D 的预言机是随机函数，那么当 $\mathcal{A}$ 被 D 作为一个子程序运行时的视图分布，和在实验 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$ 中的视图分布是相同的。这一点正好可以通过上面看出来，唯一的区别就是随机函数 f 代替了 F_k 。因此，

$$\Pr \left[D^{f(\cdot)}(1^n) = 1 \right] = \Pr \left[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1 \right]$$

其中，上面的 $f \leftarrow \text{Func}_n$ 是均匀随机选择的。

联合等式(3.4)和(3.5)和上面的等式，有

$$\Pr \left[D^{F_k(\cdot)}(1^n) = 1 \right] - \Pr \left[D^{f(\cdot)}(1^n) = 1 \right] \geq \varepsilon(n) - \frac{q(n)}{2^n}.$$

通过假设 F 是伪随机函数，那么 $\varepsilon(n) - q(n)/2^n$ 必须是可忽略的。因为 q 是多项式，这反过来暗示着 ε 是可忽略的，所以 Π 是 CPA 安全的，证毕。 ■

正如在 3.5 节中讨论的，任何 CPA 安全的定长加密方案会自动产生一个任意消息长度的 CPA 安全加密方案。将这个方案应用到刚构造出来的定长方案上，有一个任意长度的消息 $m = m_1, \dots, m_\ell$ ，其中每一个 m_i 都是 n 比特的分块，安全的加密可通过计算

$$\langle r_1, F_k(r_1) \oplus m_1, r_2, F_k(r_2) \oplus m_2, \dots, r_\ell, F_k(r_\ell) \oplus m_\ell \rangle$$

得到。（该方案能够通过删减来处理长度不是正好为 n 的倍数的消息，这里忽略细节。）于是有下面的推论。

推论 3.26 如果 F 是伪随机函数，则上面粗略描述的方案，是一个在选择明文攻击下，具备不可区分加密的任意长度消息的对称密钥加密方案。

构造方法 3.24 的有效性。在构造方法 3.24 中的 CPA 安全的加密方案以及在推论 3.26 中对任意长度消息的扩展，都存在一个缺陷，就是密文的长度（至少）是明文长度的两倍。这是因为每个大小为 n 的分组是使用一个 n 比特的字符串来加密的，且该字符串必须作为密文的一部分。3.6.4 节将给出密文长度是如何明显减小的方法。

3.6.3 伪随机置换和分组加密

令 $F: \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ 是有效的、长度保留的、带密钥的函数。如果对每个 k ，函数 $F_k(\cdot)$ 是单射（因为 F 是长度保留的，因而 F_k 实际上是双射）。把 F 称作一个带密钥的置换。如果给定 k 和 x ，存在一个多项式时间算法能够计算 $F_k(x)$ ，并且如果给定 k 和 x ，也存在一个多项式时间算法能够计算 $F_k^{-1}(x)$ ，则这个带密钥的置换是有效的。

这里约定密钥、输入和输出的长度全部是相等的，在实际中，这对构造方案来说并不是必要的。相反，输入和输出长度，一般叫做分块长度，是相同的（因为是置换，所以必须如此），但是密钥长度能够小于或者大于分块长度，取决于该构造方法。假设它们都是相等的，仅仅是为了简化概念。

使用和定义 3.23 类似的方法来定义：有效的带密钥置换 F 是伪随机置换。唯一的变化就是，现在要求无法区分 F_k （ k 是随机选择的）是随机选择的置换，而不是随机选择的函数。实际上，这仅仅只是一个有美感的结论，因为使用多项式数量的问询，随机置换和（长度保留）随机函数是不可区分的。从直觉上说，这是因为一个随机函数 f 看起来和一个随机置换是相同的，除非能够找到一对不同的值 x 和 y ，使得 $f(x) = f(y)$ （因为在这种情况下，函数不可能是置换）。但是，用多项式数量的问询找到这样的点 x 和 y 的概率是可忽略的。下面命题的证明留作练习。

命题 3.27 如果 F 是一个伪随机置换，则也是一个伪随机函数。

如果 F 是一个有效的伪随机置换，那么基于 F 的密码学方案可能需要诚实方除了要计算置换 F_k 本身以外，还要计算逆 F_k^{-1} 。这潜在地引入了新的安全问题，这种问题没有被 F 是伪随机的这一事实所覆盖。在这样的情况下，可能需要更强的需求，即 F_k 和一个随机置换是不可区分的，即使区分器可以使用逆置换预言机。如果 F 有这个属性，称其为强伪随机置换。正式陈述如下：

定义 3.28 令 $F : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ 是有效的带密钥的置换。如果对于所有概率多项式时间区分器 D ，存在一个可忽略函数 negl ，使得

$$|\Pr [D^{F_k(\cdot), F_k^{-1}(\cdot)}(1^n) = 1] - \Pr [D^{f(\cdot), f^{-1}(\cdot)}(1^n) = 1]| \leq \text{negl}(n),$$

则称 F 是强伪随机置换，其中 $k \leftarrow \{0, 1\}^n$ 是随机均匀选择的， f 是从 n 比特字符串置换的集合中均匀随机选择的。

当然，任何强伪随机置换同时也是一个伪随机置换。

前面已经注意到，一个流密码能够被建模成为一个伪随机发生器。类比在实际中强伪随机置换的情况，则是分组密码。不幸的是，在文献中并不是经常提到，一个分组密码实际上被认为是一个强伪随机置换。明确地用这种方法来建模分组密码，使得依赖于分组密码的许多实际构造方案的形式化分析成为可能。这些构造包括（本章学习的）加密方案、消息鉴别码（在第 4 章学习）、认证协议以及更多。此外，当证明一个构造的安全性时，声明分组密码是被建模成为伪随机置换还是强伪随机置换是很重要的。尽管现在使用的大部分分组密码是设计用来满足后者，更强的需求，但基于前者的方案能够被证明是安全的，弱的假设是更可取的（因为对分组密码的要求更容易满足）。

相对于流密码，分组密码自身是不安全的加密方案。但是，构造分组密码能够被用于构造安全的加密方案。例如，在构造方法 3.24 中使用分组密码产生了一个 CPA 安全的对称密钥加密方案。与此相反，只是计算 $c := F_k(m)$ 的加密方案不是 CPA 安全的，这里 F_k 是强伪随机置换。分组密码作为构造模块的加密方案和使用分组密码本身进行加密的加密方案间的区别

非常重要，也常常被忽视。

强伪随机置换在设计和分析有效密码学方案时非常有用，在本书的余下部分，将只使用伪随机置换（没必要强伪随机置换）。

3.6.4 加密操作模式

操作模式是使用分组密码（即伪随机置换）来加密任意长度消息的基本方法。在推论 3.26 中，已经看到加密模式的一个例子，尽管从密文的长度角度来说，不是很有效率。在这一节中，将看到许多具有较低密文扩展（定义为密文长度和消息长度的差别）的加密操作模式。

在继续之前，注意到通过在尾部添加一个 1，然后使用足够多的 0 填充，可将任意长度的消息填充为总长度为分组大小的倍数。因此，假设明文消息的长度正好是分组大小的倍数。贯穿这一节，认为伪随机置换/分组密码 F 的分组长度为 n ，并且考虑由 ℓ 个分组（每个分组的长度为 n ）组成的消息的加密。这里将说明四种加密操作模式，并且讨论其安全性。

模式 1：电子密码本（ECB）模式。 这是可能的最简单的加密操作模式。给定一个明文消息 $m = m_1, m_2, \dots, m_\ell$ ，密文是通过分别“加密”每个分组获得的，其中这里的“加密”意味着直接对明文分组应用伪随机置换。也就是说， $c = \langle F_k(m_1), F_k(m_2), \dots, F_k(m_\ell) \rangle$ 。如图 3.5 所示，其中 $\ell = 3$ 。解密过程执行的方式很显然就是使用有效可计算的 F_k^{-1} 。

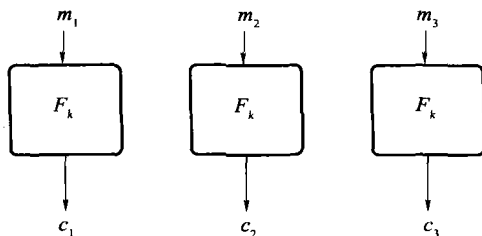


图 3.5 电子密码本（ECB）模式

这里，加密过程是确定性的，因此，该操作模式不可能是 CPA 安全的（参见在定义 3.21 后的讨论）。更糟糕的是，在窃听者存在的情况下，ECB 加密模式并不具备不可区分性。这是因为如果相同的分组在明文中重复了两次，那么在密文中就能够发觉有重复的分组。因此，非常容易区分：是两个相同分组组成的明文的加密，还是两个不同分组组成的明文的加密。注意这不仅仅是一个“理论问题”，通过观察这种方式产生的密文，能够掌握到很多信息。因此，ECB 模式应该永远不被使用。（这里提到它只是因为它的历史重要性。）

模式 2：密码分组链接（CBC）模式。 在这种模式中，首先要选择一个长度为 n 的随机初始向量（IV）。然后，通过对当前明文和前一个密文的异或值做伪随机置换，来获得下一个密文分组。也就是说，令 $c_0 := IV$ ，然后 $i = 1$ 到 ℓ ，令 $c_i := F_k(c_{i-1} \oplus m_i)$ 。最终的密文是 $\langle c_0, c_1, \dots, c_\ell \rangle$ ，如图 3.6 所示。注意 IV 不加密并作为密文的一部分发送，这对解密来说是非常关键的。

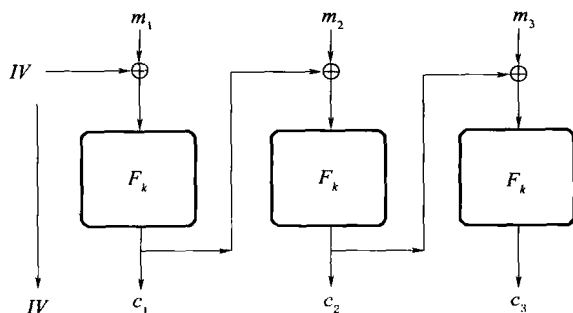


图 3.6 加密块链 (CBC) 模式

重要的是 CBC 模式的加密是概率的。已证明，如果 F 为伪随机置换，那么 CBC 加密模式是 CPA 安全的。该模式的主要缺陷是加密必须依次执行，因为要加密明文分组 m_i ，我们需要密文分组 c_{i-1} 。因此，如果并行处理是可行的，则 CBC 模式不是最有效率的选择。

有人可能会想在每次加密时使用不同的 IV （而不是随机的 IV ）就可以了；比如，首先使用 $IV = 1$ ，然后每次对 IV 增加 1。练习 3.16 要求说明这个 CBC 加密操作模式的变体是不安全的。

模式 3：输出反馈 (OFB) 模式。 在这里说明的第三种模式被叫做 OFB。本质上说，该模式是使用分组密码来生成伪随机流，然后和消息作异或操作。首先，选择一个随机 $IV \leftarrow \{0, 1\}^n$ ，然后通过 IV 用下面的方式生成一个流（独立于明文）：定义 $r_0 := IV$ ，令流的第 i 个分组 r_i 为 $r_i := F_k(r_{i-1})$ 。则每个明文分组通过和对应的流分组做异或操作来达到加密的目的；也就是 $c_i := m_i \oplus r_i$ 。（参见图 3.7 的图形描述。）和 CBC 模式一样，为了解密， IV 是密文的一部分；相对于 CBC 模式，这里不需要 F 是可逆的（事实上， F 甚至不需要是个置换）。

该模式也是概率的，并且能够证明如果 F 是伪随机函数，则它是 CPA 安全的加密方案。这里，加密和解密都必须依次执行。从另一方面说，该模式有个优点是大块的计算（也就是伪随机流的计算）能够独立于实际加密消息进行。所以，在预处理之前，准备好一个流是可能的，此后加密明文（一旦已知）是非常快的。

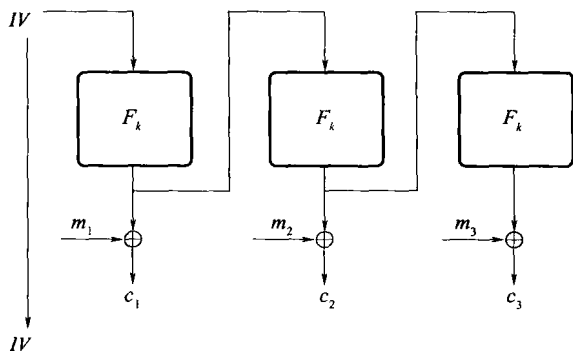


图 3.7 输出反馈 (OFB) 模式

模式 4: 计数器 (CTR) 模式。有很多不同的 CTR 加密模式的变体, 这里描述随机化的计数器模式。和 OFB 一样, 计数器模式能够看成是一种用分组密码生成伪随机流的方法。首先, 选择一个随机 $IV \leftarrow \{0, 1\}^n$, 该 IV 经常用 ctr 表示。通过计算 $r_i := F_k(ctr + i)$ (其中 ctr 和 i 被看作是整数, 加法是模 2^n 的加法) 生成一个流。最后第 i 个密文分组计算为 $c_i := r_i \oplus m_i$, 并且 IV 再次被作为密文的一部分发送, 如图 3.8 所示。需要强调的是, 该解密不要求 F 是可逆的, 甚至不要求是置换。

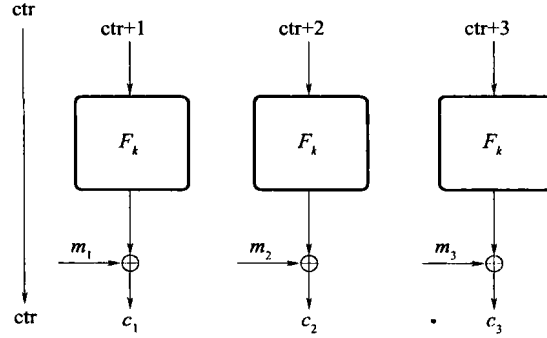


图 3.8 计数器 (CTR) 模式

计数器模式有很多重要的特点。首先也是最重要的, 随机计数模式 (即每次消息被加密, 当 ctr 是随机均匀地选择的时候) 是 CPA 安全的, 这将在下面证明。第二, 加密和解密都能完全并行, 和 OFB 模式一样, 在预处理之前生成独立于消息的伪随机流是可能的。最后, 只解密第 i 个密文分组而不解密任何其他密文是可能的, 该特点被叫做随机访问。上述特点让计数模式看上去非常有吸引力。

定理 3.29 如果 F 是一个伪随机函数, 那么在选择明文攻击下, 随机计数模式 (描述如上) 具备不可区分加密。

证明 正如定义 3.25 的证明一样, 对现在的定理, 首先说明当使用一个真正随机的函数时, 随机计数模式是 CPA 安全的。然后证明, 用一个伪随机函数替换随机函数不会令该方案不安全。

当挑战密文在实验 $\text{PrivK}^{\text{CPA}}$ 中被加密时, 令 ctr^* 表示使用的初始值 ctr 。从直觉上看, 当在随机计数模式中使用一个随机函数 f 时, 只要使用 $ctr^* + i$ 来加密挑战密文的每个分组 c_i , 且 $ctr^* + i$ 从未被加密预言机用于回答以前的问询, 就可以达到安全性。这是因为: 如果 $ctr^* + i$ 从未被加密预言机用于回答以前的加密问询, 那么 $f(ctr^* + i)$ 就将是一个完全随机的值, 那么用这个值和明文的一个分组作异或操作, 和通过 “一次一密” 相比, 具有相同的效果。当使用随机函数时, 随机计数模式是 CPA 安全的证明转化为界定 $ctr^* + i$ 以前被使用过的概率。

$\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ 表示随机计数模式的加密方案, 然后令 $\tilde{\Pi} = (\tilde{\text{Gen}}, \tilde{\text{Enc}}, \tilde{\text{Dec}})$ 是和 Π 相同的加密方案, 唯一的区别是 $\tilde{\Pi}$ 使用的是一个真正的随机函数, 而不是一个伪随机置换 F 。也就是说, $\tilde{\text{Gen}}(1^n)$ 选择一个随机函数 $f \leftarrow \text{Func}_n$, 并且除了 f 代替了 F_k 外, $\tilde{\text{Enc}}$ 和 Enc 加密方式是一样的。(当然, $\tilde{\text{Gen}}$ 和 $\tilde{\text{Enc}}$ 都不是有效的算法, 但是这对定义含有 $\tilde{\Pi}$ 的实验这一目的来说没有影响。) 现在说明对每个概率多项式时间的敌手 $\mathcal{A}$, 存在一个可忽略函数 negl , 使得

$$\Pr \left[\text{PrivK}_{\mathcal{A}, \tilde{\Pi}}^{\text{CPA}}(n) = 1 \right] \leq \frac{1}{2} + \text{negl}(n) \quad (3.6)$$

实际上, 不需要作出任何关于 $\mathcal{A}$ 的运行时间 (或者是计算能力) 的假设; 仅仅要求 $\mathcal{A}$ 对其加密预言机 (每一次问询都是多项式长度的消息) 作出多项式数量的问询就足够了, 此外, $\mathcal{A}$ 输出多项式长度的 m_0, m_1 。

令 q 是 $\mathcal{A}$ 作出的预言机问询数量的多项式上界, 也是任何这样问询的最大长度以及 m_0, m_1 的最大长度。固定安全参数为某个值 n 。令 ctr^* 表示当挑战密文被加密时, 使用的初始值 ctr , 并且令 ctr_i 表示当加密预言机回答 $\mathcal{A}$ 的第 i 个预言机问询时, 使用的初始值 ctr 。当挑战密文被加密时, 函数 f 被应用在值 $\text{ctr}^* + 1, \dots, \text{ctr}^* + \ell^*$ 上, 其中 $\ell^* \leq q(n)$ 是 m_0, m_1 的长度。当第 i 个预言机问询被回答时, 函数 f 被应用在值 $\text{ctr}^* + 1, \dots, \text{ctr}^* + \ell_i$ 上, 其中 $\ell_i \leq q(n)$ 是 (以分组表示) 被请求加密的消息的长度。这里有两种情况被考虑:

情况 1。不存在任意的 $i, j, j' \geq 1$ ($j \leq \ell_i$, 且 $j' \leq \ell^*$) 使得 $\text{ctr}_i + j = \text{ctr}^* + j'$ 。在这种情况下, 当加密挑战密文时, 使用的值 $f(\text{ctr}^* + 1), \dots, f(\text{ctr}^* + \ell^*)$ 是彼此独立的, 并且是均匀分布的, 因为当加密任何敌手的预言机问询时, f 没有在这些输入上使用过。这意味着: 挑战密文的计算是通过异或随机比特流和消息 m_b 来计算的, 所以在这种情况下, $\mathcal{A}$ 输出 $b' = b$ 的概率正好是 $1/2$ (同 “一次一密” 中的情况一样)。

情况 2。存在任意的 $i, j, j' \geq 1$ ($j \leq \ell_i$, 并且 $j' \leq \ell^*$) 使得 $\text{ctr}_i + j = \text{ctr}^* + j'$ 。也就是, 用来加密第 i 个加密预言机问询的第 j 分组的值和用来加密挑战密文的第 j' 分组的值是相同的。在这种情况下, $\mathcal{A}$ 很容易确定哪一个消息被加密且给出挑战密文 (因为敌手能够从它的第 i 个预言机问询的答案中推断出 $f(\text{ctr}_i + j) = f(\text{ctr}^* + j')$ 的值)。

现在, 分析上述情况发生的概率。如果 ℓ^* 和每一个 ℓ_i 尽可能地大, 概率能够达到最大, 所以假设对所有 i , 都有 $\ell^* = \ell_i = q(n)$ 。令 Overlap_i 表示序列 $\text{ctr}_i + 1 \dots \text{ctr}_i + q(n)$ 重叠于序列 $\text{ctr}^* + 1 \dots \text{ctr}^* + q(n)$ 的事件, 令 Overlap 表示对某个 i , Overlap_i 发生的事件。因为最多有 $q(n)$ 个预言机问询, 由一致限, 得

$$\Pr[\text{Overlap}] \leq \sum_{i=1}^{q(n)} \Pr[\text{Overlap}_i] \quad (3.7)$$

固定 ctr^* , 事件 Overlap_i 发生是当 ctr_i 满足

$$\text{ctr}^* + 1 - q(n) \leq \text{ctr}_i \leq \text{ctr}^* + q(n) - 1$$

因为 ctr_i 有 $2q(n) - 1$ 个值使得 Overlap_i 发生, 并且 ctr_i 是随机均匀地从 $\{0, 1\}^n$ 中选择的, 于是

$$\Pr[\text{Overlap}_i] = \frac{2q(n) - 1}{2^n}$$

将其和等式(3.7)结合, 得出 $\Pr[\text{Overlap}] \leq 2q(n)^2 / 2^n$ 。

于是能够轻易地界定 $\mathcal{A}$ 成功的概率:

$$\begin{aligned} & \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1] = \\ & \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1 \wedge \text{Overlap}] + \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1 \wedge \overline{\text{Overlap}}] \leq \Pr[\text{Overlap}] + \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) \\ & = 1 | \overline{\text{Overlap}}] \leq \frac{2q(n)^2}{2^n} + \frac{1}{2} \end{aligned}$$

因为 q 是多项式, $2q(n)^2/2^n$ 是可忽略的, 式(3.6)得证。也就是说, 该(虚构的)方案 $\tilde{\Pi}$ 是 CPA 安全的。

证明的下一步骤是说明 Π (即感兴趣的方案)是 CPA 安全的; 也就是说, 对任何概率多项式时间 $\mathcal{A}$, 存在一个可忽略函数 negl' , 使得

$$\Pr \left[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1 \right] \leq \frac{1}{2} + \text{negl}'(n).$$

从直觉来说, 这是因为只要是一个多项式时间敌手, 用在 Π 中使用的伪随机函数 F_k 替换在 $\tilde{\Pi}$ 中使用的随机函数 f 应该“没有影响”。当然, 该直觉应该被严格地证明。该证明和在定理 3.25 的证明中的相应步骤是非常相似的, 所以将证明留作练习。 ■

分组长度和安全性。 所有上面的模式(除了 ECB 是不安全的)都使用了随机 IV。IV 有随机化加密过程的效果, 并且确保(概率很高)分组密码总是使用之前没有用过的输入。这一点很重要, 正如定理 3.25 和定理 3.29 的证明所述, 如果分组密码的一个输入被多次使用, 那么安全性就将破坏。(比如, 在计数模式的情况下, 相同的伪随机字符串将和两个不同的明文分组进行异或。)有趣的是, 这说明了在评价安全性中起重要作用的不仅仅是分组密码的密钥长度, 还包括分组长度。例如, 假设使用一个 64 比特分组长度的分组密码, 定理 3.29 的证明中说明, 在随机计数模式中, 即使使用这一分组长度的完全随机函数(也就是说即使分组密码是“完美的”), 敌手在选择明文攻击中, 向加密预言机作出 q 次问询, 并且每个问询都是 q 个分组长, 则仍然能够以约为 $\frac{1}{2} + \frac{q^2}{2^{63}}$ 的概率成功。尽管这是渐进可忽略的(当分组长度随着安全参数 n 增长时), 但是从任何实践意义来说, 当 $q \approx 2^{30}$ 时(对这个特别的分组长度)安全性不再满足。应用中有人可能切换到分组长度更大的分组密码(2^{30} 仅仅是 1 吉字节, 没有考虑现在的存储需求)。

其他的操作模式。 在最近几年, 引入了许多不同的操作模式, 每一个都在某种情形中提供了独特的优势。尽管如此, 普遍来说, CBC、OFB 以及 CTR 模式满足大部分需要 CPA 安全的应用。

加密模式和消息篡改。 在很多密码学文献中, 操作模式也会在有效防止敌手篡改密文方面相互比较。这里, 并不包含这样的比较, 因为消息完整性或者消息鉴别的问题应该和加密分开处理。从定义的意义来说, 上面提到的模式都没有能够实现消息完整性。下一章将更深入地讨论。

流密码和分组密码。 正如这里的 OFB 和计数器模式, 让分组密码工作在“流密码模式”是可能的(即生成一个伪随机比特流然后将其和明文异或)。更进一步, 一个分组密码能够被用来生成多个(独立的)伪随机流, 相比之下(一般的)一个流密码被限制只能生成单个这样的流。这提出了一个问题: 分组密码和流密码, 哪一个更合适? 流密码的唯一优势就是它们相对来说更有效, 尽管这个优势可能仅仅只是二分之一, 除非正在使用资源受限的设备, 比如 PDA 或者移动电话。^①从另外一个方面看, 流密码看起来远不如分组密码好理解(在实践中)。有很多非常出色的分组密码都是有效的, 并且被认为是高度安全的(将在第 5 章学习两种)。相比之下, 流密码看起来更经常被攻破, 对它们的信心也比较低。更可能的是, 流密码以某种方式被乱用, 如相同的伪随机流被使用两次。因此, 通常推荐使用分组密码, 除非某些原因使得分组密码不可能被使用。

^① 特别地, 根据文献[42]中的评估: 用比特/秒来测量, 在一个典型的家用 PC 上, 流密码 RC4 仅仅是分组密码 AES 的两倍快。

3.7 CCA 安全性

直到现在，已经定义了抵抗两种类型攻击的安全性：一种是被动攻击，只是窃听；另一种是主动攻击，执行选择明文攻击。第三种类型的攻击，叫做选择密文攻击，比前面两种更为强大。在选择密文攻击中，提供敌手的不仅仅是能够加密其选择的消息的能力（正如在选择明文攻击里面的那样），还提供了解密其选择的密文的能力（稍后讨论一个例外）。正式地说，除了加密预言机之外，也让敌手使用解密预言机。正式的定义以及深入的讨论推迟到后面。

考虑到下面对任意对称密钥加密方案 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ ，敌手 $\mathcal{A}$ ，安全参数值 n 的实验。

CCA 不可区分实验 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cca}}(n)$:

- (1) 通过运行 $\text{Gen}(1^n)$ 生成一个密钥 k 。
- (2) 敌手 $\mathcal{A}$ 被指定输入 1^n ，可使用预言机 $\text{Enc}_k(\cdot)$ 和 $\text{Dec}_k(\cdot)$ 。它输出一对长度相等的消息 m_0, m_1 。
- (3) 随机选择比特 $b \leftarrow \{0, 1\}$ ，则计算密文 $c \leftarrow \text{Enc}_k(m_b)$ 并将其提供给 $\mathcal{A}$ 。我们把 c 叫做挑战密文。
- (4) 敌手 $\mathcal{A}$ 继续使用预言机 $\text{Enc}_k(\cdot)$ 和 $\text{Dec}_k(\cdot)$ ，但是不允许用挑战密文本身来问询 $\text{Dec}_k(\cdot)$ 。最终， $\mathcal{A}$ 输出一个比特 b' 。
- (5) 如果 $b' = b$ ，实验的输出为 1，否则输出 0。

定义 3.30 一个对称密钥加密方案 Π 满足如下条件：对所有的概率多项式时间敌手 $\mathcal{A}$ ，存在一个可忽略的函数 negl ，使得

$$\Pr [\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cca}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$$

则称其为选择密文攻击条件下不可区分加密（或者是 CCA 安全的）其中概率来源于实验中的所有随机因素。

在上面的实验中，敌手使用解密预言机是不被限制的，除了有一个限制：敌手不能要求解密挑战密文本身。该限制是很必要的，否则很明显，任何加密方案都不可能满足定义 3.30。

在这一点上，读者可能想知道选择密文攻击是否能够真实地模拟任何真实世界中的攻击。正如在选择明文攻击中的情况那样，不期望诚实方来解密敌手选择的任意密文。无论如何，存在一种场景，敌手可能能够影响解密的内容，然后掌握一些关于结果的部分信息。举例如下：

(1) 在中途岛（参见 3.5 节）的情况中，可以想象美国密码分析专家可能也试图向日本发送已加密的消息，然后监视他们的动作。这样的行为（比如，类似军队的调动等）能够提供重要的关于潜在明文的信息。

(2) 想象一个用户和他们的银行通信，其中所有的通信都是经过加密的。如果该通信没有被认证，则敌手能够代表用户发送特定密文；银行将解密这些密文，敌手可能能够从结果中

掌握到某些信息。例如，如果一个密文对应着一个不合语法的明文（比如，一个莫名其妙的消息，或者一个格式不正确的简单消息），该敌手能够通过银行的反应（比如接下来的通信模式）推断出一些信息。

(3) 加密经常在高级别的协议中使用到；比如，一个加密方案可能被作为认证协议的一部分来使用，其中一方发送一份密文给对方，对方解密然后返回结果。在这种情况下，一个诚实方可能正好扮演了一个解密预言机的角色，所以该方案必须是 CCA 安全的。

前面研究方案的不安全性。 到目前为止，已给出的加密方案中，都不是 CCA 安全的。将在构造方法 3.24 中给出说明，其中加密方式为 $\text{Enc}_k(m) = \langle r, F_k(r) \oplus m \rangle$ 。该方案不是 CCA 安全的事实能够很容易地说明如下。一个在 CCA 不可区分实验中的敌手 $\mathcal{A}$ 能够选择 $m_0 = 0^n$ 以及 $m_1 = 1^n$ 。那么，接收到密文 $c = \langle r, s \rangle$ ，敌手 $\mathcal{A}$ 能够翻动 s 的第一个比特，然后要求解密这个改动过的密文 c' 。因为 $c' \neq c$ ，该询问是允许的，解密预言机的回答要么是 10^{n-1} （在这种情况下，显然 $b = 0$ ）或者是 01^{n-1} （在这种情况下，显然 $b = 1$ ）。这个例子说明了为什么 CCA 安全性更严格。特别地，任何允许密文以“逻辑方式”被操纵的加密方案都不是 CCA 安全的。因此，CCA 安全实际上意味着一个非常重要的特点叫做“不可延展性”。粗略地说，一个不可延展的加密方案具备这个特点：如果敌手试图去修改一个指定的密文，结果要么是一个非法密文，要么是一个和原文没有任何关系的明文的密文。目前给出的加密模式中，没有一种是 CCA 安全的，这一点的说明留作练习。

构造一个 CCA 安全加密方案。 4.8 节将说明如何构造一个 CCA 安全的加密方案。后面将说明其构造，这是因为构造 CCA 安全的加密方案需要使用第 4 章详述的工具。

参考文献和扩展阅读材料介绍

密码学中的现代计算方法最早出现在 Goldwasser 和 Micali 的论文[69]中。该论文引入了语义安全的概念，并且说明了语义安全能够在公钥加密的情形中实现（参见第 9 章和第 10 章）。该论文也提议了不可区分性的概念（即定义 3.8），可以看到不可区分性意味着语义安全。逆命题见文献[104]。

抵御选择明文攻击的安全性的正式定义在 Luby 的[96]和 Bellare 等的[9]中提到。选择密文攻击（在公钥加密的情形中）的正式定义首先被 Naor 和 Yung^[107]，Rackoff 和 Simon^[121]提出，在文献[50]和[9]中也被考虑到。如果要了解其他对称密钥加密安全的概念，可以参见文献[86]。

Blum 和 Micali 的文献[23]引入了伪随机发生器的概念，并且在基于特定数论假设基础上，证明了其存在性。在同一工作中，Blum 和 Micali 也指出了在构造方法 3.15 中，伪随机发生器和对称密钥加密之间的联系。由 Blum 和 Micali 给出的伪随机发生器的定义和本书中（定义 3.14）给出的定义不同，后者起源于 Yao 的文献[149]，Yao 说明了两种形式化的表述是等价的。Yao 也说明了基于一般假设的伪随机发生器的构造方法，我们将在第 6 章探讨这个结论。

伪随机函数在 Goldreich 等的文献[67]中被定义和构造，并且他们在接下来的工作^[66]中说明了伪随机函数在加密上的应用。Luby 和 Rackoff 在文献[97]中研究了伪随机置换和强伪随机置换。当然，他们从这些工作开始对分组密码进行理论研究，但分组密码已经使用了很多年。

在文献[108]中, ECB、CBC 以及 OFB 操作模式 (也包含 CFB 模式, 这里没有包含其他模式) 连同 DES 分组密码^[108]一起被标准化。CBC 和 CTR 加密模式在文献[9]中已经证明是 CPA 安全的。其他近期的一些加密模式, 参见 <http://csrc.nist.gov/CryptoToolkit>。一个不错的但稍有些过时的对实际中使用的流密码的概论能够在文献[99]的第 6 章中找到。RC4 流密码在文献[125]中被讨论, 对其最近的攻击以及后果的讨论可在文献[56]中找到。

练 习

- 3.1 证明命题 3.6。
- 3.2 现在已知的, 找到一个 n 比特素因数的最佳算法的运行时间为 $2^{c \cdot n^{\frac{1}{3}} (\log n)^{\frac{2}{3}}}$ 。假设 4 兆赫的计算机以及 $c = 1$ (给定表达式的单位为时钟周期), 估算数字的大小, 使得该数字在接下来的100年内不能被因数分解。
- 3.3 如果 Π 能够加密任意长度的消息, 并且在实验 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cca}}(n)$ 中敌手不被限制地输出等长消息, 证明定义 3.8 不能被满足。
提示: 当 Π 被用来加密单个比特时, 令 $q(n)$ 为密文长度的多项式上界。然后考虑一个输出 $m_0 \in \{0, 1\}$ 的敌手以及一个随机 $m_1 \in \{0, 1\}^{q(n)+2}$ 。
- 3.4 假设 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$, 对 $k \in \{0, 1\}^n$, 算法 Enc_k 仅仅是为长度最多为 $\ell(n)$ (某个多项式 ℓ) 的消息定义的。构造一个即使当敌手在实验 $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(n)$ 中, 没有被限制要求输出等长的消息时仍然满足定义 3.8 的方案。
- 3.5 证明定义 3.8 和定义 3.9 是等价的。
- 3.6 令 G 为一个伪随机发生器, 其中 $|G(s)| > 2 \cdot |s|$ 。
① 定义 $G'(s) \stackrel{\text{def}}{=} G(s0^{|s|})$ 。 G' 必然是伪随机发生器吗?
② 定义 $G'(s) \stackrel{\text{def}}{=} G(s_1 \cdots s_{n/2})$, 其中 $s = s_1 \cdots s_n$ 。 G' 必然是伪随机发生器吗?
- 3.7 假设一个伪随机函数存在, 证明在窃听者存在的情况下, 存在一个加密方案, 该加密方案具备不可区分的多次加密 (根据定义 3.18), 但不是 CPA 安全的 (根据定义 3.21)。
提示: 该方案不必“自然而然”。将需要用到一个事实: 在选择明文攻击中, 敌手能够适应性地选择对加密预言机的问询。
- 3.8 无条件证明一个有效伪随机函数 $F: \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ 的存在性, 其中输入长度是安全参数的对数 (密钥的长度是安全参数的多项式)。
提示: 实现一个随机函数, 输入为对数输入长度。
- 3.9 从任意伪随机函数中, 表述一个变输出长度的伪随机发生器的构造方法。证明这个构造方法满足定义 3.17。
- 3.10 令 G 为伪随机发生器, 定义 $G'(s)$ 为 G 的输出, 并且被缩短到 n 比特 (其中 $|s| = n$)。证明函数 $F_k(x) = G'(k) \oplus x$ 不是伪随机的。
- 3.11 证明命题 3.27 (也就是说, 证明任意伪随机置换也是一个伪随机函数)。
提示: 说明在多项式时间内, 一个随机置换不能从一个随机函数中区分开来 (使用附录 A.4 的结论)。
- 3.12 通过自然而然地改写定义 3.12, 定义抵抗选择明文攻击的完善保密的概念。说明该定义是不能被实现的。

- 3.13 假设 F 是伪随机置换。说明存在一个函数 F' ，该函数是伪随机置换但非强伪随机置换。
提示：构造一个 F' ，使得 $F'(k) = 0^{|k|}$ 。
- 3.14 令 F 为伪随机置换，定义一个定长加密方案(Gen, Enc, Dec)如下：输入 $m \in \{0, 1\}^{n/2}$ ，密钥 $k \in \{0, 1\}^n$ ，算法Enc选择一个长度为 $n/2$ 随机字符串 $r \leftarrow \{0, 1\}^{n/2}$ ，并且计算 $c := F_k(r \| m)$ 。
说明如何解密，并证明该方案对长度为 $n/2$ 的消息是 CPA 安全的。（如果正寻找一个真正的挑战，证明在 F 是强伪随机置换的条件下，该方案是 CCA 安全的。）和构造方法 3.24 相比，该方案的优势和缺陷分别是什么？
- 3.15 令 F 为伪随机函数， G 是一个伪随机发生器，扩展因子 $\ell(n) = n + 1$ 。对下面每个加密方案，在窃听者存在的情况下，说明该方案是否具备不可区分加密，以及说明该方案是否是 CPA 安全的。在每一种情况下，共享密码都是随机的 $k \in \{0, 1\}^n$ 。
- ① 加密 $m \in \{0, 1\}^{2n+2}$ ，解析 m 为 $m_1 \| m_2$ ，其中 $|m_1| = |m_2|$ ，然后发送 $\langle G(k) \oplus m_1, G(k) \oplus m_2 \rangle$ 。
 - ② 加密 $m \in \{0, 1\}^{n+1}$ ，选择一个随机 $r \leftarrow \{0, 1\}^n$ ，然后发送 $\langle r, G(r) \oplus m \rangle$ 。
 - ③ 加密 $m \in \{0, 1\}^n$ ，发送 $m \oplus F_k(0^n)$ 。
 - ④ 加密 $m \in \{0, 1\}^{2n}$ ，解析 m 为 $m_1 \| m_2$ ，其中 $|m_1| = |m_2|$ ，然后随机选择 $r \leftarrow \{0, 1\}^n$ ，最后发送 $\langle r, m_1 \oplus F_k(r), m_2 \oplus F_k(r + 1) \rangle$ 。
- 3.16 考虑一个 CBC 加密模式的变体：每次当一个消息被加密时，发送方只是简单地对 IV 增加1（而不是每次随机选择 IV ）。说明该方案是非 CPA 安全的。
- 3.17 陈述已知的所有不同加密模式的解密准则。哪些模式的解密可以并行运算？
- 3.18 完成定理 3.29 的证明。
- 3.19 令 F 为伪随机函数，满足： $k \in \{0, 1\}^n$ ，函数 F_k 为 n 比特输入到 n 比特输出的映射。（该章已假设 $\ell_{in}(n) = \ell_{out}(n) = n$ 。）
- ① 考虑使用该形式的一个 F 来实现计数模式。对于 ℓ_{in}, ℓ_{out} 中的哪个函数来说，得出来的加密方式是 CPA 安全的？
 - ② 考虑使用上面的 F 来实现计数模式，但是只能处理固长 $\ell(n)$ （总是 $\ell_{out}(n)$ 的整数倍）的消息。对于 $\ell_{in}, \ell_{out}, \ell$ 中的哪个函数来说，得出来的加密方式是 CPA 安全的？在窃听者存在的情况下，对于 $\ell_{in}, \ell_{out}, \ell$ 中的哪个函数来说，该方案具备不可区分加密？
- 3.20 函数 $g: \{0, 1\}^n \leftarrow \{0, 1\}^n$ ，令 $g^*(\cdot)$ 为预言机：输入 1^n ，随机均匀地选择 $r \leftarrow \{0, 1\}^n$ ，并且返回 $(r, g(r))$ 。假设一个带密钥的函数 F 是一个弱伪随机函数，如果对所有的 PPT 算法 D ，存在一个可忽略函数 negl ，使得：

$$|\Pr[D^{F_k^*(\cdot)}(1^n) = 1] - \Pr[D^{f^*(\cdot)}(1^n) = 1]| \leq \text{negl}(n)$$

其中 $k \leftarrow \{0, 1\}^n$ ， $f \leftarrow \text{Func}_n$ 是随机均匀地选择的。

- ① 证明：如果 F 是伪随机的，那么它是弱伪随机。
- ② 令 F' 为伪随机函数，然后定义

$$F_k(x) = \begin{cases} F'_k(x) & \text{若 } x \text{ 是偶数} \\ F'_k(x+1) & \text{若 } x \text{ 是奇数} \end{cases}$$

证明 F 是弱伪随机，但不是伪随机。

③ 使用弱伪随机函数 F 的计数模式加密是 CPA 安全的吗？在窃听者存在的情况下，它是否必然具备不可区分加密？证明给出的答案。

④ 基于弱伪随机函数，构造一个 CPA 安全的加密方案。

提示：在本章中存在这样一个构造。

3.21 令 $\Pi_1 = (\text{Gen}_1, \text{Enc}_1, \text{Dec}_1)$ 和 $\Pi_2 = (\text{Gen}_2, \text{Enc}_2, \text{Dec}_2)$ 为两个加密方案，已知至少有一个是 CPA 安全的。问题是不知道哪个是 CPA 安全的，哪个不是。说明如何构造一个加密方案 Π 使得：只要 Π_1 和 Π_2 中至少有一个是 CPA 安全的，则 Π 一定是 CPA 安全的。试着对答案给出一个完整的证明。

提示：从原始明文中，生成两份明文消息，这样掌握了两者之一，都不会泄露明文的内容，但是都掌握了两者，就能够产生出原始的明文。

3.22 说明 CBC、OFB 以及计数模式的加密不是 CCA 安全的加密方案（无论 F 是什么）。